

Posts and Telecommunications Institute of Technology

Faculty of Information Technology



TIỂU LUẬN v2

Class : E22CNPM01

Group : 05

Student : Nguyễn Nhật Hưng - B22DCDT148

CHƯƠNG 1. Từ Monolithic đến Microservices và DDD.....	8
1.1 Giới thiệu Monolithic Architecture.....	8
1.1.1 Khái niệm.....	8
1.1.2 Cấu trúc điển hình.....	8
1.1.3 Ví dụ thực tế — Hệ thống E-Commerce Monolithic.....	8
1.1.5 Khi nào nên dùng Monolithic.....	9
1.2 Microservices Architecture.....	9
1.2.1 Khái niệm.....	9
1.2.2 Đặc điểm.....	10
1.2.3 So sánh Monolithic vs Microservices.....	10
1.2.5 Về Monolith.....	12
1.2.6 Nguyên tắc thiết kế.....	13
1.2.7. Ví dụ các công ty lớn dùng Microservices.....	13
1.2.8. Ví dụ các công ty lớn dùng Microservices.....	14
1.2.9. Từ Monolith sang Microservice.....	15
1.3 Domain-Driven Design (DDD).....	17
1.3.1 Mục tiêu.....	17
1.3.2 Các khái niệm cốt lõi.....	17
1.3.3 Context Map — ShopMe.....	18
1.3.4 DDD trong Microservices.....	19
1.4 Case Study: ShopMe — Decomposition theo DDD.....	20
1.4.1 Mô tả bài toán.....	20
1.4.2 Bước 1: Xác định Domain.....	20
1.4.3 Bước 2: Xác định Bounded Context.....	20
1.4.5 Bước 4: Xác định quan hệ giữa các Service.....	22
1.5 Case Study Healthcare.....	22
1.6 Kết luận Chương 1.....	31
CHƯƠNG 2. Phát triển Hệ thống ShopMe Microservices.....	32
2.1 Xác định yêu cầu.....	32
2.1.1 Functional Requirements.....	32
2.1.2 Non-functional Requirements.....	33
2.2 Phân rã hệ thống theo DDD.....	34
2.2.1 Bounded Context và Service Mapping.....	34
2.2.2 Nguyên tắc thiết kế.....	34
2.2.3. Luồng hoạt động theo ngôn ngữ tự nhiên.....	35
2.3 Thiết kế Product Services (Django).....	36
2.3.1 Phân loại sản phẩm.....	36
2.3.2 Model tổng quát.....	37

2.3.3 API Endpoints.....	38
2.4 Thiết kế User Service (Django).....	39
2.4.1 Phân loại người dùng và RBAC.....	39
2.4.2 Diagram.....	39
2.4.3 Đăng ký tài khoản.....	40
2.4.5 JWT Authentication Flow.....	42
2.4.6 API Endpoints.....	43
2.5 Thiết kế Cart Service.....	44
2.5.1 Model.....	44
2.5.3 Diagram.....	44
2.5.4 Thêm vào giỏ hàng.....	45
2.5.6 API.....	45
2.6 Thiết kế Order Service.....	47
2.6.1 Model và Workflow.....	47
2.6.3 Order Workflow.....	48
2.7 Thiết kế Payment Service.....	49
2.7.1 Model.....	49
2.7.2 Trạng thái Payment.....	49
2.7.2 Diagram.....	49
2.7.3. Tạo đơn và thanh toán.....	50
2.7.4. API.....	50
2.8 Thiết kế Shipping Service.....	51
2.8.1 Model và Trạng thái.....	51
2.8.2 Diagram.....	52
2.8.3. Shipping.....	52
2.8.3. API.....	53
2.9 Luồng hệ thống tổng thể (End-to-End)- sau đăng nhập.....	54
2.10 Thiết kế Database cho từng Service.....	55
2.10.1 Lựa chọn Database theo Service.....	55
2.10.2 So sánh MySQL vs PostgreSQL.....	56
2.10.3 Database Diagram — Core Services.....	57
2.11. Kết luận Chương 2.....	57
CHƯƠNG 3. AI Service cho Tư vấn Sản phẩm.....	58
3.1 Mục tiêu.....	58
3.3 Thu thập dữ liệu — User Behavior.....	59
3.3.1 Cấu trúc dữ liệu hành vi.....	59
3.4 Mô hình LSTM (Sequence Modeling).....	60
3.4.1 Ý tưởng.....	60

3.4.2 Model chi tiết.....	61
3.4.3 LSTM.....	65
3.5 Knowledge Graph với Neo4j.....	70
3.6 RAG — Retrieval-Augmented Generation.....	73
3.6.1 Pipeline.....	73
3.6.2 Vector Database — FAISS.....	74
3.7 Kết hợp thành model.....	77
3.8 UC-AI.....	78
3.8 API Endpoints của AI Service.....	79
3.9 Tech Stack và Triển khai.....	80
3.10 Kết luận Chương 3.....	81
CHƯƠNG 4. Xây dựng Hệ thống ShopMe Hoàn chỉnh.....	82
4.1 Kiến trúc tổng thể.....	82
4.1.1 System Overview.....	82
4.1.2 Architecture Diagram.....	83
4.2 API Gateway (Nginx).....	83
4.2.1 Vai trò.....	83
4.2.2 Cấu hình nginx.conf.....	84
4.3 Authentication — JWT- Luồng xác thực.....	89
4.4 Giao tiếp giữa các Service.....	89
4.5 Docker hóa hệ thống.....	89
4.5.1 Dockerfile — Django services.....	89
4.5.2 docker-compose.yml — Cấu trúc.....	89
4.6 Triển khai và Vận hành.....	100
4.6.1. Triển khai.....	100
Khởi động lần đầu.....	100
4.6.2. Kiểm tra.....	101
4.7 Minh chứng dự án.....	101
4.8 Đánh giá hệ thống.....	115
4.8.1 Hiệu năng.....	115
4.8.2 Khả năng mở rộng.....	115
4.8.4 Nhược điểm và Hướng khắc phục.....	115
4.9 Kết luận Chương 4.....	116
Chương 5. Kết luận.....	117
TÀI LIỆU THAM KHẢO. Tài liệu tham khảo.....	118

CHƯƠNG 1. Từ Monolithic đến Microservices và DDD

1.1 Giới thiệu Monolithic Architecture

1.1.1 Khái niệm

Monolithic Architecture là mô hình kiến trúc phần mềm trong đó toàn bộ các thành phần của hệ thống — bao gồm giao diện người dùng (UI), logic nghiệp vụ (Business Logic) và tầng truy cập dữ liệu (Data Access Layer) — được xây dựng, đóng gói và triển khai cùng nhau như một khối duy nhất (single deployable unit).

1.1.2 Cấu trúc điển hình

Một ứng dụng Monolithic điển hình bao gồm ba tầng chính:

- Presentation Layer (UI) — Tầng giao diện người dùng
- Business Logic Layer — Tầng xử lý nghiệp vụ
- Data Access Layer — Tầng truy cập cơ sở dữ liệu

1.1.3 Ví dụ thực tế — Hệ thống E-Commerce Monolithic

Trước khi chuyển sang kiến trúc Microservices, một hệ thống thương mại điện tử điển hình theo mô hình Monolithic sẽ bao gồm tất cả các module trong một codebase duy nhất:

- Quản lý sản phẩm (Product Management)
- Giỏ hàng (Cart)
- Đặt hàng và thanh toán (Order & Payment)
- Quản lý người dùng (User & Authentication)
- Giao hàng (Shipping)

Với ShopMe, nếu xây dựng theo Monolithic, toàn bộ 10 nhóm sản phẩm (laptop, mobile, tablet, clothes, shoes, book, food, beauty, furniture, sport), hệ thống AI gợi ý, module người dùng, giỏ hàng, đơn hàng, thanh toán và giao hàng sẽ nằm trong một project duy nhất.

1.1.4 Nhược điểm chi tiết

- Khó mở rộng (Hard to scale): Phải scale toàn bộ hệ thống thay vì từng module có tải cao
- Coupling cao: Một thay đổi nhỏ trong module Product có thể ảnh hưởng đến module Order hoặc User
- Rủi ro triển khai (Deploy risk): Một lỗi nhỏ trong feature mới có thể làm sập toàn hệ thống
- Khó phát triển nhóm (Team collaboration): Nhiều team cùng sửa một codebase dẫn đến xung đột
- Khó áp dụng công nghệ mới: Toàn bộ hệ thống phụ thuộc vào một tech stack duy nhất

1.1.5 Khi nào nên dùng Monolithic

- Hệ thống nhỏ, ít tính năng
- MVP (Minimum Viable Product) — phát triển nhanh để validate ý tưởng
- Team ít người (dưới 5 developer)
- Yêu cầu triển khai đơn giản, không cần scale phức tạp

1.2 Microservices Architecture

1.2.1 Khái niệm

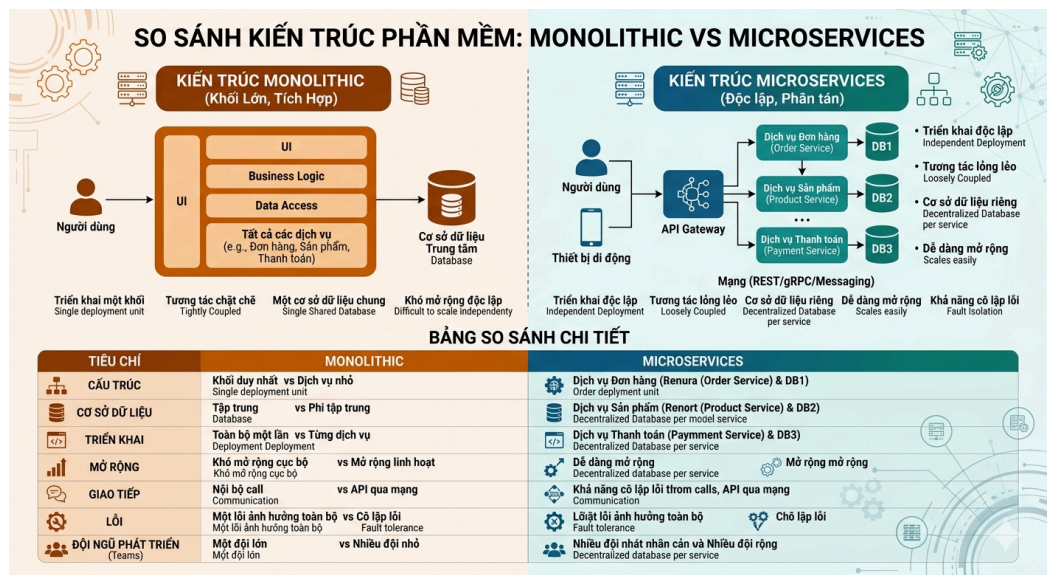
Microservices là một phong cách kiến trúc phần mềm trong đó hệ thống được chia thành các dịch vụ nhỏ, độc lập (loosely coupled services), mỗi service chịu trách nhiệm cho một chức năng nghiệp vụ cụ thể. Các service giao tiếp với nhau thông qua API (thường là REST/HTTP hoặc gRPC) và có thể được triển khai độc lập.

1.2.2 Đặc điểm

- Mỗi service có cơ sở dữ liệu riêng (Database per Service pattern)
- Giao tiếp qua API (REST/HTTP hoặc Message Queue)
- Deploy độc lập — có thể cập nhật một service mà không ảnh hưởng service khác

- Mỗi service có thể dùng ngôn ngữ, framework, database riêng phù hợp nhất

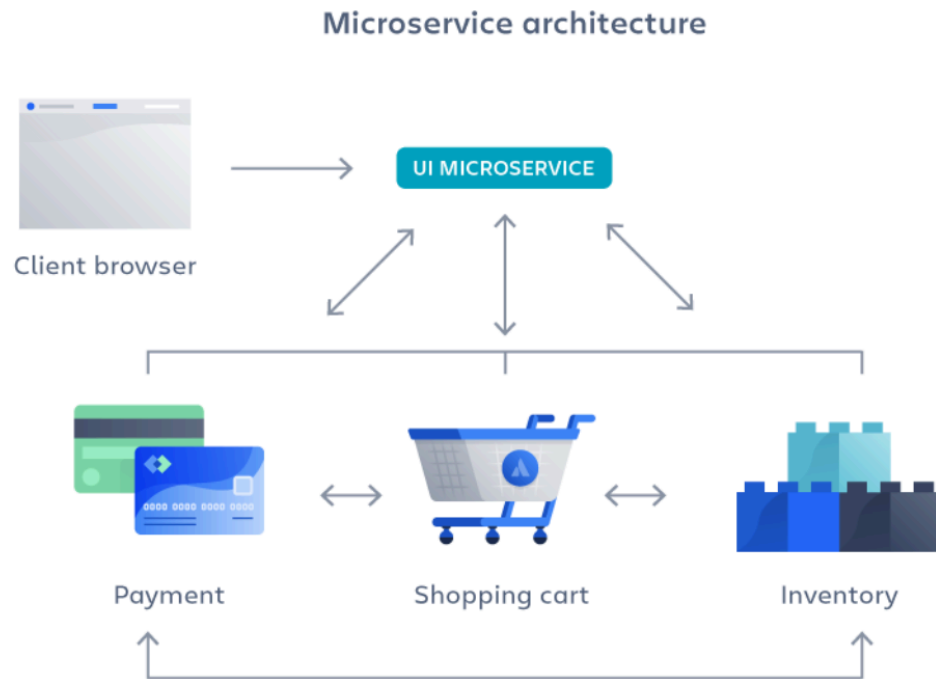
1.2.3 So sánh Monolithic vs Microservices



Tiêu chí	Monolithic	Microservices
Triển khai	Một lần (toàn hệ thống)	Độc lập từng service
Scale	Toàn bộ hệ thống	Từng service riêng lẻ
Coupling	Cao — phụ thuộc chặt	Thấp — loosely coupled
Team	Một team, một codebase	Nhiều team độc lập
Công nghệ	Một tech stack duy nhất	Đa dạng theo service
Độ phức tạp vận hành	Thấp	Cao — cần DevOps tốt

Bảng 1.1: So sánh Monolithic và Microservices

1.2.4 Về Microservices



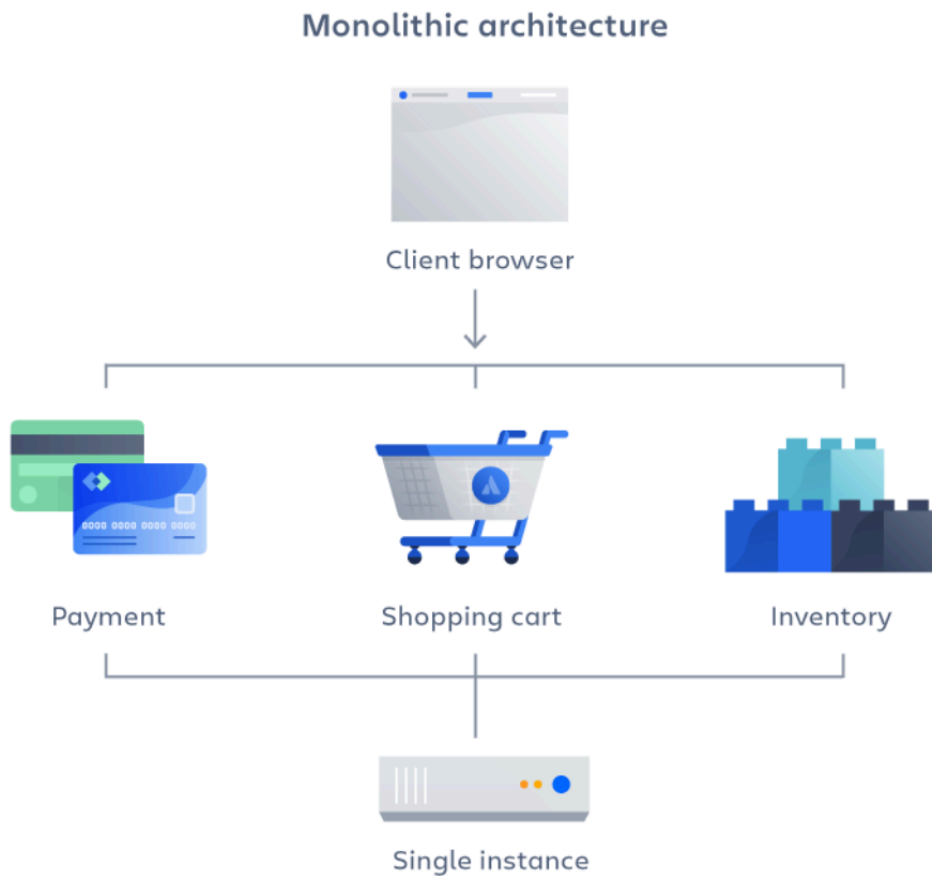
Ưu điểm:

- Scale độc lập từng service theo nhu cầu tải
- Tăng tốc độ phát triển — các team làm việc song song
- Fault isolation — lỗi trong một service không làm sập toàn hệ
- Phù hợp hệ thống lớn, phức tạp với nhiều domain nghiệp vụ
- Dễ áp dụng công nghệ mới cho từng service

Nhược điểm:

- Phức tạp hơn khi phát triển và vận hành hệ thống phân tán
- Khó debug — trace request qua nhiều service
- Cần infrastructure tốt (Docker, Kubernetes, service discovery)
- Độ trễ mạng trong giao tiếp giữa các service

1.2.5 Về Monolith



Ưu điểm

- Đơn giản trong phát triển — toàn bộ code nằm trong một project
- Dễ debug — không cần trace qua nhiều service
- Triển khai dễ dàng — chỉ cần build và deploy một ứng dụng
- Hiệu năng tốt hơn (gọi hàm nội bộ, không qua network)
- Phù hợp với hệ thống nhỏ hoặc giai đoạn MVP

Nhược điểm:

- Khó scale — phải scale toàn bộ hệ thống, không scale riêng từng phần
- Khó bảo trì khi hệ thống lớn dần (codebase phình to)
- Các team dễ bị phụ thuộc lẫn nhau (conflict code, merge)

- Khó áp dụng công nghệ mới — thay đổi ảnh hưởng toàn hệ thống
- Một lỗi có thể ảnh hưởng toàn bộ ứng dụng (không có fault isolation tốt)

1.2.6 Nguyên tắc thiết kế

Microservice:

- Single Responsibility — mỗi service chỉ làm một việc
- Loose Coupling — service tương tác qua API, không phụ thuộc nội bộ
- High Cohesion — các chức năng liên quan đến nhau nằm trong cùng service
- Database per Service — mỗi service quản lý dữ liệu riêng

Monolith:

- Shared Responsibility — nhiều chức năng nằm chung trong một ứng dụng
- Tight Coupling — các module phụ thuộc lẫn nhau trong cùng codebase
- Lower Cohesion — dễ bị trộn logic giữa các domain nếu không thiết kế tốt
- Shared Database — toàn bộ hệ thống dùng chung một database

1.2.7. Ví dụ các công ty lớn dùng Microservices

1. Netflix

- Ban đầu Netflix dùng kiến trúc monolith.
- Khi lượng người dùng tăng cực mạnh, họ chuyển sang microservices trên cloud.
- Hệ thống được chia thành hàng trăm service nhỏ:
 - recommendation service
 - billing service
 - streaming service
 - authentication service
- Lợi ích:
 - deploy từng phần riêng
 - scale độc lập
 - nếu một service lỗi không làm sập toàn hệ thống

2. Amazon

- Amazon là case nổi tiếng về microservices.
- Mỗi chức năng là một service riêng:
 - giỏ hàng
 - thanh toán
 - tìm kiếm
 - quản lý kho
 - gợi ý sản phẩm
- Các team phát triển độc lập từng service.
- Đây là lý do Amazon có thể update hệ thống liên tục mà không downtime lớn.

1.2.8. Ví dụ các công ty lớn dùng Microservices

1. GitHub

- GitHub nổi tiếng vẫn giữ “modular monolith” trong thời gian dài.
- Dù rất lớn nhưng phần lớn backend vẫn nằm trong một ứng dụng Rails khổng lồ.
- Họ chọn monolith vì:
 - code dễ trace
 - debug đơn giản
 - transaction/database dễ quản lý
 - developer onboarding nhanh hơn

2. Shopify

- Shopify phục vụ hàng triệu cửa hàng nhưng core system vẫn thiên về monolith.
- Họ dùng “Monolith First”.
- Lý do:
 - giảm độ phức tạp distributed system
 - dễ maintain hơn microservices quá sớm
 - tốc độ phát triển feature nhanh

1.2.9. Từ Monolith sang Microservice

Giai đoạn 1: Khởi đầu với Monolith

Ở giai đoạn đầu, đa số doanh nghiệp công nghệ đều xây dựng hệ thống dưới dạng Monolith. Toàn bộ chức năng như đăng nhập, quản lý sản phẩm, giỏ hàng, thanh toán hay vận chuyển đều nằm trong cùng một ứng dụng và sử dụng chung cơ sở dữ liệu. Kiến trúc này giúp phát triển nhanh, dễ triển khai và phù hợp với các startup hoặc dự án mới vì đội ngũ phát triển còn nhỏ.

Tuy nhiên, khi hệ thống còn đơn giản thì Monolith hoạt động rất hiệu quả, nhưng theo thời gian, codebase bắt đầu lớn dần và trở nên khó quản lý hơn.

Giai đoạn 2: Hệ thống tăng trưởng và xuất hiện vấn đề

Khi số lượng người dùng tăng mạnh, doanh nghiệp cần bổ sung thêm tính năng và mở rộng đội ngũ phát triển. Lúc này Monolith bắt đầu bộc lộ nhiều hạn chế. Chỉ một thay đổi nhỏ cũng có thể yêu cầu build và deploy lại toàn bộ hệ thống. Nếu một module xảy ra lỗi, toàn bộ ứng dụng có nguy cơ bị ảnh hưởng.

Ngoài ra, nhiều nhóm phát triển cùng làm việc trên một codebase lớn dễ gây xung đột source code, làm giảm tốc độ phát triển sản phẩm. Việc scale cũng trở nên khó khăn vì muốn tăng hiệu năng cho một chức năng nhỏ thì phải scale toàn bộ ứng dụng.

Giai đoạn 3: Modular Monolith

Để giảm độ phức tạp, doanh nghiệp thường chưa chuyển ngay sang Microservices mà sẽ tổ chức lại hệ thống thành các module độc lập bên trong Monolith. Ví dụ: module User, Product, Order hay Payment được tách logic rõ ràng hơn nhưng vẫn chạy trong cùng một ứng dụng.

Giai đoạn này giúp chuẩn hóa kiến trúc, giảm sự phụ thuộc giữa các phần của hệ thống và chuẩn bị cho việc tách service sau này. Đây được xem là bước chuyển tiếp quan trọng vì nếu Monolith ban đầu quá lộn xộn thì việc tách Microservices sẽ rất khó khăn.

Giai đoạn 4: Tách các service đầu tiên

Sau khi hệ thống đã được modular hóa, doanh nghiệp bắt đầu tách các thành phần ít phụ thuộc ra thành service riêng. Những service thường được tách đầu tiên là:

- Authentication Service
- Payment Service
- Notification Service
- Search Service

Mỗi service có API riêng và có thể được deploy độc lập. Điều này giúp các nhóm phát triển làm việc song song mà không ảnh hưởng trực tiếp đến nhau.

Ví dụ, nếu Payment Service cần cập nhật tính năng mới, đội phát triển có thể deploy riêng service đó mà không cần dừng toàn bộ hệ thống.

Giai đoạn 5: Giao tiếp giữa các Microservices

Khi số lượng service tăng lên, hệ thống cần cơ chế giao tiếp hiệu quả hơn. Các service bắt đầu trao đổi dữ liệu thông qua:

- REST API
- gRPC
- Message Queue như RabbitMQ hoặc Kafka

Ví dụ, khi người dùng tạo đơn hàng:

1. Order Service xử lý đơn hàng.
2. Payment Service xử lý thanh toán.
3. Sau khi thanh toán thành công, Shipping Service nhận event để tạo vận chuyển.

Hệ thống lúc này hoạt động theo kiểu phân tán thay vì tập trung như Monolith trước đây.

Giai đoạn 6: Hoàn thiện hệ sinh thái Microservices

Khi kiến trúc Microservices mở rộng, doanh nghiệp cần bổ sung thêm nhiều công nghệ hỗ trợ như:

- API Gateway
- Service Discovery
- Docker và Kubernetes
- CI/CD Pipeline
- Logging tập trung
- Monitoring và Tracing

1.3 Domain-Driven Design (DDD)

1.3.1 Mục tiêu

Domain-Driven Design (DDD) là phương pháp thiết kế phần mềm tập trung vào việc mô hình hóa hệ thống theo nghiệp vụ kinh doanh (business domain), giúp đội phát triển và chuyên gia nghiệp vụ có chung ngôn ngữ (Ubiquitous Language) và cách hiểu về hệ thống.

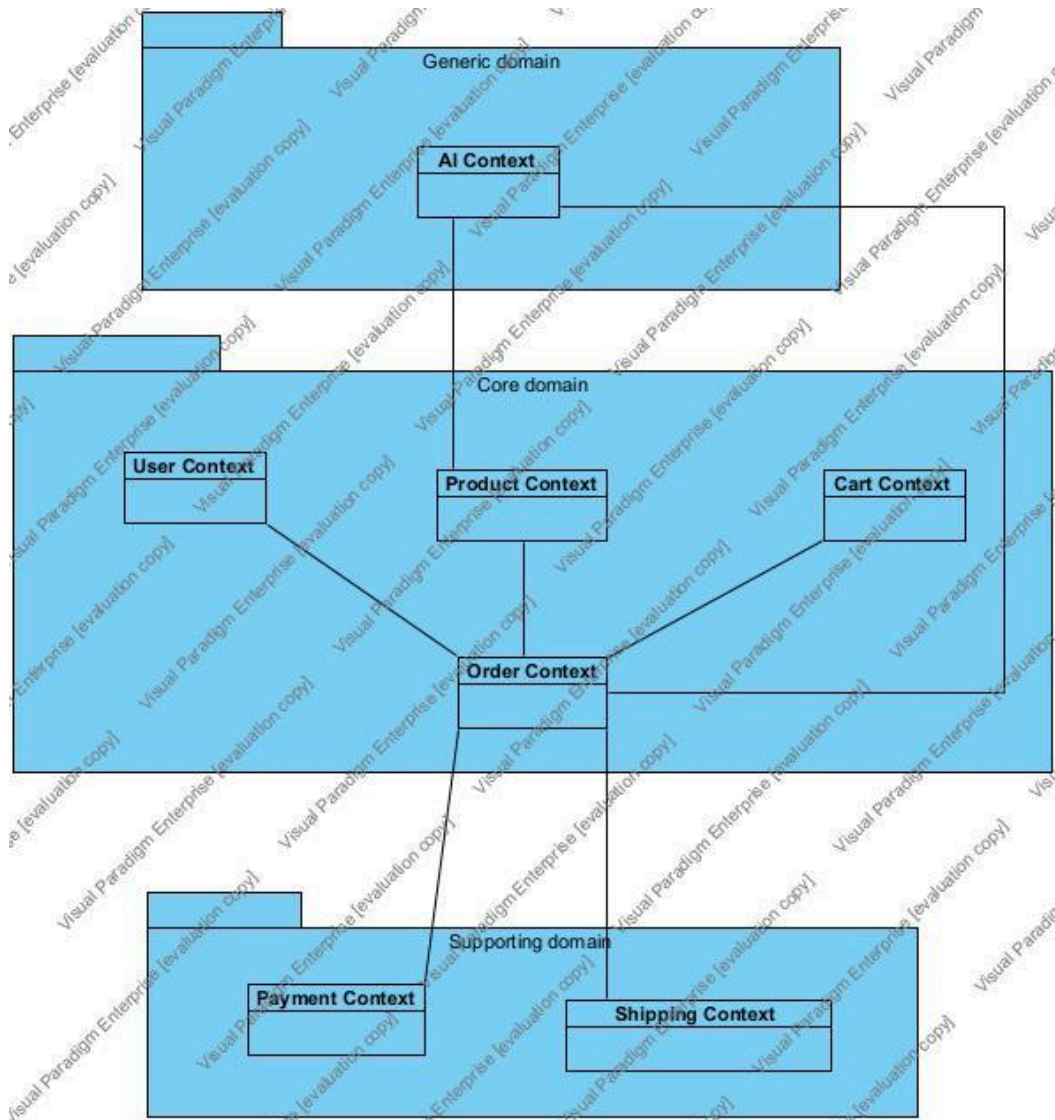
1.3.2 Các khái niệm cốt lõi

DDD bao gồm các khái niệm nền tảng sau:

- Entity: Đối tượng có định danh duy nhất (ID). Ví dụ: User, Product, Order trong ShopMe
- Value Object: Đối tượng không có ID, so sánh bằng giá trị. Ví dụ: Address, Money, Price
- Aggregate: Nhóm các Entity và Value Object liên quan. Ví dụ: Order + OrderItem
- Bounded Context: Ranh giới domain rõ ràng, xác định phạm vi áp dụng của model
- Ubiquitous Language: Ngôn ngữ chung giữa developer và domain expert

1.3.3 Context Map — ShopMe

Context Map thể hiện mối quan hệ giữa các Bounded Context trong hệ thống ShopMe. Trong đó mỗi bounded context được ánh xạ thành một microservice độc lập:



Hình 1.1: Context Map của hệ thống ShopMe

1.3.4 DDD trong Microservices

- Mỗi Bounded Context = 1 Microservice độc lập
- Tránh chia service theo technical layer (không chia theo Controller/Service/Repository)
- Ranh giới service được xác định bởi nghiệp vụ, không phải công nghệ

1.4 Case Study: ShopMe — Decomposition theo DDD

1.4.1 Mô tả bài toán

ShopMe là hệ thống thương mại điện tử đa danh mục sản phẩm, cho phép người dùng duyệt và mua sắm sản phẩm từ 10 nhóm danh mục khác nhau, với hệ thống AI gợi ý sản phẩm và chatbot tư vấn tích hợp. Hệ thống cần hỗ trợ nhiều team phát triển độc lập và scale linh hoạt theo nhu cầu từng module.

1.4.2 Bước 1: Xác định Domain

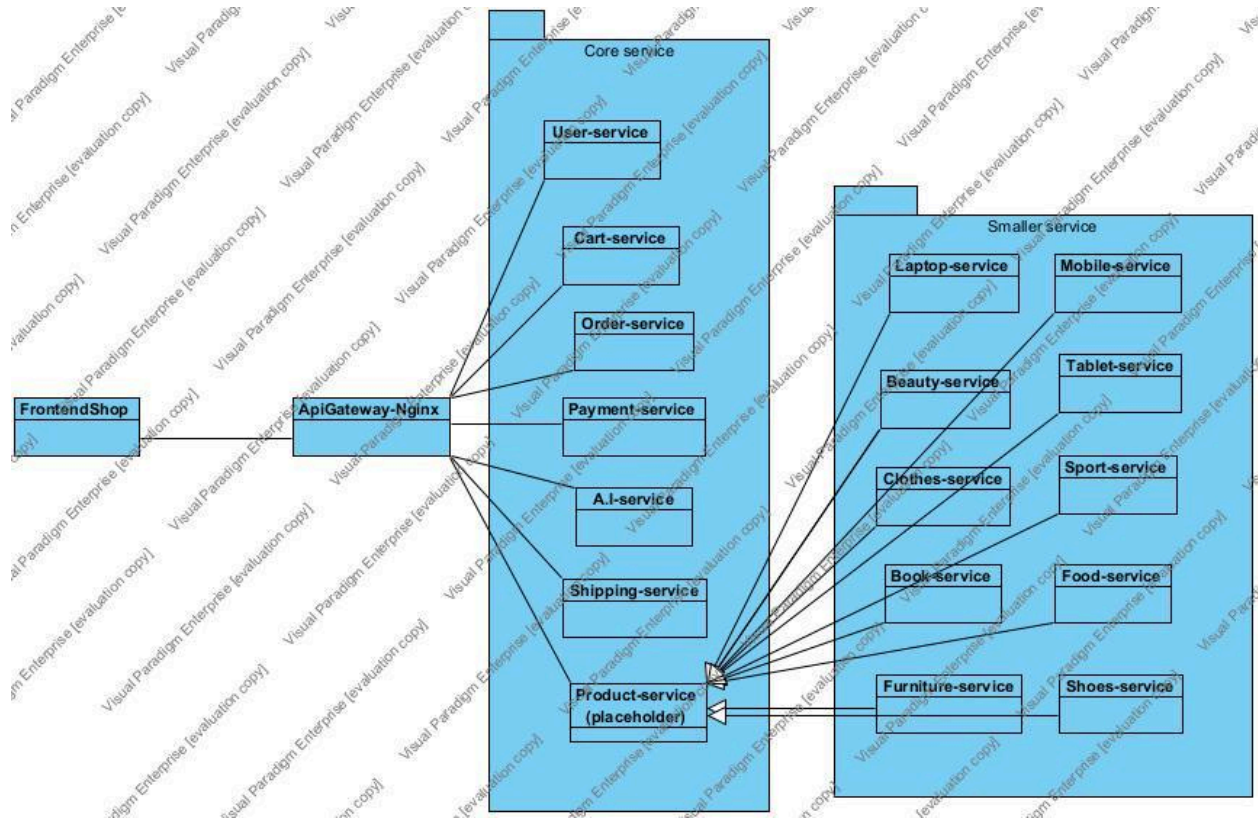
- Product Domain — quản lý 10 nhóm sản phẩm, mỗi sản phẩm là 1 service (laptop, mobile, tablet, clothes, shoes, book, food, beauty, furniture, sport)
- User Domain — quản lý người dùng (admin, staff, customer) và xác thực JWT
- Cart Domain — quản lý giỏ hàng người dùng
- Order Domain — quản lý đặt hàng và vòng đời đơn hàng
- Payment Domain — xử lý thanh toán
- Shipping Domain — quản lý giao hàng
- AI Domain — gợi ý sản phẩm và chatbot tư vấn

1.4.3 Bước 2: Xác định Bounded Context

Bounded Context	Trách nhiệm chính	Aggregates
User Context	Xác thực, phân quyền RBAC	User, Role, Permission
Product Context (×10)	Danh mục sản phẩm, tồn kho	Product, Category
Cart Context	Giỏ hàng người dùng	Cart, CartItem
Order Context	Đặt hàng, vòng đời đơn	Order, OrderItem
Payment Context	Xử lý giao dịch thanh toán	Payment
Shipping Context	Quản lý vận chuyển	Shipment
AI Context	Gợi ý, chatbot, recommendation	RecommendationRun, ChatSession

Bảng 1.2: Bounded Contexts của ShopMe

1.4.4 Bước 3: Phân rã thành Microservices



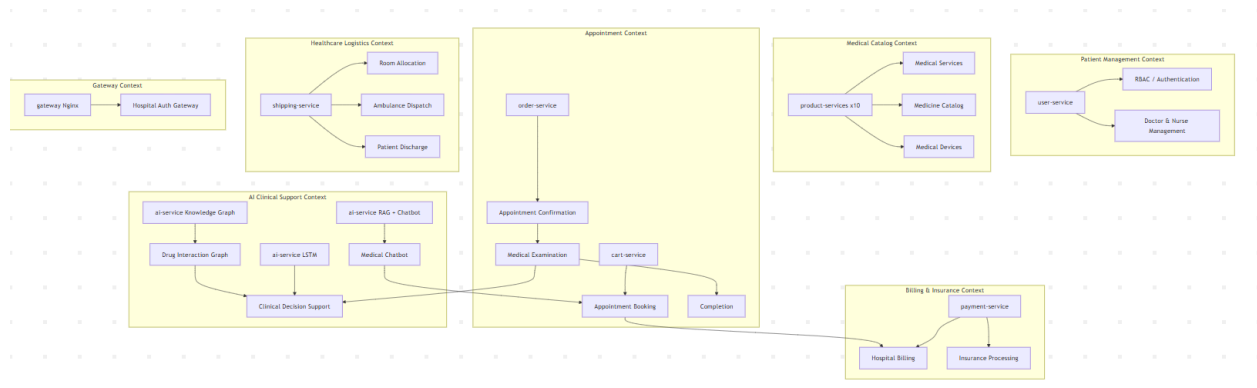
Hình 1.2: Phân rã ShopMe thành các Microservice

1.4.5 Bước 4: Xác định quan hệ giữa các Service

- Cart Service phụ thuộc Product Services (đọc thông tin sản phẩm khi thêm vào giỏ)
- Order Service phụ thuộc Cart Service (tạo đơn từ giỏ hàng)
- Order Service gọi Payment Service sau khi xác nhận đơn
- Payment Service thành công → kích hoạt Shipping Service
- AI Service đọc behavior log từ các tương tác người dùng
- Tất cả giao tiếp qua REST API thông qua API Gateway

1.5 Case Study Healthcare

ShopMe Service	Bounded Context gốc	Healthcare tương đương	Bounded Context mới
user-service	Xác thực, phân quyền RBAC	Patient Portal	Quản lý bệnh nhân, bác sĩ, y tá
product-services (×10)	Danh mục sản phẩm theo ngành	Medical Catalog	Danh mục dịch vụ khám, thuốc, thiết bị
cart-service	Giỏ hàng tạm thời	Appointment Basket	Chọn lịch hẹn và dịch vụ trước khi xác nhận
order-service	Vòng đời đơn hàng	Appointment Service	Vòng đời lịch hẹn: đặt → xác nhận → khám → hoàn thành
payment-service	Xử lý giao dịch tài chính	Billing Service	Hóa đơn viện phí, bảo hiểm y tế
shipping-service	Quản lý vận chuyển vật lý	Logistics Service	Điều phối phòng khám, xe cấp cứu, xuất viện
ai-service (LSTM)	Gợi ý sản phẩm theo chuỗi hành vi	Clinical Decision Support	Gợi ý phác đồ điều trị dựa trên lịch sử bệnh án
ai-service (RAG + Chatbot)	Tư vấn sản phẩm qua LLM	Medical Chatbot	Hỏi đáp triệu chứng, hướng dẫn dùng thuốc
ai-service (Knowledge Graph)	Quan hệ sản phẩm-người dùng	Drug Interaction Graph	Quan hệ thuốc-bệnh-tương tác
gateway (Nginx)	API Gateway duy nhất	Auth Gateway	Cổng xác thực bệnh viện



Áp dụng DDD, hệ thống Healthcare được phân rã thành 6 microservice độc lập:

Browser / Mobile App

|



Ngix Gateway :80

|

- | — /auth /users → patient-service :8000 (Django + MySQL)
- | — /appointments → appointment-service :8001 (Django + PostgreSQL)
- | — /billing → billing-service :8002 (Django + PostgreSQL)
- | — /prescriptions → prescription-service :8003 (Django + PostgreSQL)
- | — /catalog → catalog-service :8004 (Django + PostgreSQL)
- | — /ai → medical-ai-service :8005 (FastAPI + MongoDB)

Mỗi service sở hữu database riêng, không service nào truy cập trực tiếp DB của service khác — đúng nguyên tắc **Database per Service**.

Tạo Service với Django:

```
PS D:\shopme> mkdir appointment-service && cd appointment-service
>> python -m venv venv && source venv/bin/activate
>> pip install django djangorestframework psycopg2-binary
>>
>> django-admin startproject config .
>> python manage.py startapp appointments
```

File sau khi được tạo:

appointment-service/

├── config/

| ├── settings.py

| ├── urls.py

| └── wsgi.py

├── appointments/

| ├── models.py

| ├── views.py

| ├── serializers.py

| └── urls.py

├── manage.py

└── requirements.txt

Cấu hình setting.py:

```
# config/settings.py
import os

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.environ.get('DB_NAME', 'healthcare_appointments'),
        'USER': os.environ.get('DB_USER', 'healthcare'),
        'PASSWORD': os.environ.get('DB_PASSWORD', 'healthcare123'),
        'HOST': os.environ.get('DB_HOST', 'db-postgres'),
        'PORT': '5432',
    }
}

INSTALLED_APPS = [
    'django.contrib.contenttypes',
    'django.contrib.auth',
    'rest_framework',
    'appointments',
]

# URL các service liên quan (giao tiếp nội bộ)
PATIENT_SERVICE_URL = os.environ.get('PATIENT_SERVICE_URL', 'http://patient-service:8000')
CATALOG_SERVICE_URL = os.environ.get('CATALOG_SERVICE_URL', 'http://catalog-service:8004')
BILLING_SERVICE_URL = os.environ.get('BILLING_SERVICE_URL', 'http://billing-service:8002')
```

Định nghĩa Model:

```
from django.db import models

class Appointment(models.Model):
    STATUS_CHOICES = [
        ('pending', 'Chờ xác nhận'),
        ('confirmed', 'Đã xác nhận'),
        ('in_progress', 'Đang khám'),
        ('completed', 'Hoàn thành'),
        ('cancelled', 'Đã hủy'),
    ]

    patient_id = models.IntegerField() # FK logic đến patient-service
    doctor_id = models.IntegerField(null=True) # FK logic đến catalog-service
    patient_name = models.CharField(max_length=255)
    phone = models.CharField(max_length=20)
    appointment_date = models.DateTimeField()
    symptoms = models.TextField(blank=True) # Mô tả triệu chứng
    status = models.CharField(
        max_length=20,
        choices=STATUS_CHOICES,
        default='pending'
    )

    total_fee = models.DecimalField(max_digits=10, decimal_places=2, default=0)
    note = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'appointments'
        ordering = ['-created_at']

    def __str__(self):
        return f"Appointment #{self.id} - {self.patient_name} - {self.status}"

class AppointmentService(models.Model):
    """Các dịch vụ được chọn trong một lịch hẹn (xét nghiệm, siêu âm, v.v.)"""
    appointment = models.ForeignKey(
        Appointment,
        on_delete=models.CASCADE,
        related_name='services'
    )

    service_code = models.CharField(max_length=50) # mã dịch vụ từ catalog-service
    service_name = models.CharField(max_length=255)
    quantity = models.IntegerField(default=1)
    unit_fee = models.DecimalField(max_digits=10, decimal_places=2)

    class Meta:
        db_table = 'appointment_services'

    @property
    def subtotal(self):
        return self.unit_fee * self.quantity
```

Serializer:

```
from rest_framework import serializers
from .models import Appointment, AppointmentService

class AppointmentServiceSerializer(serializers.ModelSerializer):
    subtotal = serializers.ReadOnlyField()

    class Meta:
        model = AppointmentService
        fields = ['id', 'service_code', 'service_name', 'quantity', 'unit_fee', 'subtotal']

class AppointmentSerializer(serializers.ModelSerializer):
    services = AppointmentServiceSerializer(many=True, read_only=True)

    class Meta:
        model = Appointment
        fields = [
            'id', 'patient_id', 'doctor_id', 'patient_name', 'phone',
            'appointment_date', 'symptoms', 'status',
            'total_fee', 'note', 'services', 'created_at',
        ]
]
```

Views:

```
import requests
from django.conf import settings
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
from .models import Appointment, AppointmentService
from .serializers import AppointmentSerializer

class AppointmentListView(APIView):

    def get(self, request):
        """Lấy danh sách lịch hẹn, lọc theo patient_id nếu có."""
        patient_id = request.query_params.get('patient_id')
        qs = Appointment.objects.all()
        if patient_id:
            qs = qs.filter(patient_id=patient_id)
        return Response(AppointmentSerializer(qs, many=True).data)

    def post(self, request):
        """Đặt lịch hẹn mới – tương đương tạo Order trong ShopMe."""
        data = request.data
        services = data.get('services', [])

        if not services:
            return Response({'error': 'Phải chọn ít nhất một dịch vụ'}, status=400)

        total_fee = sum(
            float(s.get('unit_fee', 0)) * int(s.get('quantity', 1))
            for s in services
        )
```

```

appointment = Appointment.objects.create(
    patient_id      = data.get('patient_id'),
    doctor_id       = data.get('doctor_id'),
    patient_name    = data.get('patient_name', ''),
    phone           = data.get('phone', ''),
    appointment_date = data.get('appointment_date'),
    symptoms        = data.get('symptoms', ''),
    note            = data.get('note', ''),
    total_fee       = total_fee,
    status          = 'pending',
)
for s in services:
    AppointmentService.objects.create(
        appointment = appointment,
        service_code = s['service_code'],
        service_name = s.get('service_name', ''),
        quantity     = int(s.get('quantity', 1)),
        unit_fee     = float(s.get('unit_fee', 0)),
    )
try:
    requests.post(
        f"{settings.BILLING_SERVICE_URL}/billing/create",
        json={
            'appointment_id': appointment.id,
            'amount': total_fee,
            'patient_id': appointment.patient_id,
        },
        timeout=5
    )
except Exception:
    pass # Billing xử lý async, không block lịch hẹn
return Response(AppointmentSerializer(appointment).data, status=201)

```

```

class AppointmentDetailView(APIView):

    def get(self, request, pk):
        try:
            appt = Appointment.objects.get(pk=pk)
            return Response(AppointmentSerializer(appt).data)
        except Appointment.DoesNotExist:
            return Response({'error': 'Không tìm thấy lịch hẹn'}, status=404)

    def put(self, request, pk):
        """Cập nhật trạng thái lịch hẹn (bác sĩ / lễ tân thực hiện)."""
        try:
            appt = Appointment.objects.get(pk=pk)
            new_status = request.data.get('status')
            if new_status:
                appt.status = new_status
                appt.save()
            return Response(AppointmentSerializer(appt).data)
        except Appointment.DoesNotExist:
            return Response({'error': 'Không tìm thấy lịch hẹn'}, status=404)

```

Routing:

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.AppointmentListView.as_view()), # GET list / POST create
    path('<int:pk>', views.AppointmentDetailView.as_view()), # GET detail / PUT update
]

# config/urls.py
from django.urls import path, include

urlpatterns = [
    path('appointments/', include('appointments.urls')),
    path('health', lambda r, **kw: __import__('rest_framework').response.Response({'status': 'ok'})),
]
```

Dockerfile:

```
FROM python:3.12-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

CMD ["sh", "-c", "python manage.py migrate && python manage.py runserver 0.0.0.0:8001"]
```

Requirement.txt:

```
# requirements.txt
django==4.2
djangorestframework==3.15
psycpg2-binary==2.9
requests==2.31
```

DockerCompose registering:

```
appointment-service:
  build: ./appointment-service
  ports:
    - "8001:8001"
  environment:
    DB_HOST:      db-postgres
    DB_NAME:      healthcare_appointments
    DB_USER:      healthcare
    DB_PASSWORD:  healthcare123
    BILLING_SERVICE_URL: http://billing-service:8002
  depends_on:
    - db-postgres
```

1.6 Kết luận Chương 1

Monolithic phù hợp cho các hệ thống nhỏ và MVP, nhưng khi quy mô tăng lên như ShopMe với 10 danh mục sản phẩm, AI service, và nhiều module nghiệp vụ phức tạp, Microservices là lựa chọn tất yếu. DDD cung cấp nền tảng lý thuyết vững chắc để phân rã hệ thống theo đúng ranh giới nghiệp vụ, tránh chia service theo tầng kỹ thuật. Kết quả là ShopMe được thiết kế với các service độc lập, sở hữu database riêng và giao tiếp qua REST API.

CHƯƠNG 2. Phát triển Hệ thống ShopMe Microservices

2.1 Xác định yêu cầu

2.1.1 Functional Requirements

#	Chức năng	Service đảm nhận
1	Đăng ký, đăng nhập, phân quyền RBAC (admin/staff/customer)	user-service
2	Quản lý 10 nhóm sản phẩm (laptop, mobile, tablet, clothes, shoes, book, food, beauty, furniture, sport)	10 services sản phẩm
3	Xem danh sách, chi tiết sản phẩm theo danh mục, kiểm tra tồn kho	product services
4	Thêm/xóa/cập nhật giỏ hàng	cart-service
5	Đặt hàng, xem lịch sử đơn hàng	order-service
6	Xử lý thanh toán (COD, banking)	payment-service
7	Quản lý trạng thái giao hàng	shipping-service
8	Gợi ý sản phẩm cá nhân hóa (LSTM + Graph + RAG)	ai-service

9	Chatbot tư vấn sản phẩm tích hợp Ollama LLM	ai-service
10	Tìm kiếm sản phẩm cross-service	ai-service

Bảng 2.1: Functional Requirements của ShopMe

2.1.2 Non-functional Requirements

Yêu cầu	Mô tả chi tiết
Scalability	Scale độc lập từng service. Product services có thể scale riêng khi traffic cao. AI service scale theo nhu cầu inference.
High Availability	Engine của AI luôn trả về kết quả kể cả khi Neo4j hoặc MongoDB không khả dụng. Fallback về cosine search khi FAISS không cài được.
Security	JWT authentication qua user-service. API Gateway kiểm tra token trước khi forward request. Không chia sẻ secret key giữa các service.
Performance	API Gateway (Nginx) proxy với timeout hợp lý. AI service cache FAISS index và LSTM model khi startup để tránh load lại mỗi request.
Maintainability	Mỗi service có Dockerfile riêng, deploy độc lập. Database migration tách biệt theo service. Seed data chạy tự động khi startup.

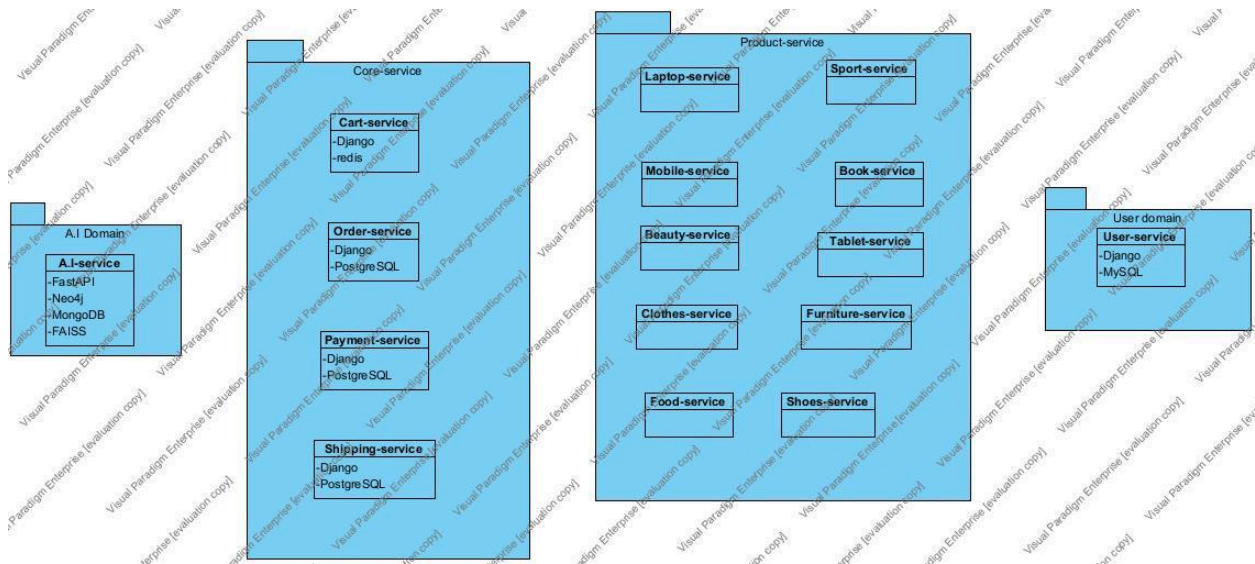
Observability

Log tập trung qua Docker Compose. Health endpoint /health trên mỗi service.

Bảng 2.2: Non-functional Requirements

2.2 Phân rã hệ thống theo DDD

2.2.1 Bounded Context và Service Mapping



Hình 2.1: Bounded Context Map và Database per Service

2.2.2 Nguyên tắc thiết kế

- Mỗi Bounded Context có database riêng — không service nào truy cập DB của service khác
- Giao tiếp qua REST API — không gọi trực tiếp vào internal layer của service khác
- Product services dùng cùng codebase template nhưng deploy độc lập với database riêng
- AI service dùng đa database: Neo4j (knowledge graph), MongoDB (behavior log), FAISS (vector search)

2.2.3. Luồng hoạt động theo ngôn ngữ tự nhiên:

Giai đoạn Đăng ký & Đăng nhập: UserRegistered → UserLoggedIn → JWTTokenIssued

Giai đoạn Khám phá sản phẩm: ProductCatalogViewed → ProductSearched → ProductDetailViewed → ProductAddedToWishlist

Giai đoạn Giỏ hàng: CartItemAdded → CartItemUpdated → CartItemRemoved → CouponApplied → CartAbandoned

Giai đoạn Đặt hàng: OrderPlaced → StockReserved → OrderConfirmed → OrderCancelled

Giai đoạn Thanh toán: PaymentInitiated → PaymentCompleted → PaymentFailed → RefundProcessed

Giai đoạn Giao hàng: ShipmentCreated → ShipmentDispatched → OrderDelivered → DeliveryFailed

Giai đoạn AI: BehaviorLogged → RecommendationGenerated → ChatSessionStarted → ChatResponseGenerated

2.3 Thiết kế Product Services (Django)

2.3.1 Phân loại sản phẩm

ShopMe có 10 nhóm danh mục sản phẩm, mỗi nhóm chạy như một microservice Django độc lập với database PostgreSQL riêng. Cùng một data model tổng quát nhưng seed data khác nhau:

Service	Danh mục	Port	Database	Số SP
laptop-service	Máy tính xách tay	8010	shopme_laptops	8
mobile-service	Điện thoại di động	8011	shopme_mobiles	8
tablet-service	Máy tính bảng	8012	shopme_tablets	6
clothes-service	Thời trang	8013	shopme_clothes	8
shoes-service	Giày dép	8014	shopme_shoes	8
book-service	Sách	8015	shopme_books	8
food-service	Thực phẩm	8016	shopme_foods	8
beauty-service	Làm đẹp	8017	shopme_beauty	8

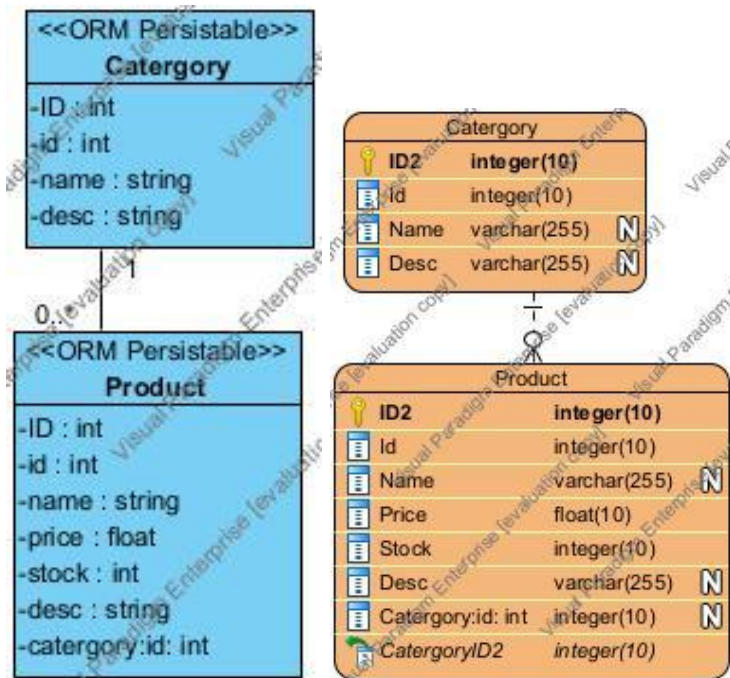
furniture-service	Nội thất	8018	shopme_furniture	8
sport-service	Thể thao	8019	shopme_sport	8

Bảng 2.3: Danh sách Product Services

2.3.2 Model tổng quát

```
from django.db import models
class Product(models.Model):
    name = models.CharField(max_length=255)
    price = models.FloatField()
    old_price = models.FloatField(null=True, blank=True)
    stock = models.IntegerField(default=0)
    rating = models.FloatField(default=0)
    reviews_count = models.IntegerField(default=0)
    description = models.TextField(blank=True)
```

```
    badge = models.CharField(max_length=30, blank=True)
    is_active = models.BooleanField(default=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)
    material = models.CharField(max_length=100, blank=True)
    color = models.CharField(max_length=50, blank=True)
    width_cm = models.FloatField(null=True, blank=True)
    height_cm = models.FloatField(null=True, blank=True)
    weight_kg = models.FloatField(null=True, blank=True)
    room_type = models.CharField(max_length=50, blank=True)
    class Meta:
        db_table = 'products'
        ordering = ['-created_at']
    def __str__(self): return self.name
```



2.3.3 API Endpoints

Method	Endpoint	Mô tả
GET	/<named-product>/	Danh sách tất cả sản phẩm(laptop,...)
GET	/<named-product>/{id}/	Chi tiết sản phẩm
GET	/<named-product>/featured/	Sản phẩm nổi bật
POST	/<named-product>t/check-stock/	Kiểm tra tồn kho
GET	/health	Health check endpoint



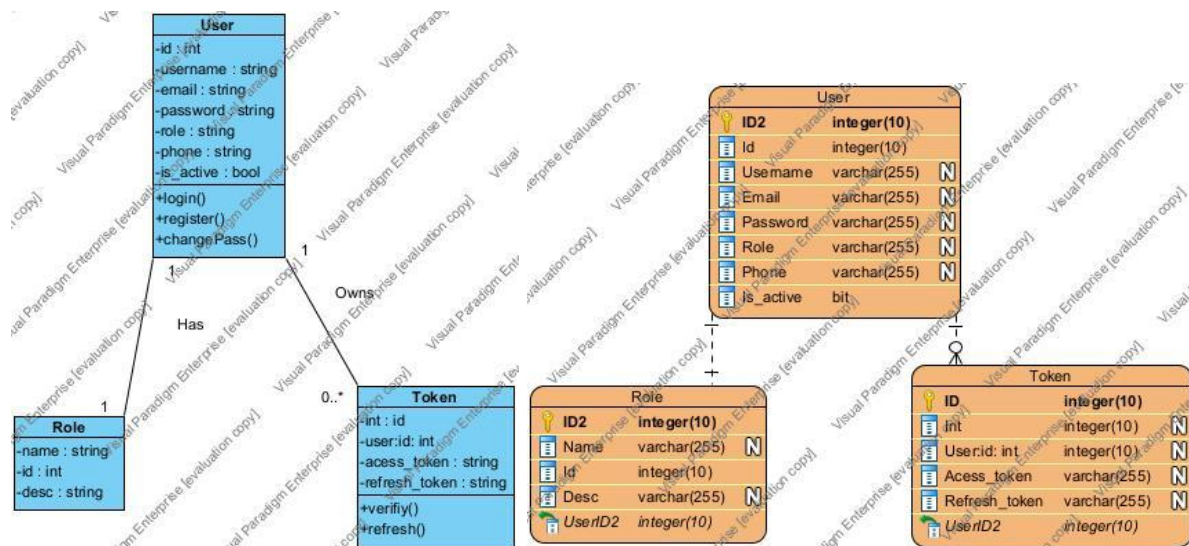
Bảng 2.4: API Endpoints của Product Service (ví dụ: laptop-service)

2.4 Thiết kế User Service (Django)

2.4.1 Phân loại người dùng và RBAC

- Admin: Toàn quyền hệ thống — CRUD sản phẩm, người dùng, đơn hàng
- Staff: Xử lý đơn hàng, quản lý vận hành, xem báo cáo
- Customer: Mua hàng, xem sản phẩm, quản lý giỏ hàng và đơn hàng của mình

2.4.2 Diagram



2.4.3 Model

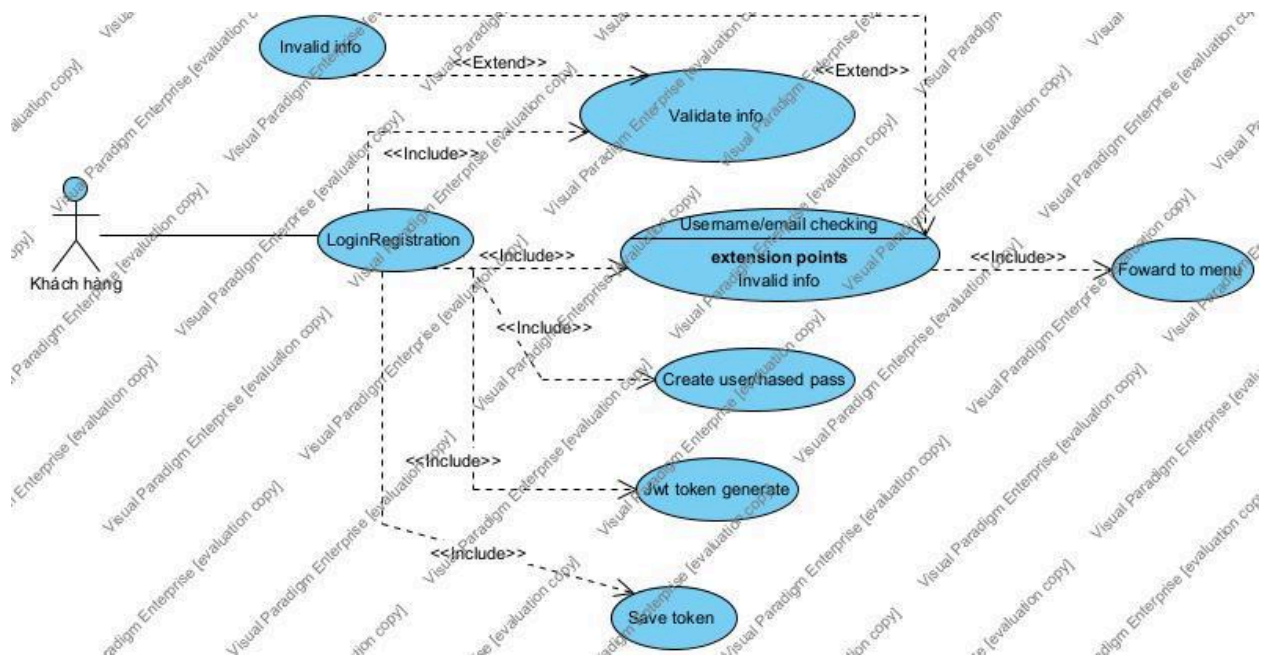
```
from django.contrib.auth.models import AbstractUser
from django.db import models

class User(AbstractUser):
    ROLE_CHOICES = (
        ('admin', 'Admin'),
        ('staff', 'Staff'),
        ('customer', 'Customer'),
    )
    role = models.CharField(max_length=20, choices=ROLE_CHOICES, default='customer')
    phone = models.CharField(max_length=20, blank=True)
    address = models.TextField(blank=True)
    avatar = models.CharField(max_length=10, default='👤')

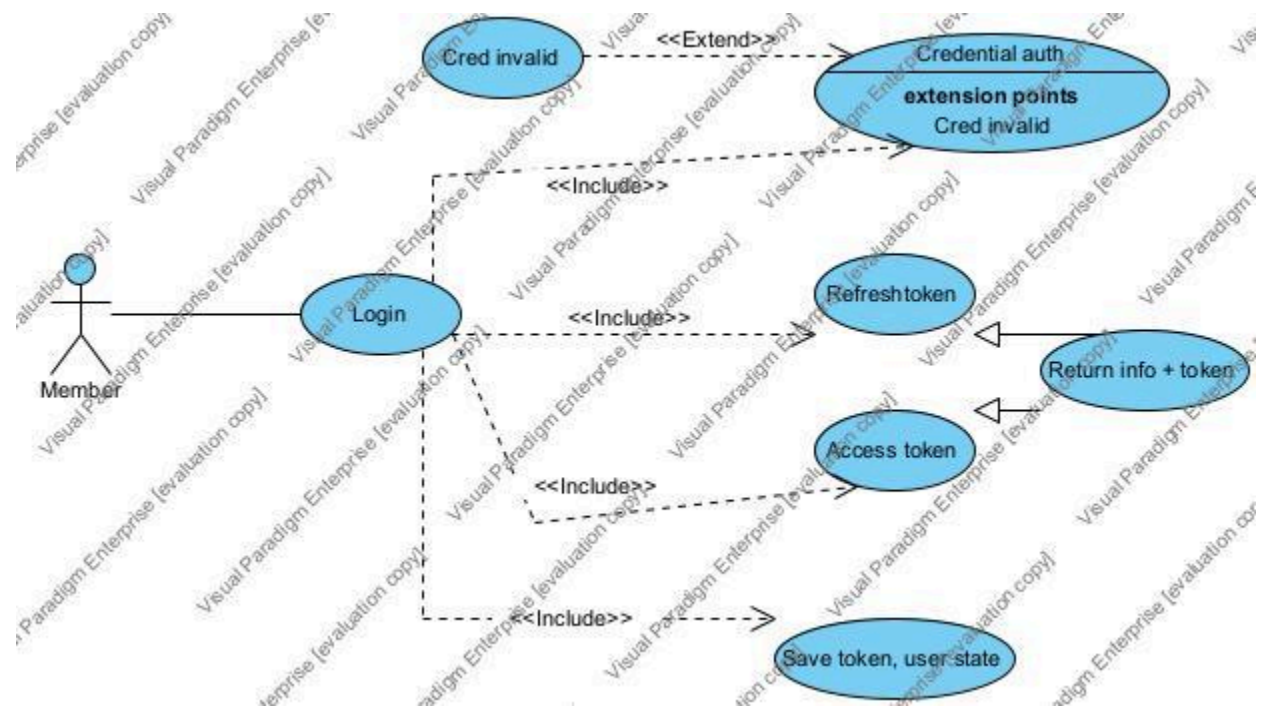
    class Meta:
        db_table = 'users'

    def __str__(self):
        return f"{self.username} ({self.role})"
```

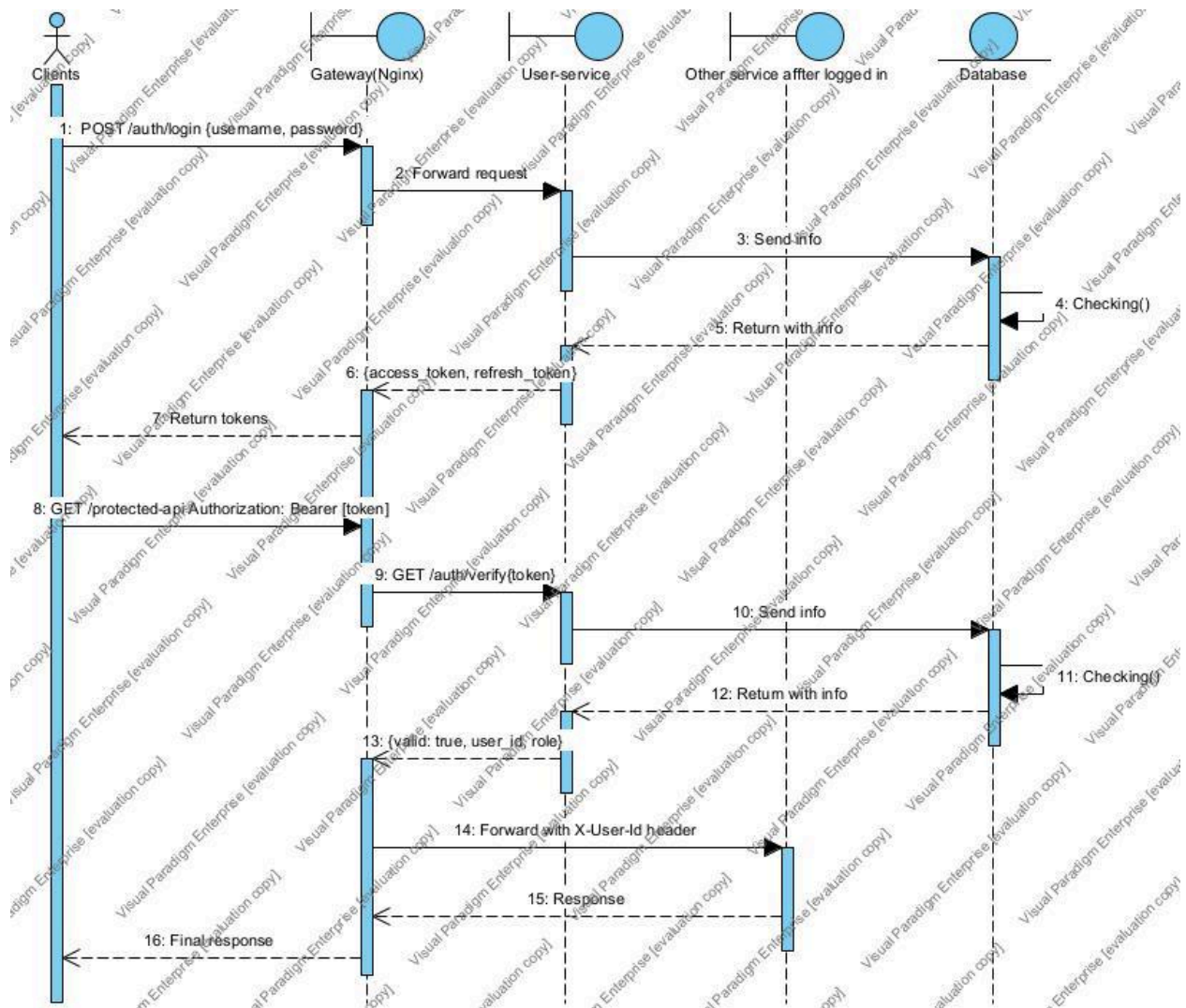
2.4.3 Đăng ký tài khoản



2.4.4. Đăng nhập



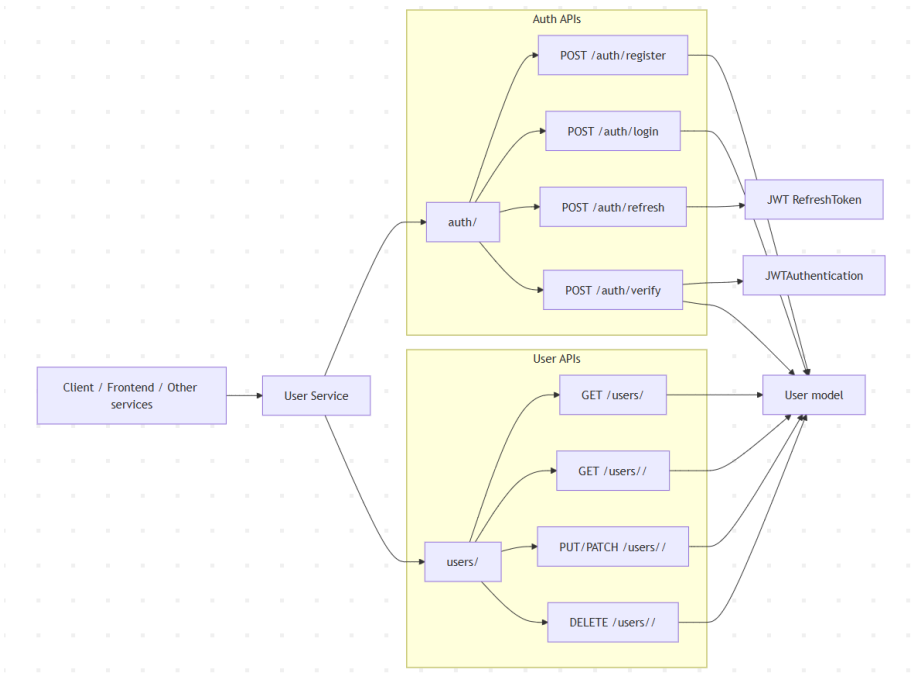
2.4.5 JWT Authentication Flow



Hình 2.2: Luồng xác thực JWT trong ShopMe

2.4.6 API Endpoints

Method	Endpoint	Mô tả
POST	/auth/register	Đăng ký tài khoản mới
POST	/auth/login	Đăng nhập, trả về JWT
POST	/auth/refresh	Làm mới access token
GET	/auth/verify	Xác thực token (dùng bởi Gateway)
GET	/users/me	Thông tin người dùng hiện tại
GET	/users/	Danh sách người dùng (admin only)



Bảng 2.5: API Endpoints của user-service

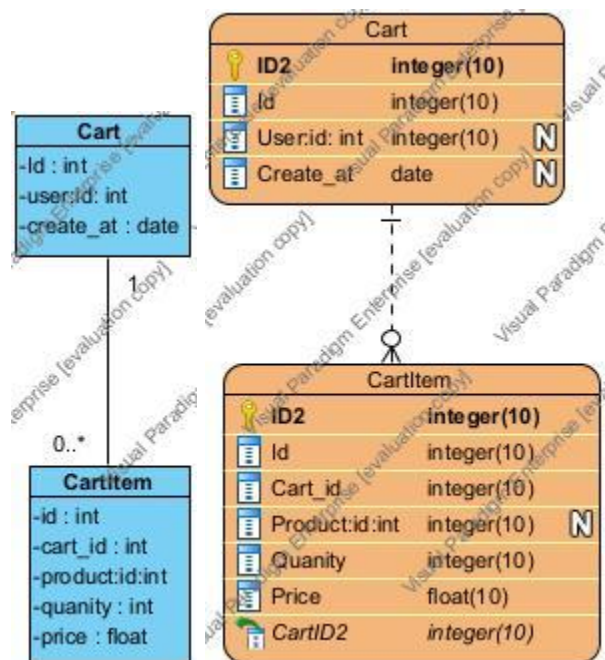
2.5 Thiết kế Cart Service

2.5.1 Model

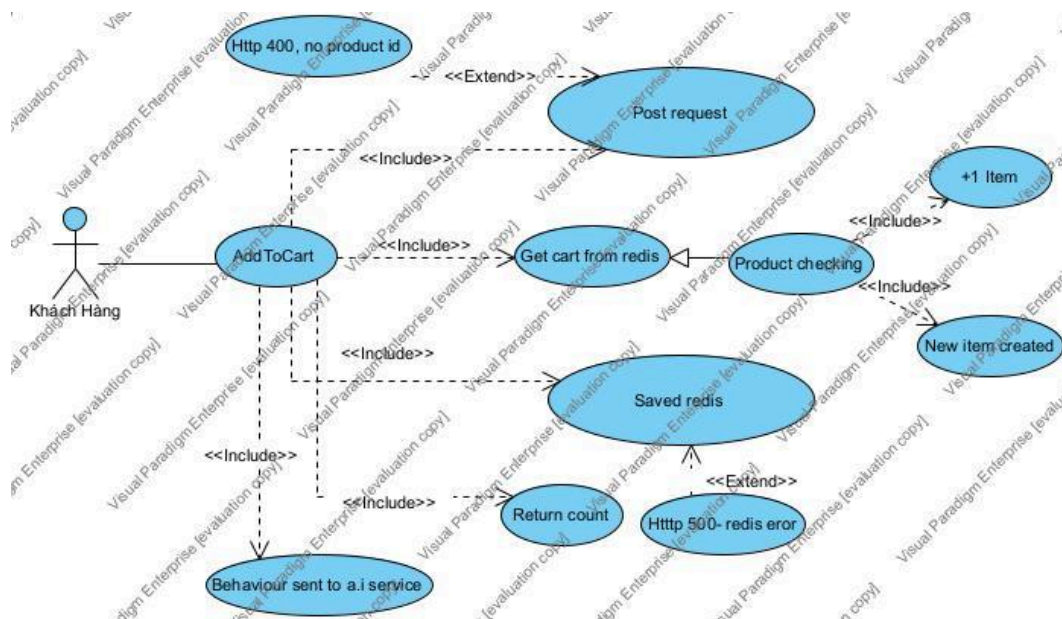
```
class Cart(models.Model):
    user_id = models.IntegerField()
    created_at = models.DateTimeField(auto_now_add=True)

class CartItem(models.Model):
    cart = models.ForeignKey(Cart, on_delete=models.CASCADE)
    product_id = models.CharField(max_length=50)
    service = models.CharField(max_length=50)
    quantity = models.IntegerField(default=1)
    price = models.FloatField()
```

2.5.3 Diagram

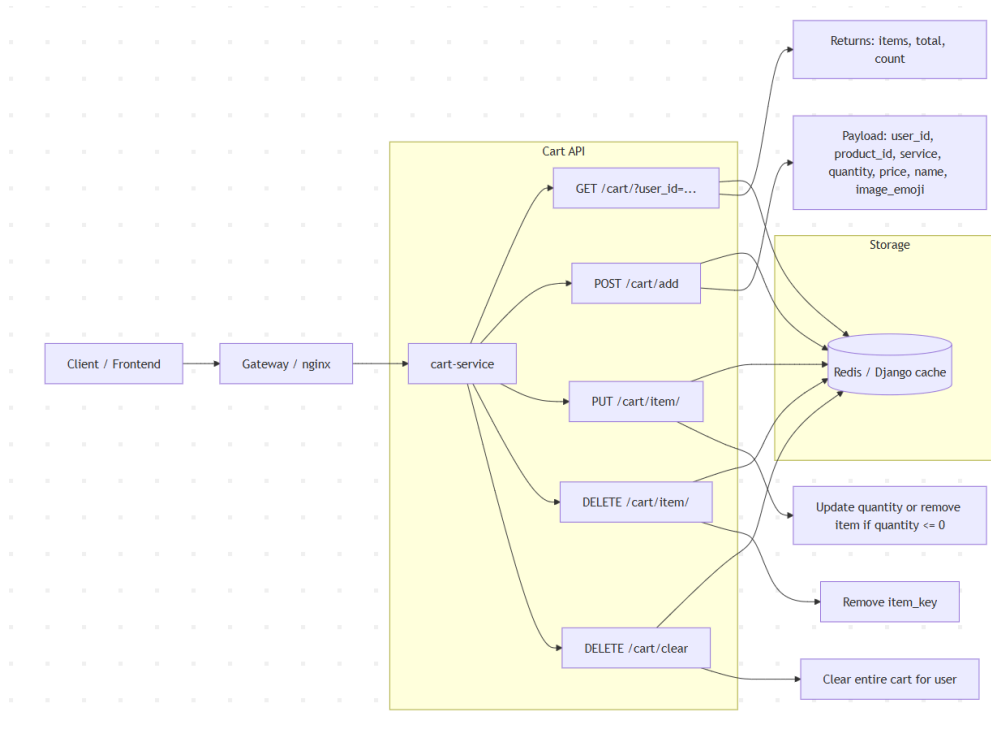


2.5.4 Thêm vào giỏ hàng



2.5.6 API

Method	Endpoint	Mô tả
GET	/cart/	Xem giỏ hàng hiện tại
POST	/cart/add	Thêm sản phẩm vào giỏ
PUT	/cart/update	Cập nhật số lượng
DELETE	/cart/remove	Xóa item khỏi giỏ
DELETE	/cart/clear	Xóa toàn bộ giỏ hàng



Bảng 2.6: API Endpoints của cart-service

2.6 Thiết kế Order Service

2.6.1 Model và Workflow

```
from django.db import models

class Order(models.Model):
    STATUS_CHOICES = [
        ('pending', 'Chờ xử lý'), ('confirmed', 'Đã xác nhận'),
        ('paid', 'Đã thanh toán'), ('shipping', 'Đang giao'), ('delivered', 'Đã giao'), ('cancelled', 'Đã hủy'),
    ]
    PAYMENT_METHODS = [('cash', 'Tiền mặt'), ('transfer', 'Chuyển khoản'), ('card', 'Thẻ'), ('momo', 'MoMo')]

    user_id = models.IntegerField()
    customer_name = models.CharField(max_length=255, blank=True)
    status = models.CharField(max_length=20, choices=STATUS_CHOICES, default='pending')
    total_price = models.FloatField(default=0)
    address = models.TextField()
    phone = models.CharField(max_length=20, blank=True)
    payment_method = models.CharField(max_length=20, choices=PAYMENT_METHODS, default='cash')
    note = models.TextField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'orders'
        ordering = ['-created_at']

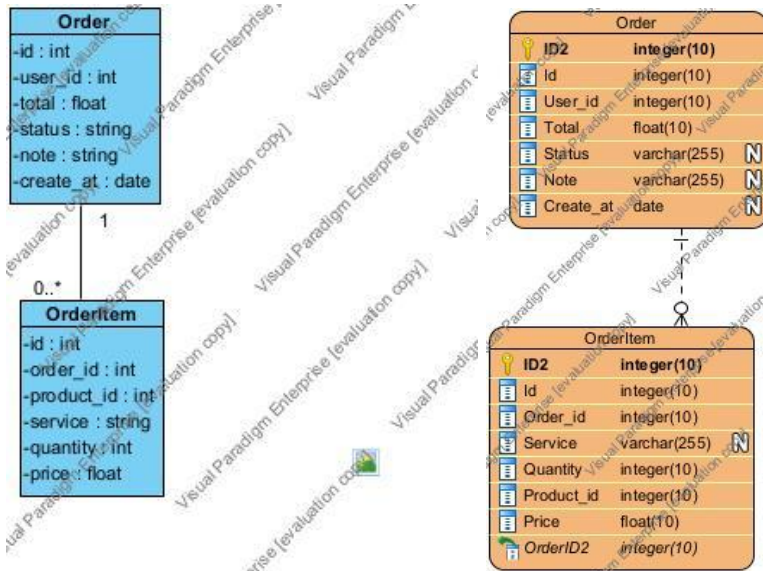
    def __str__(self):
        return f"Order #{self.id} - {self.status}"

class OrderItem(models.Model):
    order = models.ForeignKey(Order, on_delete=models.CASCADE, related_name='items')
    product_id = models.IntegerField()
    service_name = models.CharField(max_length=50)
    product_name = models.CharField(max_length=255)
    product_emoji = models.CharField(max_length=10, default='🍷')
    quantity = models.IntegerField()
    unit_price = models.FloatField()

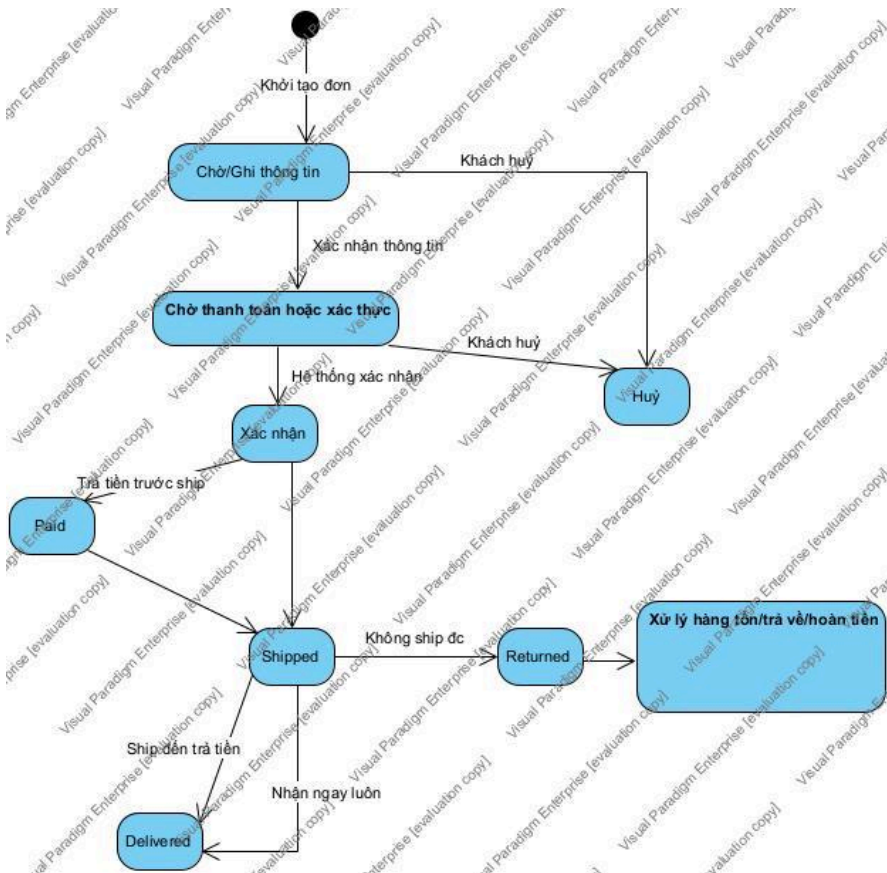
    class Meta:
        db_table = 'order_items'

    @property
    def subtotal(self):
        return self.unit_price * self.quantity
```


2.6.2 Diagram



2.6.3 Order Workflow



Hình 2.3: State Machine của Order

2.7 Thiết kế Payment Service

2.7.1 Model

```
from django.db import models
import uuid

class Payment(models.Model):
    STATUS = [('pending', 'Đang xử lý'), ('success', 'Thành công'), ('failed', 'Thất bại'), ('refunded', 'Hoàn tiền')]
    METHODS = [('cash', 'Tiền mặt'), ('transfer', 'Chuyển khoản'), ('card', 'Thẻ tín dụng'), ('momo', 'MoMo')]

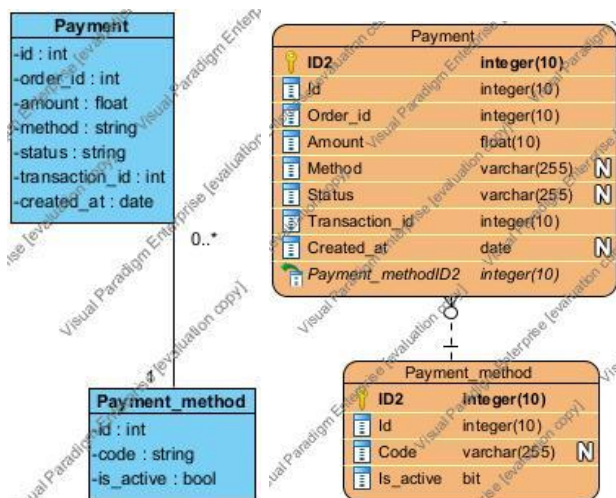
    order_id = models.IntegerField(unique=True)
    amount = models.FloatField()
    method = models.CharField(max_length=20, choices=METHODS, default='cash')
    status = models.CharField(max_length=20, choices=STATUS, default='pending')
    transaction_id = models.CharField(max_length=100, default='', blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'payments'
        ordering = ['-created_at']
```

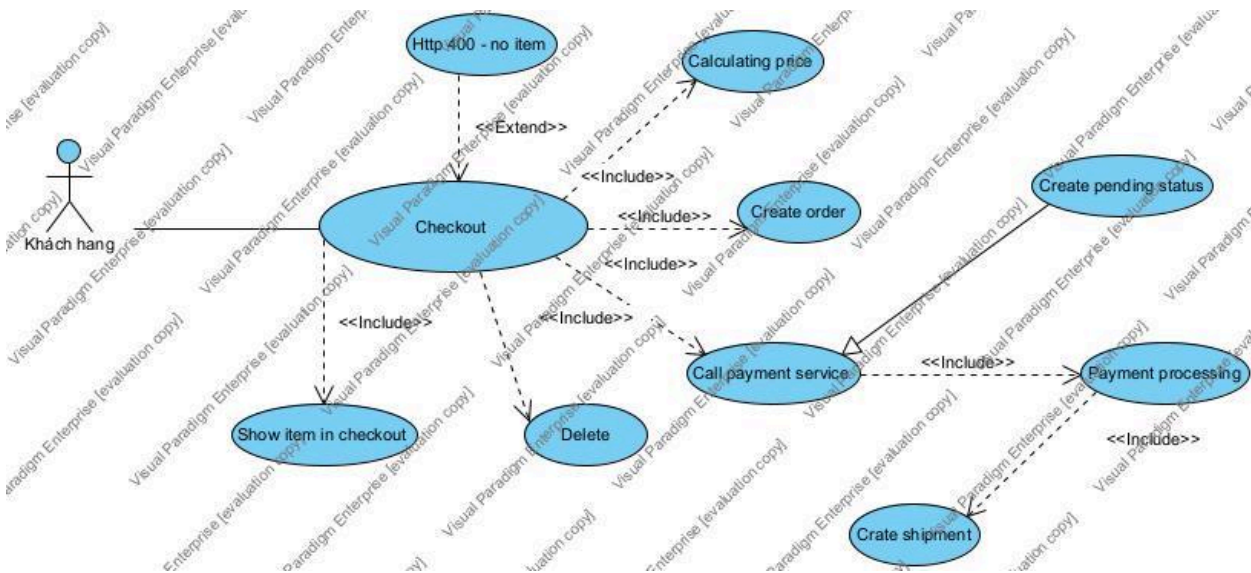
2.7.2 Trạng thái Payment

- pending — Đang chờ xử lý
- success — Thanh toán thành công → trigger Shipping Service
- failed — Thanh toán thất bại → rollback Order về pending

2.7.2 Diagram

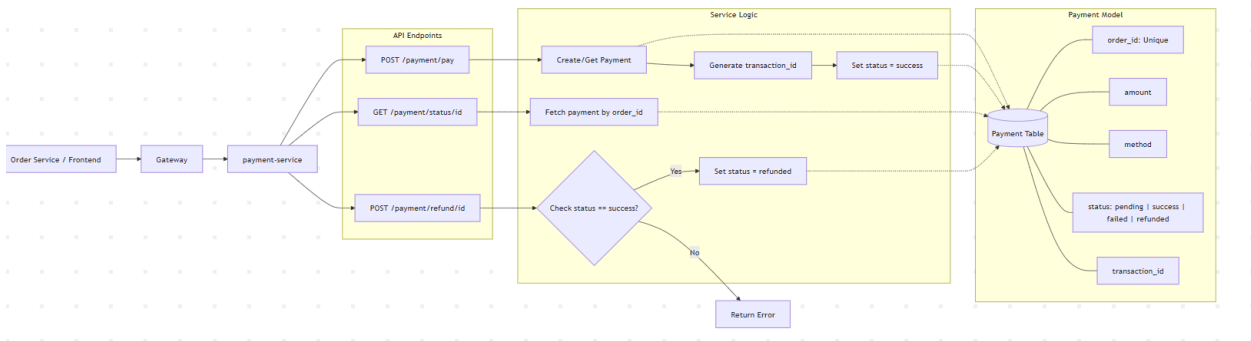


2.7.3. Tạo đơn và thanh toán



2.7.4. API

Method	Endpoint	Mô tả
POST	/payment/pay	Tạo hoặc lấy payment theo order_id, sinh transaction_id, và set trạng thái success.
GET	/payment/status/<order_id>	Lấy trạng thái và thông tin payment theo order_id.
POST	/payment/refund/<order_id>	Hoàn tiền payment nếu trạng thái hiện tại là success.



2.8 Thiết kế Shipping Service

2.8.1 Model và Trạng thái

```
from django.db import models
import uuid
from datetime import date, timedelta

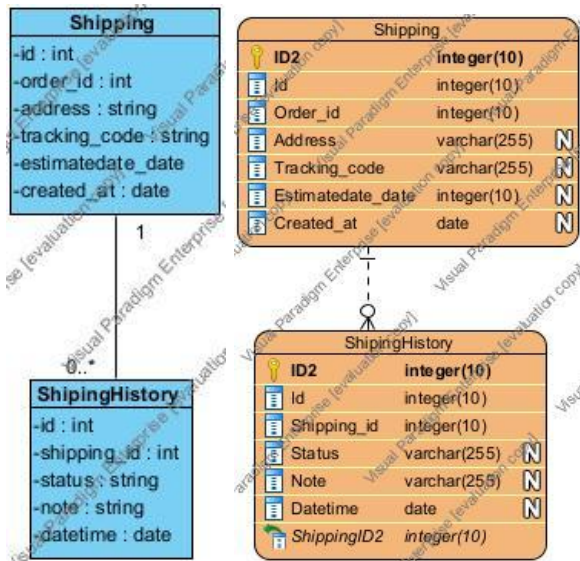
class Shipment(models.Model):
    STATUS = [
        ('processing', 'Đang xử lý'), ('shipping', 'Đang giao'),
        ('delivered', 'Đã giao'), ('failed', 'Giao thất bại'),
    ]
    order_id = models.IntegerField(unique=True)
    address = models.TextField()
    phone = models.CharField(max_length=20, blank=True)
    status = models.CharField(max_length=20, choices=STATUS, default='processing')
    tracking_code = models.CharField(max_length=50, default='', blank=True)
    estimated_date = models.DateField(null=True, blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    class Meta:
        db_table = 'shipments'
        ordering = ['-created_at']

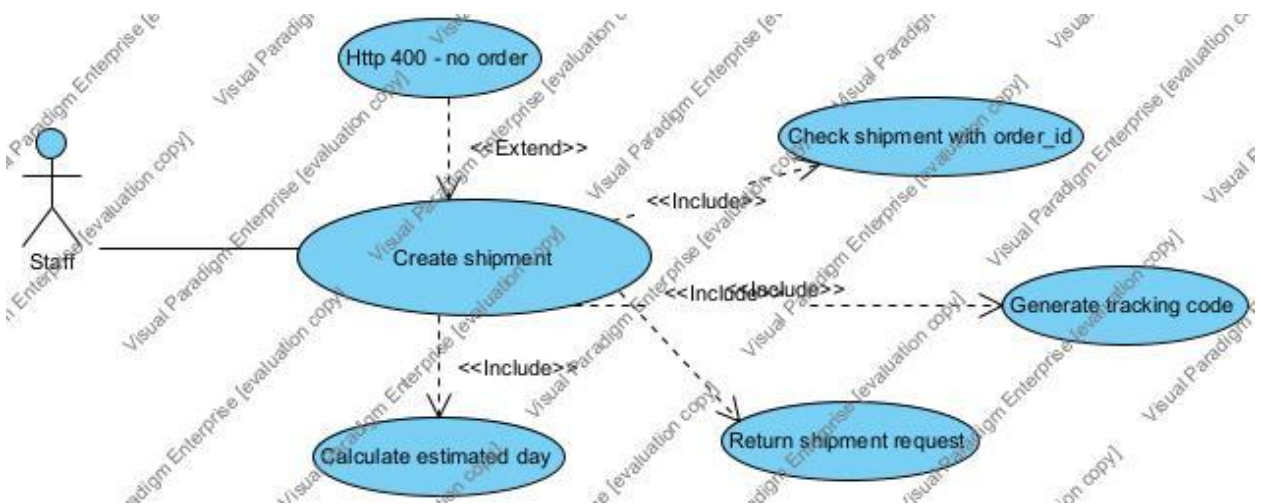
    def save(self, *args, **kwargs):
        if not self.tracking_code:
            self.tracking_code = f"SM{uuid.uuid4().hex[:8].upper()}"
        if not self.estimated_date:
            self.estimated_date = date.today() + timedelta(days=3)
        super().save(*args, **kwargs)
```

- processing — Đang chuẩn bị hàng
- shipping — Đang vận chuyển
- delivered — Giao thành công

2.8.2 Diagram

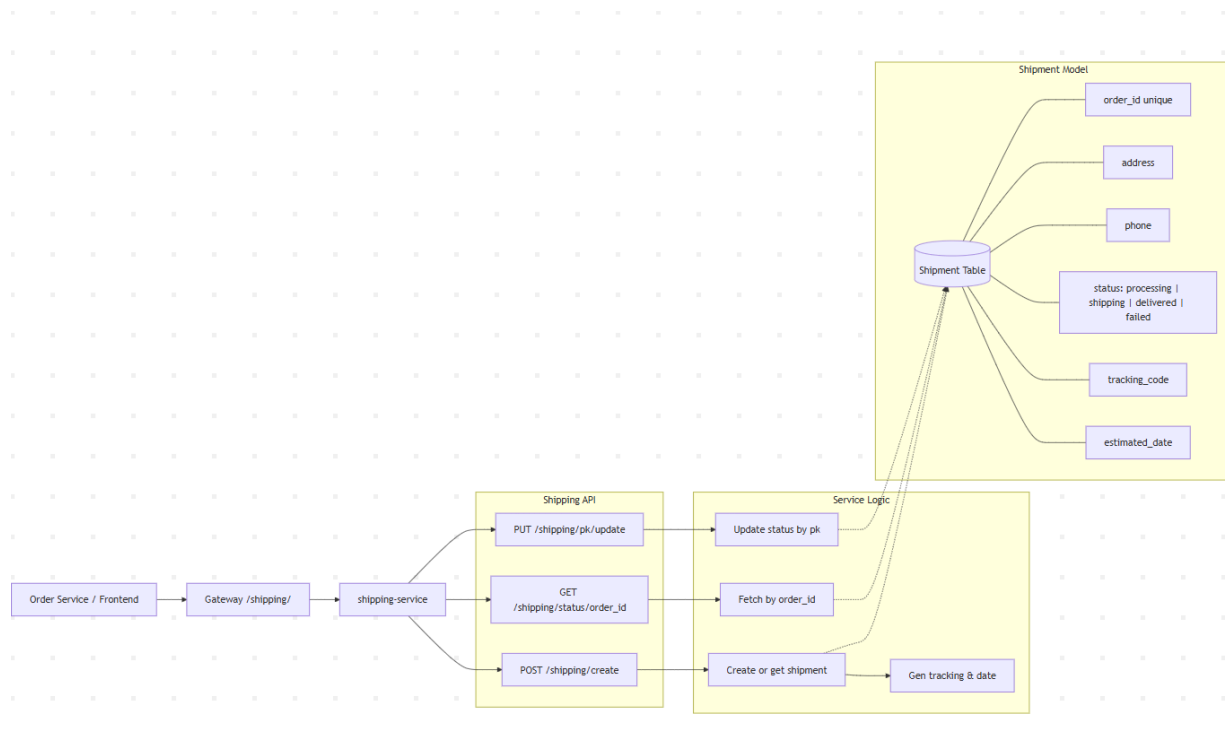


2.8.3. Shipping

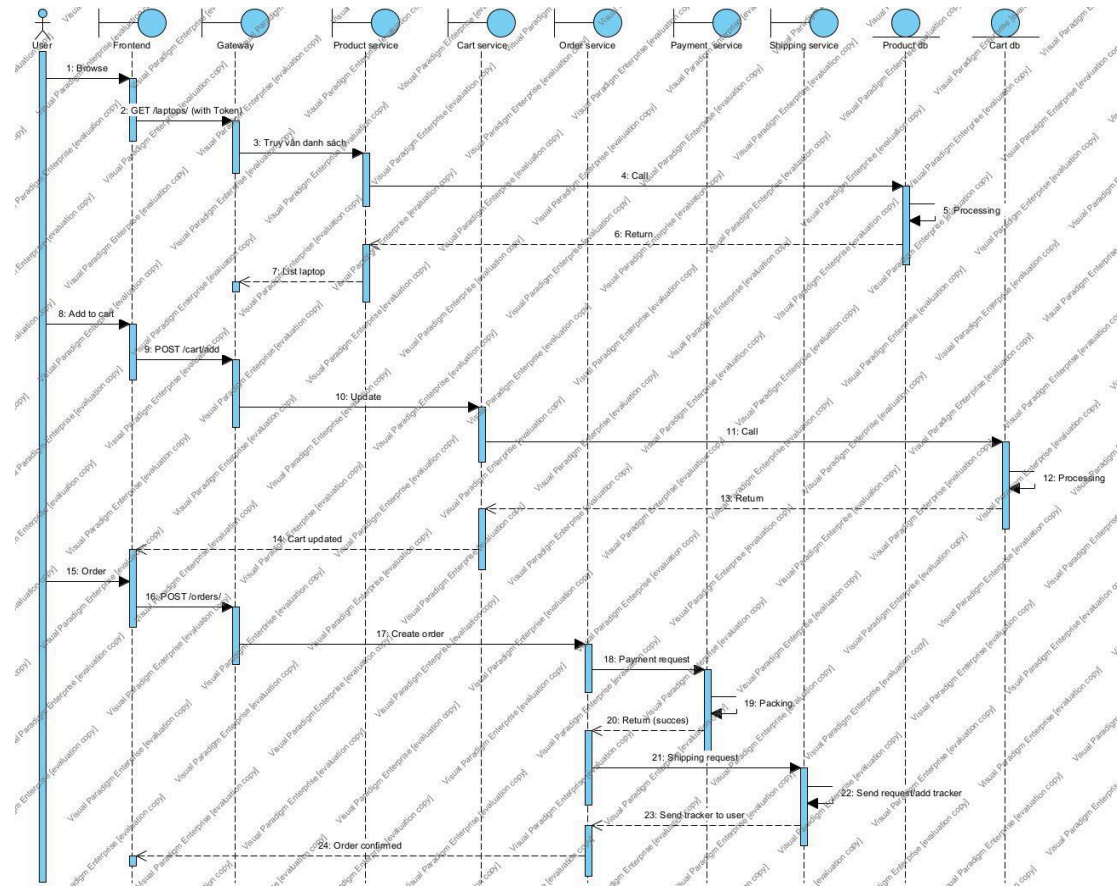


2.8.3. API

Method	Endpoint	Mô tả
POST	/shipping/create	Tạo shipment mới theo order_id. Nếu shipment đã tồn tại thì lấy lại bản ghi cũ. Body cần order_id, có thể gửi thêm address và phone.
GET	/shipping/status/<order_id>	Lấy thông tin shipment theo order_id.
PUT	/shipping<pk>/update	Cập nhật trạng thái shipment theo khóa chính pk. Body có field status.



2.9 Luồng hệ thống tổng thể (End-to-End)- sau đăng nhập



Hình 2.4: Sequence Diagram luồng mua hàng End-to-End

2.10 Thiết kế Database cho từng Service

2.10.1 Lựa chọn Database theo Service

Service	Database	Lý do chọn	Schema
user-service	MySQL 8.4	Phù hợp authentication, RBAC, phổ biến	shopme_users
product services (×10)	PostgreSQL	Hỗ trợ JSON tốt, tối ưu queries phức tạp	shopme_{category}
cart-service	Redis + PostgreSQL	Redis cho session nhanh, PG cho persist	shopme_cart
order-service	PostgreSQL	ACID transactions cho đơn hàng	shopme_orders
payment-service	PostgreSQL	ACID cho giao dịch tài chính	shopme_payments
shipping-service	PostgreSQL	Tracking, timeline vận chuyển	shopme_shipping
ai-service	Neo4j + MongoDB + FAISS	Graph DB, document store, vector search	—

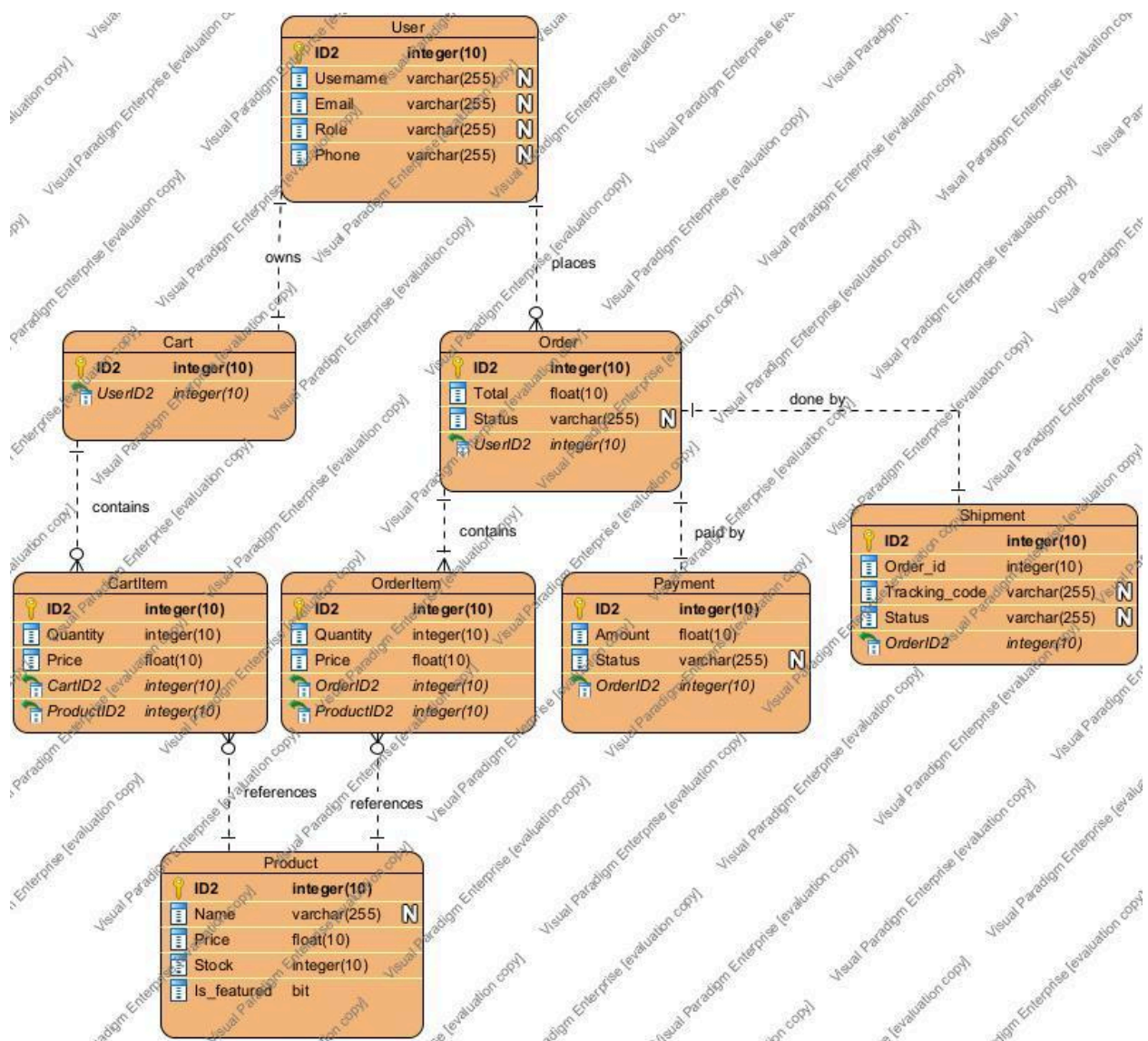
Bảng 2.7: Database per Service trong ShopMe

2.10.2 So sánh MySQL vs PostgreSQL

Tiêu chí	MySQL 8.4	PostgreSQL 15
Hiệu năng	Tốt cho read-heavy workload	Tốt cho complex queries
JSON Support	JSON (trung bình)	JSONB (mạnh, có index)
ACID	Đầy đủ (InnoDB)	Đầy đủ, mạnh hơn
Full-text Search	Hạn chế	Có built-in
Extensions	Ít hơn	PostGIS, pg_vector, v.v.
Dùng trong ShopMe	user-service	9 services còn lại

Bảng 2.8: So sánh MySQL vs PostgreSQL

2.10.3 Database Diagram — Core Services



Hình 2.5: Database Diagram các Core Services

2.11. Kết luận Chương 2

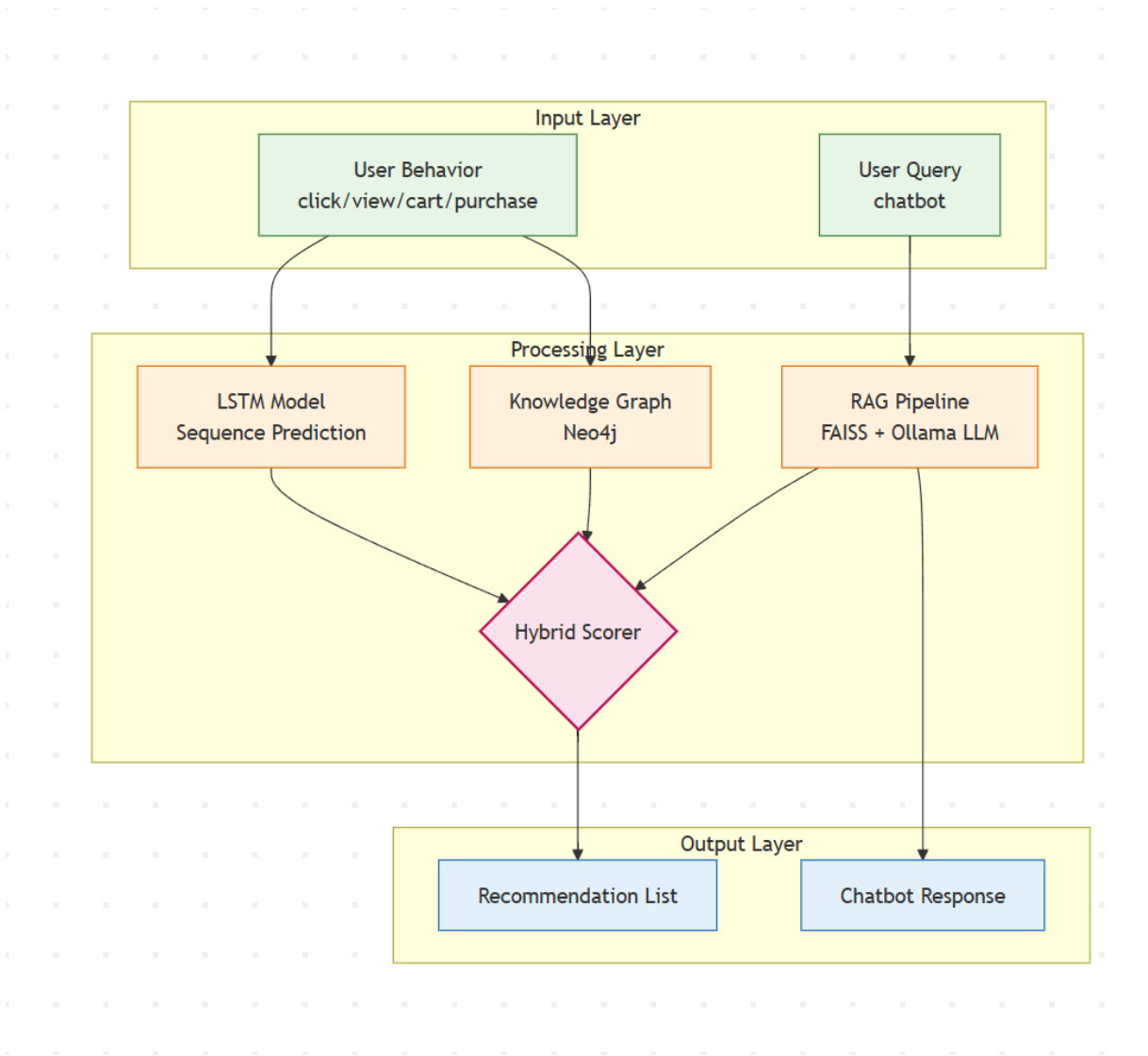
ShopMe được phân rã thành 17 microservice độc lập, mỗi service sở hữu database riêng theo đúng nguyên tắc Database per Service. 10 product services dùng cùng template Django nhưng deploy độc lập với dữ liệu riêng. Luồng mua hàng end-to-end đi qua 5 service theo thứ tự: user → product → cart → order → payment → shipping. Kiến trúc này đảm bảo loose coupling, fault isolation và khả năng scale độc lập từng module.

CHƯƠNG 3. AI Service cho Tư vấn Sản phẩm

3.1 Mục tiêu

AI Service trong ShopMe được xây dựng như một microservice độc lập (FastAPI), có nhiệm vụ cá nhân hóa trải nghiệm mua sắm thông qua hai chức năng chính: gợi ý sản phẩm tự động và chatbot tư vấn sản phẩm tích hợp LLM. Service này kết hợp ba kỹ thuật AI: LSTM sequence modeling, Knowledge Graph (Neo4j), và RAG (Retrieval-Augmented Generation).

3.2 Kiến trúc AI Service



Hình 3.1: Kiến trúc AI Service của ShopMe

3.3 Thu thập dữ liệu — User Behavior

3.3.1 Cấu trúc dữ liệu hành vi

```
{
  'user_id':      'user_123',
  'product_id':  'laptop_1',
  'service':      'laptop',
  'action':       'view | add_to_cart | purchase | click',
  'timestamp':    '2026-04-27T10:30:00Z'
}
```

3.3.2 Ví dụ dataset

user_id	product_id	action	service	timestamp
user_1	laptop_1	view	laptop	2026-04-27 08:01
user_1	laptop_3	view	laptop	2026-04-27 08:05
user_1	laptop_1	add_to_cart	laptop	2026-04-27 08:10
user_2	mobile_2	view	mobile	2026-04-27 09:15
user_2	mobile_2	purchase	mobile	2026-04-27 09:20

```
user_id,product_id,action,timestamp
1,laptop_4,click,2025-11-24 00:58:00
1,mobile_4,view,2025-11-24 01:00:00
1,laptop_1,click,2025-11-24 01:02:00
1,furniture_2,click,2025-11-24 01:37:00
1,tablet_1,click,2025-11-24 02:29:00
1,furniture_1,add_to_cart,2025-11-24 03:14:00
1,food_2,view,2025-11-24 04:03:00
1,laptop_4,view,2025-11-24 04:26:00
2,laptop_2,view,2025-06-26 00:19:00
2,beauty_1,add_to_cart,2025-06-26 00:43:00
2,shoes_2,view,2025-06-26 01:26:00
2,furniture_2,purchase,2025-06-26 02:21:00
2,beauty_1,view,2025-06-26 02:51:00
2,furniture_2,view,2025-06-26 03:14:00
2,furniture_2,click,2025-06-26 03:58:00
2,food_2,click,2025-06-26 04:33:00
3,food_1,view,2025-05-06 00:50:00
3,mobile_4,add_to_cart,2025-05-06 01:42:00
3,clothes_3,view,2025-05-06 02:41:00
3,food_1,view,2025-05-06 03:23:00
3,food_1,view,2025-05-06 03:40:00
3,food_1,click,2025-05-06 03:57:00
3,shoes_1,add_to_cart,2025-05-06 04:23:00
3,shoes_2,click,2025-05-06 04:29:00
4,sport_2,view,2025-01-25 00:39:00
4,book_1,view,2025-01-25 01:15:00
4,laptop_1,click,2025-01-25 01:23:00
4,mobile_2,add_to_cart,2025-01-25 02:13:00
4,sport_2,view,2025-01-25 02:24:00
4,beauty_3,add_to_cart,2025-01-25 02:41:00
4,furniture_1,view,2025-01-25 03:40:00
4,beauty_3,view,2025-01-25 04:21:00
5,book_1,purchase,2025-09-17 00:39:00
5,beauty_1,view,2025-09-17 01:03:00
5,sport_1,add_to_cart,2025-09-17 01:19:00
5,mobile_2,purchase,2025-09-17 01:25:00
5,sport_1,view,2025-09-17 02:14:00
5,mobile_3,view,2025-09-17 02:45:00
5,mobile_2,view,2025-09-17 03:41:00
5,mobile_3,purchase,2025-09-17 04:30:00
6,beauty_3,view,2025-12-20 00:16:00
6,tablet_2,view,2025-12-20 00:52:00
6,food_1,view,2025-12-20 01:38:00
6,food_1,view,2025-12-20 01:41:00
6,laptop_3,click,2025-12-20 01:59:00
6,food_1,click,2025-12-20 02:46:00
```

Bảng(Ảnh) 3.1: Ví dụ User Behavior Dataset

3.4 Mô hình LSTM (Sequence Modeling)

3.4.1 Ý tưởng

LSTM (Long Short-Term Memory) được dùng để dự đoán sản phẩm tiếp theo người dùng có thể quan tâm, dựa trên chuỗi hành vi lịch sử. Mô hình học từ pattern: nếu người dùng xem laptop A, laptop B, thường tiếp theo họ sẽ xem laptop C hoặc chuyển sang mobile.

3.4.2 Model chi tiết

```
"""
Nguyễn Nhật Hưng - B22DCDT148
"""
import os
import csv
import json
import math
import random
import pickle
from collections import defaultdict
from pathlib import Path

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader

# --- Config ---
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
SEQ_LEN = 10          # look-back window
EMBED_DIM = 32
HIDDEN_DIM = 64
EPOCHS = 20
BATCH_SIZE = 64
LR = 1e-3
MODEL_PATH = Path(__file__).parent / "lstm_weights.pt"
VOCAB_PATH = Path(__file__).parent / "lstm_vocab.json"
CSV_PATH = Path(__file__).parent / "user_behaviors.csv"

# Action weights (higher = more signal)
ACTION_WEIGHT = {"view": 1.0, "click": 2.0, "add_to_cart": 3.0, "purchase": 5.0}
```

```

# --- Dataset ---
class BehaviorDataset(Dataset):
    def __init__(self, sequences: list[list[int]]):
        self.X, self.Y = [], []
        for seq in sequences:
            for i in range(len(seq) - 1):
                start = max(0, i - SEQ_LEN + 1)
                window = seq[start : i + 1]
                # Pad left with 0 if needed
                padded = [0] * (SEQ_LEN - len(window)) + window
                self.X.append(padded)
                self.Y.append(seq[i + 1])
        self.X = torch.tensor(self.X, dtype=torch.long)
        self.Y = torch.tensor(self.Y, dtype=torch.long)

    def __len__(self):
        return len(self.X)

    def __getitem__(self, idx):
        return self.X[idx], self.Y[idx]

# --- Model ---
class LSTMRecommender(nn.Module):
    def __init__(self, vocab_size: int, embed_dim: int = EMBED_DIM,
                 hidden_dim: int = HIDDEN_DIM):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size + 1, embed_dim, padding_idx=0)
        self.lstm = nn.LSTM(embed_dim, hidden_dim, batch_first=True,
                             num_layers=2, dropout=0.3)
        self.dropout = nn.Dropout(0.3)
        self.fc = nn.Linear(hidden_dim, vocab_size + 1)

    def forward(self, x):
        emb = self.embedding(x)           # (B, SEQ_LEN, embed_dim)
        out, _ = self.lstm(emb)           # (B, SEQ_LEN, hidden_dim)
        out = self.dropout(out[:, -1, :]) # last timestep
        return self.fc(out)               # (B, vocab_size+1)

```

```

# — Training —
def train():
    print("📂 Loading behavior CSV...")
    if not CSV_PATH.exists():
        print(f"⚠️ CSV not found at {CSV_PATH}, skipping LSTM training")
        return

    # Build vocab: product_id → int
    user_sequences: dict[str, list[str]] = defaultdict(list)
    with open(CSV_PATH, newline="") as f:
        reader = csv.DictReader(f)
        for row in reader:
            uid = row["user_id"]
            pid = row["product_id"]
            # Repeat product in sequence based on action weight (heavier = more signal)
            w = int(ACTION_WEIGHT.get(row["action"], 1.0))
            user_sequences[uid].extend([pid] * w)

    all_products = sorted({p for seq in user_sequences.values() for p in seq})
    vocab = {p: i + 1 for i, p in enumerate(all_products)} # 0 = padding
    vocab_size = len(vocab)
    print(f" Vocab size: {vocab_size} products, {len(user_sequences)} users")

    # Save vocab
    with open(VOCAB_PATH, "w") as f:
        json.dump(vocab, f)

    # Encode sequences
    sequences = []
    for uid, seq in user_sequences.items():
        encoded = [vocab[p] for p in seq if p in vocab]
        if len(encoded) >= 2:
            sequences.append(encoded)

    dataset = BehaviorDataset(sequences)
    if len(dataset) == 0:
        print("⚠️ No training samples, skipping")
        return

    loader = DataLoader(dataset, batch_size=BATCH_SIZE, shuffle=True)

    model = LSTMRecommender(vocab_size).to(DEVICE)
    criterion = nn.CrossEntropyLoss(ignore_index=0)
    optimizer = optim.Adam(model.parameters(), lr=LR)
    scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=5, gamma=0.5)

    print(f"🚀 Training LSTM on {len(dataset)} samples for {EPOCHS} epochs...")

```

```

model.train()
for epoch in range(1, EPOCHS + 1):
    total_loss = 0
    for X, Y in loader:
        X, Y = X.to(DEVICE), Y.to(DEVICE)
        optimizer.zero_grad()
        logits = model(X)
        loss = criterion(logits, Y)
        loss.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 1.0)
        optimizer.step()
        total_loss += loss.item()
    scheduler.step()
    avg = total_loss / len(loader)
    if epoch % 5 == 0 or epoch == 1:
        print(f"    Epoch {epoch:02d}/{EPOCHS}   loss={avg:.4f}")

torch.save(model.state_dict(), MODEL_PATH)
print(f"✅ LSTM model saved → {MODEL_PATH}")
return model, vocab

```

```

# — Inference helper
class LSTMInference:
    def __init__(self):
        self.model: LSTMRecommender | None = None
        self.vocab: dict[str, int] = {}
        self.id2product: dict[int, str] = {}

    def load(self):
        if not MODEL_PATH.exists() or not VOCAB_PATH.exists():
            print("⚠️ LSTM weights not found – run train() first")
            return False
        with open(VOCAB_PATH) as f:
            self.vocab = json.load(f)
        self.id2product = {v: k for k, v in self.vocab.items()}
        vocab_size = len(self.vocab)
        self.model = LSTMRecommender(vocab_size).to(DEVICE)
        self.model.load_state_dict(torch.load(MODEL_PATH, map_location=DEVICE))
        self.model.eval()
        print(f"✅ LSTM loaded ({vocab_size} products)")
        return True

```



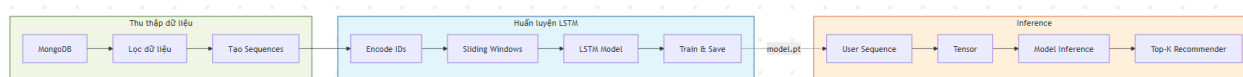
```

def predict(self, recent_products: list[str], top_k: int = 10) -> list[tuple[str, float]]:
    """Given recent product IDs, return top_k predicted next products with scores."""
    if self.model is None or not recent_products:
        return []
    encoded = [self.vocab[p] for p in recent_products if p in self.vocab]
    if not encoded:
        return []
    # Build sequence tensor
    padded = [0] * max(0, SEQ_LEN - len(encoded)) + encoded[-SEQ_LEN:]
    x = torch.tensor([padded], dtype=torch.long).to(DEVICE)
    with torch.no_grad():
        logits = self.model(x) # (1, vocab_size+1)
        probs = torch.softmax(logits[0], dim=0).cpu().numpy()
    # Get top_k (skip index 0 = padding)
    top_ids = probs[1:].argsort()[::-1][:top_k]
    results = []
    for idx in top_ids:
        product_id = self.id2product.get(idx + 1)
        if product_id:
            results.append((product_id, float(probs[idx + 1])))
    return results

if __name__ == "__main__":
    train()

```

3.4.3 LSTM



3.4.3.1. Giai đoạn Thu thập dữ liệu (Data Collection)

Đây là bước chuẩn bị nguyên liệu đầu vào cho mô hình:

- **MongoDB:** Nơi lưu trữ nhật ký hành vi người dùng (behavior logs), ví dụ: xem sản phẩm, thêm vào giỏ hàng, tìm kiếm.
- **Lọc dữ liệu:** Làm sạch dữ liệu, loại bỏ nhiễu, chỉ giữ lại các trường thông tin cần thiết như user_id, product_id, và dấu thời gian.
- **Tạo Sequences:** Gom nhóm các hành vi theo từng người dùng theo trình tự thời gian để tạo ra các chuỗi hành vi (ví dụ: User A xem Sản phẩm 1 -> Sản phẩm 2 -> Sản phẩm 3).

3.4.3.2. Giai đoạn Huấn luyện LSTM (Training)

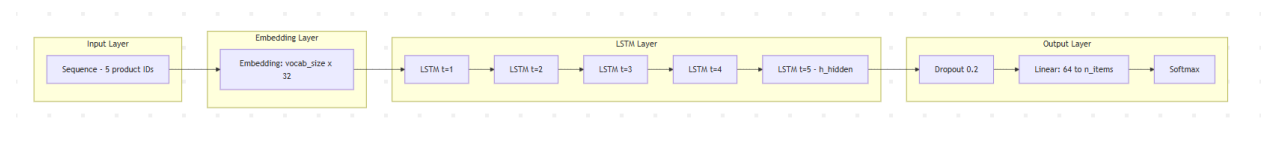
Đây là "bộ não" của hệ thống, nơi máy học cách hiểu sở thích dựa trên trình tự:

- **Encode IDs:** Chuyển đổi mã sản phẩm (product_id) dạng ký tự/string sang dạng số nguyên (integer index) để mô hình toán học có thể xử lý.
- **Sliding Windows:** Chia các chuỗi hành vi dài thành các đoạn nhỏ có độ dài cố định (cửa sổ trượt). Ví dụ: lấy 5 hành vi trước đó để dự đoán hành vi tiếp theo.
- **LSTM Model:** Sử dụng mạng LSTM – một loại mạng thần kinh hồi quy có khả năng "nhớ" được các thông tin quan trọng trong chuỗi dữ liệu dài, rất hiệu quả để nắm bắt sở thích theo thời gian.
- **Train & Save:** Quá trình tối ưu hóa trọng số của mô hình để đạt độ chính xác cao nhất và lưu lại kết quả dưới dạng file model.pt.

3.4.3.3. Giai đoạn Inference (Dự đoán thực tế)

Đây là lúc hệ thống đưa ra gợi ý cho người dùng thực tế:

- **User Sequence:** Lấy chuỗi hành vi hiện tại (mới nhất) của người dùng đang truy cập.
- **Tensor:** Chuyển đổi dữ liệu chuỗi đó thành định dạng Tensor (định dạng dữ liệu của các thư viện như PyTorch/TensorFlow) để đưa vào mô hình.
- **Model Inference:** Sử dụng file model.pt đã huấn luyện để tính toán xác suất sản phẩm tiếp theo mà người dùng có khả năng quan tâm nhất.
- **Top-K Recommender:** Trả về danh sách K sản phẩm có xác suất cao nhất để hiển thị lên màn hình (ví dụ: "Có thể bạn sẽ thích...").



A. Khối Input Layer (Tầng nhập liệu)

- **Dữ liệu đầu vào:** Hệ thống nhận vào một chuỗi (sequence) gồm **5 ID sản phẩm** mà người dùng đã tương tác gần nhất theo thứ tự thời gian.
- **Mục đích:** Cung cấp "ngữ cảnh" (context) về sở thích ngắn hạn của người dùng.

B. Khối Embedding Layer (Tầng biểu diễn đặc trưng)

- **Chức năng:** Chuyển đổi các ID sản phẩm (dạng con số rời rạc) thành các **vector mật độ cao** với kích thước **32 chiều**.
- **Ý nghĩa:** Trong không gian 32 chiều này, các sản phẩm có tính chất tương tự nhau (ví dụ: cùng loại "Laptop") sẽ nằm gần nhau, giúp mô hình hiểu được bản chất nội dung thay vì chỉ coi chúng là các con số đơn thuần.

C. Khối LSTM Layer (Tầng ghi nhớ trình tự)

- **Cơ chế:** Mô hình đọc chuỗi dữ liệu qua 5 bước thời gian ($t=1$ đến $t=5$).
- **Vai trò:** LSTM giữ lại các thông tin quan trọng từ các bước trước đó. Bước $t=5$ tạo ra một vector gọi là **Hidden State (h_{hidden})**. Vector này chính là "bản tóm tắt" tinh túy nhất về hành trình 5 sản phẩm mà người dùng đã xem.

D. Khối Output Layer (Tầng dự đoán kết quả)

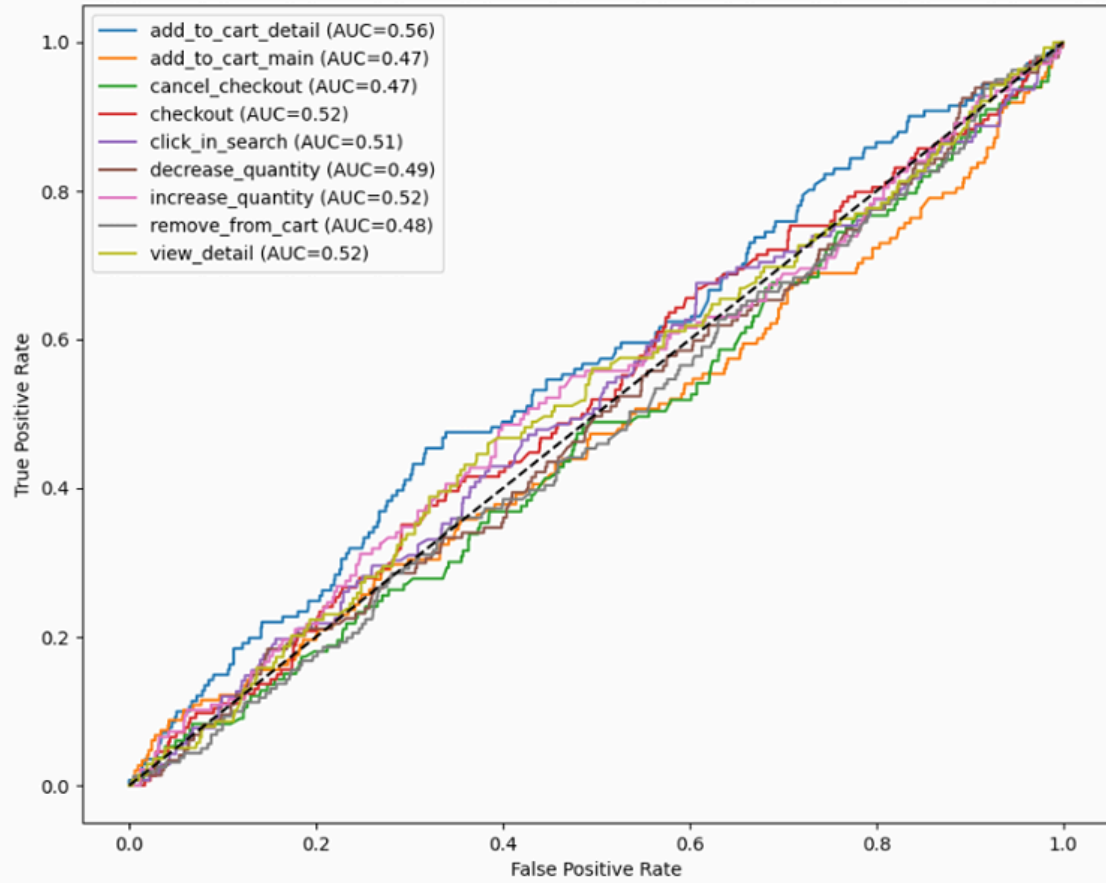
Đây là bước biến đổi vector từ tầng LSTM thành gợi ý cụ thể:

- **Dropout (0.2):** Ngẫu nhiên "tắt" 20% neuron để ép mô hình không học vẹt, tăng khả năng dự đoán đúng trên dữ liệu mới (chống Overfitting).
- **Linear (64 to n_{items}):** Chuyển vector ẩn từ tầng LSTM về không gian của toàn bộ sản phẩm trong kho (ví dụ: nếu shop bạn có 1000 món, nó sẽ map ra 1000 điểm số).
- **Softmax:** Hàm này biến các điểm số (logits) thành **xác suất (probabilities)**. Sản phẩm nào có xác suất cao nhất sẽ là sản phẩm được mô hình tin rằng người dùng muốn mua tiếp theo.

Kết quả:

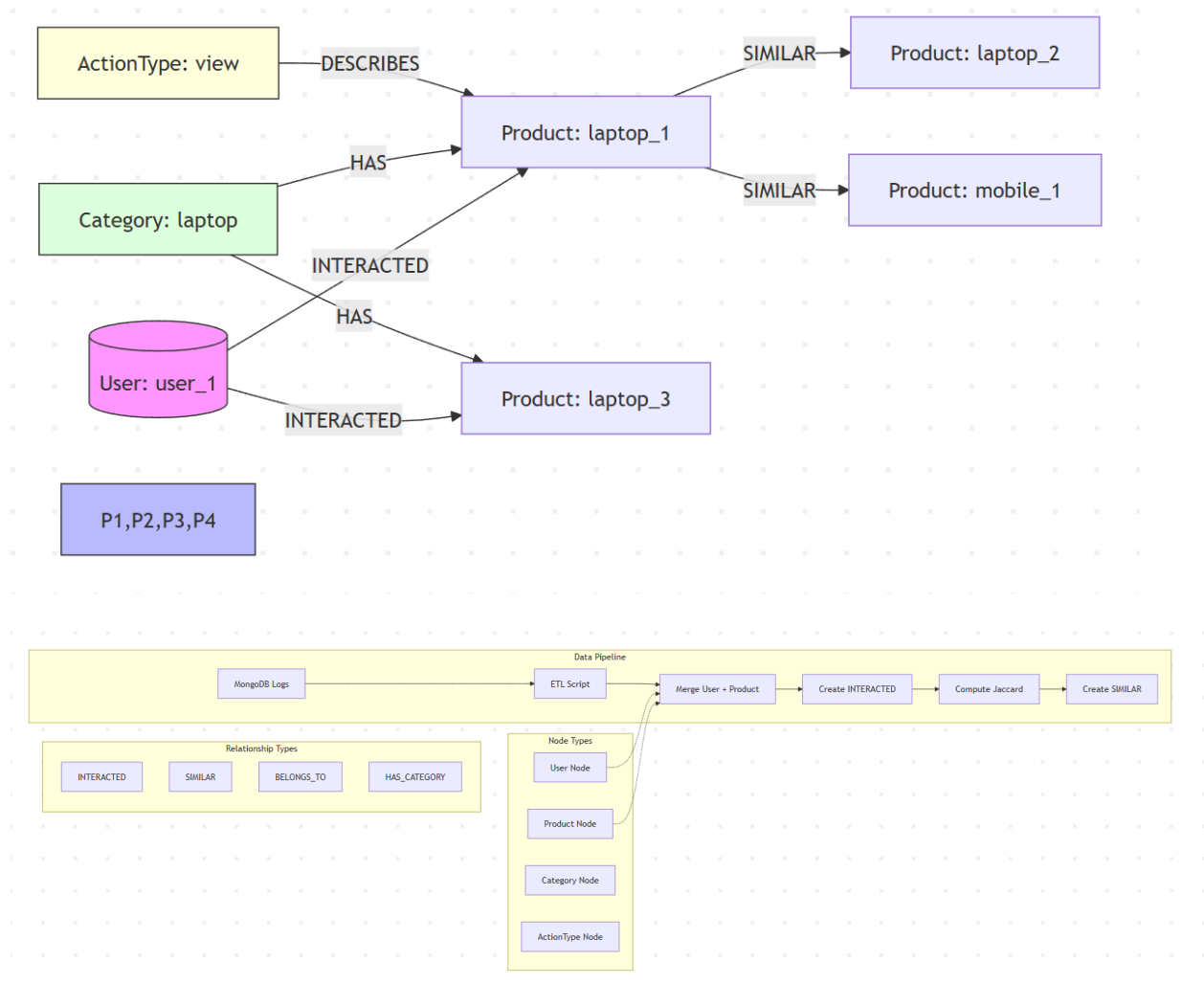


LSTM ROC Curve



3.5 Knowledge Graph với Neo4j

Mô hình đồ thị



Hình 3.2: Knowledge Graph Structure trong Neo4j

A. Data Pipeline (Luồng xử lý dữ liệu)

Đây là phân lõi vận hành hệ thống, biến dữ liệu thô thành đồ thị:

- **MongoDB Logs:** Nguồn dữ liệu đầu vào chứa nhật ký hành vi người dùng (view, cart, purchase).
- **ETL Script:** Script trích xuất, chuyển đổi và nạp dữ liệu. Nó đóng vai trò trung gian, chuẩn hóa dữ liệu từ dạng JSON/Document trong MongoDB sang cấu trúc phù hợp cho đồ thị.
- **Merge User + Product:** Quá trình kết nối hai thực thể chính (Người dùng và Sản phẩm) dựa trên hành vi tương tác.

- Create INTERACTED: Tạo ra các "cạnh" (đường nối) đại diện cho tương tác giữa người dùng và sản phẩm.
- Compute Jaccard & Create SIMILAR: Đây là bước tính toán thông minh. Thuật toán Jaccard Similarity được dùng để đo độ tương đồng giữa hai sản phẩm (dựa trên việc chúng có chung bao nhiêu người dùng tương tác). Nếu điểm tương đồng cao (>0.3), hệ thống sẽ tự động tạo một cạnh SIMILAR kết nối hai sản phẩm đó lại với nhau.

B. Node Types (Các loại nút - Đối tượng chính)

Trong đồ thị, dữ liệu được chia thành các nút:

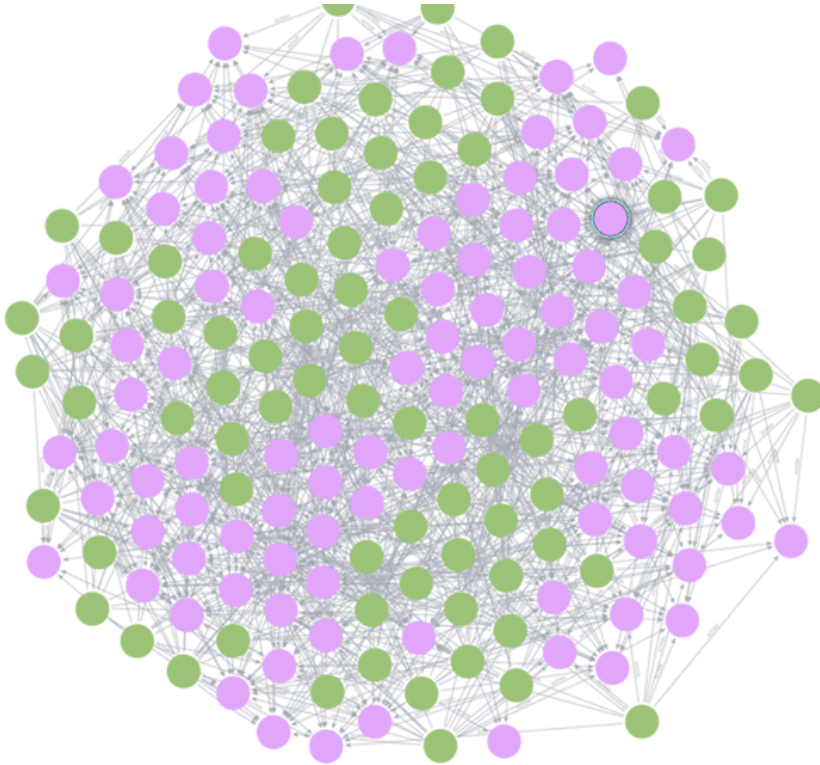
- User Node: Đại diện cho khách hàng.
- Product Node: Đại diện cho hàng hóa trong Bookstore.
- Category Node: Nhóm các sản phẩm lại với nhau.
- ActionType Node: Phân loại hành vi cụ thể (ví dụ: hành vi "Mua" sẽ có trọng số cao hơn hành vi "Xem").

C. Relationship Types (Các loại quan hệ - "Cạnh" nối)

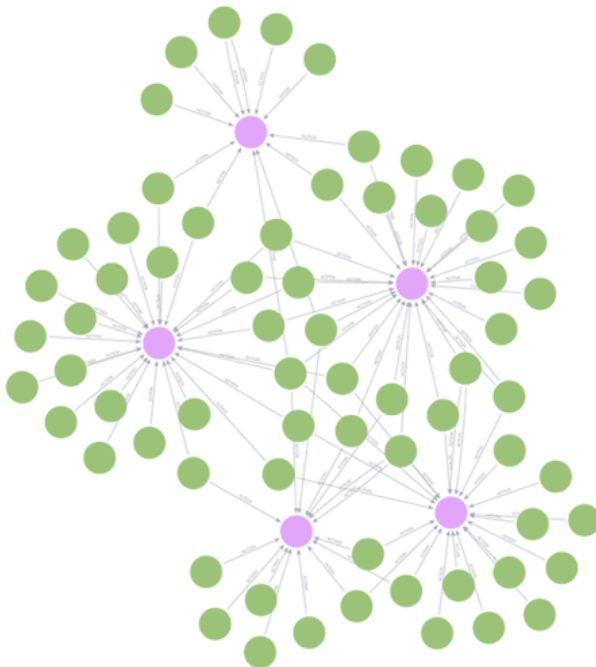
Các mối liên kết giữa các nút giúp hệ thống hiểu được bản chất ngữ nghĩa của dữ liệu:

- INTERACTED: Kết nối Người dùng với Sản phẩm.
- SIMILAR: Kết nối Sản phẩm với Sản phẩm (giúp gợi ý: "Người dùng đã xem A, có thể sẽ thích B").
- BELONGS_TO & HAS_CATEGORY: Kết nối Sản phẩm với Danh mục, giúp hệ thống gợi ý dựa trên phân loại sản phẩm.

Xây dựng knowledge Base graph (KB_Graph) với neo4j:

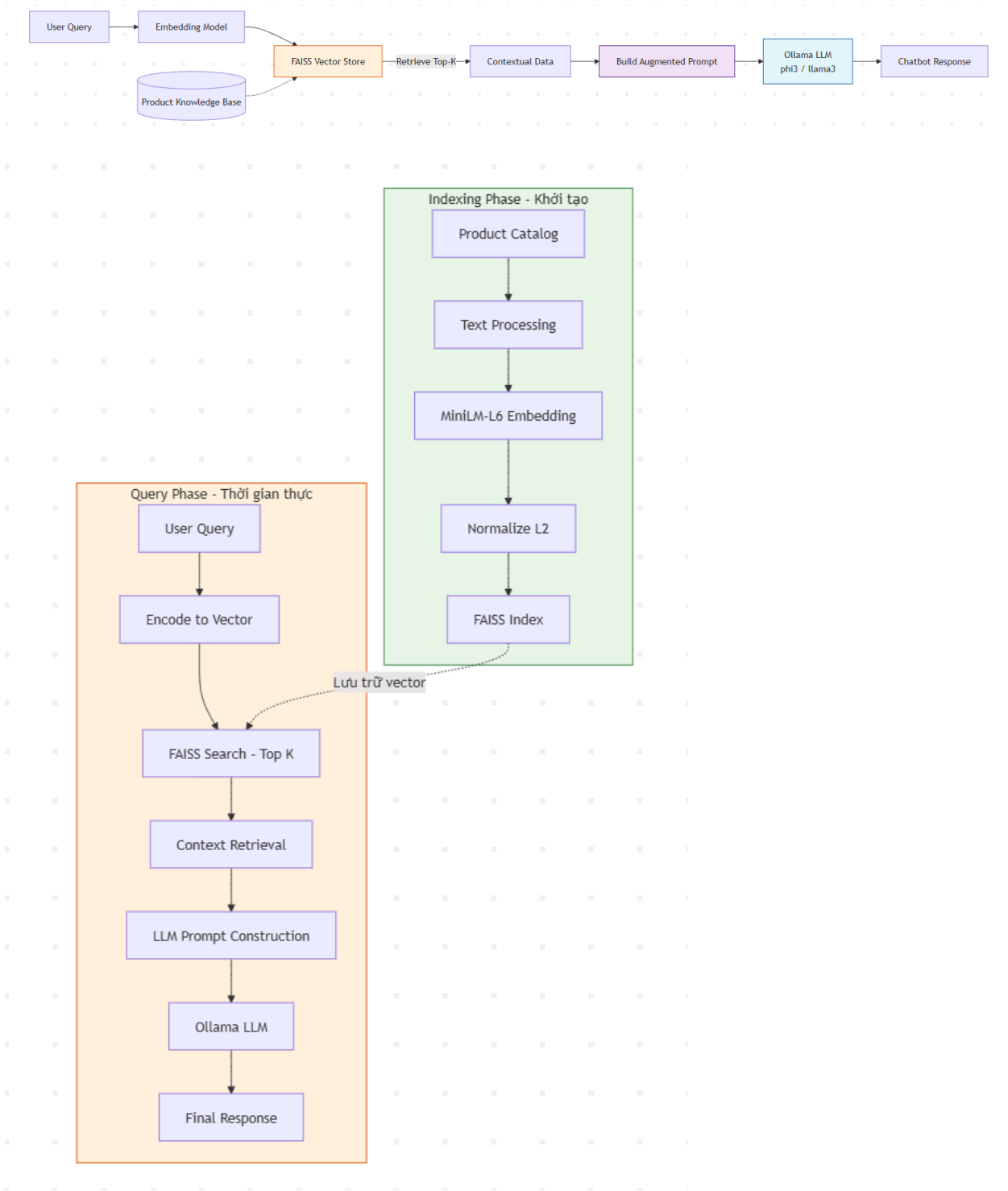


Limit 100:



3.6 RAG — Retrieval-Augmented Generation

3.6.1 Pipeline



Hình 3.3: RAG Pipeline của ShopMe Chatbot

1. Giai đoạn Indexing (Khởi tạo - Xử lý dữ liệu nguồn)

Đây là bước chuẩn bị "thư viện kiến thức" cho mô hình:

- **Product Catalog:** Nguồn dữ liệu thô (danh sách sản phẩm).
- **Text Processing:** Chuyển đổi dữ liệu thô thành dạng văn bản có cấu trúc.
- **MiniLM-L6 Embedding:** Sử dụng mô hình AI để chuyển văn bản thành các **Vector** (dãy số đại diện cho ý nghĩa của văn bản).
- **Normalize L2:** Chuẩn hóa các vector để đảm bảo tính toán khoảng cách được chính xác.
- **FAISS Index:** Lưu trữ các vector vào cơ sở dữ liệu vector chuyên dụng (FAISS), cho phép tìm kiếm nhanh trong tích tắc.

2. Giai đoạn Query (Thời gian thực - Phục vụ người dùng)

Đây là quá trình xử lý khi người dùng đặt câu hỏi:

- **User Query:** Câu hỏi từ phía người dùng.
- **Encode to Vector:** Chuyển câu hỏi của người dùng thành vector (cùng định dạng với bước Indexing).
- **FAISS Search (Top K):** So sánh vector câu hỏi với các vector sản phẩm trong FAISS để tìm ra những sản phẩm **liên quan nhất** (Top K).
- **Context Retrieval:** Lấy chi tiết thông tin của những sản phẩm vừa tìm được.
- **LLM Prompt Construction:** Đóng gói thông tin sản phẩm và câu hỏi của người dùng thành một "Prompt" (yêu cầu) hoàn chỉnh để gửi cho LLM.
- **Ollama LLM:** LLM (ví dụ Phi-3/Llama-3) sẽ đọc thông tin sản phẩm được cung cấp và câu hỏi của người dùng để đưa ra câu trả lời tự nhiên, chính xác.
- **Final Response:** Kết quả cuối cùng trả về cho người dùng.

3.6.2 Vector Database — FAISS

```
import os
import json
import pickle
import numpy as np
from pathlib import Path
from typing import Optional

try:
    import faiss
    FAISS_AVAILABLE = True
except ImportError:
    FAISS_AVAILABLE = False
    print("⚠️ faiss-cpu not installed, falling back to cosine search")

from sklearn.feature_extraction.text import TfidfVectorizer
```

```

class FAISSVectorStore:
    def __init__(self):
        self.index: Optional[object] = None # faiss index or None
        self.vectorizer: Optional[TfidfVectorizer] = None
        self.metadata: list[dict] = []
        self.vectors: Optional[np.ndarray] = None # fallback when faiss absent

    def build(self):
        """Build index from product descriptions."""
        texts = [p["text"] for p in PRODUCT_DESCRIPTIONS]
        self.metadata = PRODUCT_DESCRIPTIONS

        # TF-IDF embedding
        self.vectorizer = TfidfVectorizer(
            analyzer="word",
            ngram_range=(1, 2),
            max_features=512,
            sublinear_tf=True,
        )
        vecs = self.vectorizer.fit_transform(texts).toarray().astype(np.float32)

        # L2-normalize for cosine similarity
        norms = np.linalg.norm(vecs, axis=1, keepdims=True)
        norms[norms == 0] = 1
        vecs = vecs / norms
        self.vectors = vecs

        if FAISS_AVAILABLE:
            dim = vecs.shape[1]
            self.index = faiss.IndexFlatIP(dim) # Inner Product = cosine on normalized vecs
            self.index.add(vecs)
            # Save
            faiss.write_index(self.index, str(INDEX_PATH))
        else:
            self.index = None

        with open(META_PATH, "wb") as f:
            pickle.dump({
                "metadata": self.metadata,
                "vectorizer": self.vectorizer,
                "vectors": self.vectors,
            }, f)

        print(f"✅ FAISS index built: {len(texts)} products, dim={vecs.shape[1]}")

```

```

def load(self) -> bool:
    if not META_PATH.exists():
        return False
    try:
        with open(META_PATH, "rb") as f:
            data = pickle.load(f)
            self.metadata = data["metadata"]
            self.vectorizer = data["vectorizer"]
            self.vectors = data["vectors"]

            if FAISS_AVAILABLE and INDEX_PATH.exists():
                self.index = faiss.read_index(str(INDEX_PATH))
            print(f"✅ FAISS index loaded: {len(self.metadata)} products")
            return True
    except Exception as e:
        print(f"⚠️ FAISS load error: {e}")
        return False

```

```

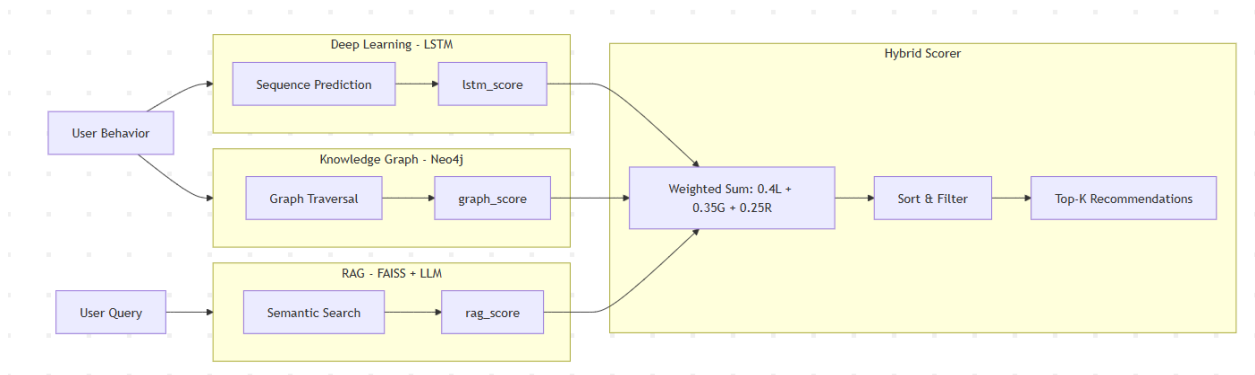
def search(self, query: str, top_k: int = 5) -> list[dict]:
    """Return top_k most similar products to query."""
    if self.vectorizer is None:
        return []
    try:
        q_vec = self.vectorizer.transform([query]).toarray().astype(np.float32)
        norm = np.linalg.norm(q_vec)
        if norm > 0:
            q_vec /= norm
        if FAISS_AVAILABLE and self.index is not None:
            scores, indices = self.index.search(q_vec, top_k)
            results = []
            for score, idx in zip(scores[0], indices[0]):
                if idx >= 0:
                    item = self.metadata[idx].copy()
                    item["vector_score"] = float(score)
                    results.append(item)
            return results
        else:
            # Fallback: numpy cosine
            sims = (self.vectors @ q_vec.T).squeeze()
            top_indices = sims.argsort()[::-1][:top_k]
            results = []
            for idx in top_indices:
                item = self.metadata[idx].copy()
                item["vector_score"] = float(sims[idx])
                results.append(item)
            return results
    except Exception as e:
        print(f"FAISS search error: {e}")
        return []
store: Optional[FAISSVectorStore] = None

```

Thành phần	Kỹ thuật	Trọng số	Mạnh ở
LSTM	Sequence prediction	40%	Dự đoán next item từ chuỗi hành vi
Graph (Neo4j)	Graph traversal	35%	Quan hệ sản phẩm, collaborative filtering
RAG (FAISS + LLM)	Semantic search	25%	Tìm kiếm ngữ nghĩa, chatbot

Bảng 3.2: Hybrid Model Components

3.7 Kết hợp thành model



1. Đầu vào (Inputs)

- **User Behavior (Hành vi người dùng):** Gồm lịch sử xem, giỏ hàng, và các tương tác trước đó. Đây là nguồn dữ liệu chính cho các mô hình AI.
- **User Query (Truy vấn người dùng):** Câu hỏi hoặc từ khóa tìm kiếm trực tiếp của người dùng.

2. Ba luồng xử lý chính (Three Pillars)

Hệ thống sử dụng ba phương pháp để tính toán "điểm số" (score) cho mỗi sản phẩm tiềm năng:

- **Deep Learning - LSTM (lstm_score):** Dựa trên trình tự thời gian. Mô hình này "học" được thói quen của người dùng (ví dụ: sau khi xem Laptop A, họ thường xem tiếp Chuột B), từ đó dự đoán sản phẩm tiếp theo trong chuỗi hành vi.

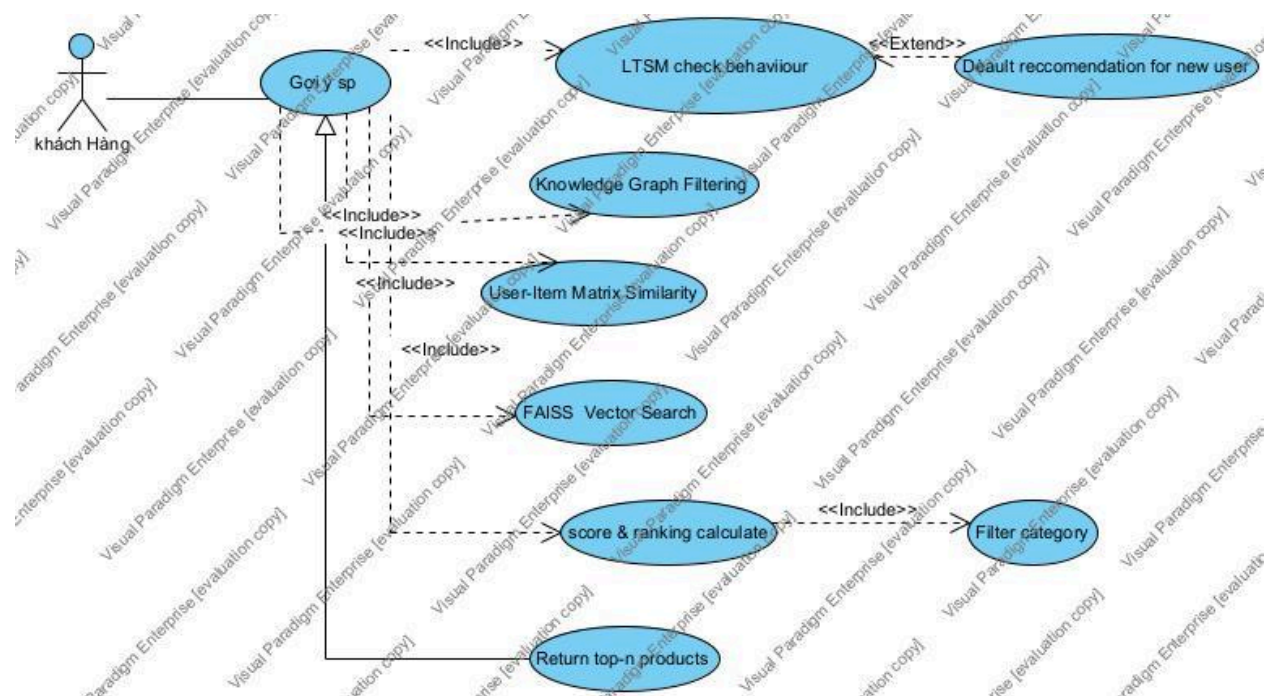
- **Knowledge Graph - Neo4j (graph_score):** Dựa trên mối quan hệ. Hệ thống sử dụng cơ sở dữ liệu đồ thị để tìm các sản phẩm có liên quan (ví dụ: các sản phẩm cùng danh mục, cùng nhà sản xuất hoặc được nhiều người dùng khác mua cùng nhau).
- **RAG - FAISS + LLM (rag_score):** Dựa trên ngữ nghĩa. Khi người dùng nhập câu truy vấn, mô hình sẽ tìm kiếm các sản phẩm phù hợp nhất về mặt ý nghĩa (Semantic Search) và dùng LLM để phân tích ngữ cảnh, đảm bảo kết quả đúng với ý định người dùng.

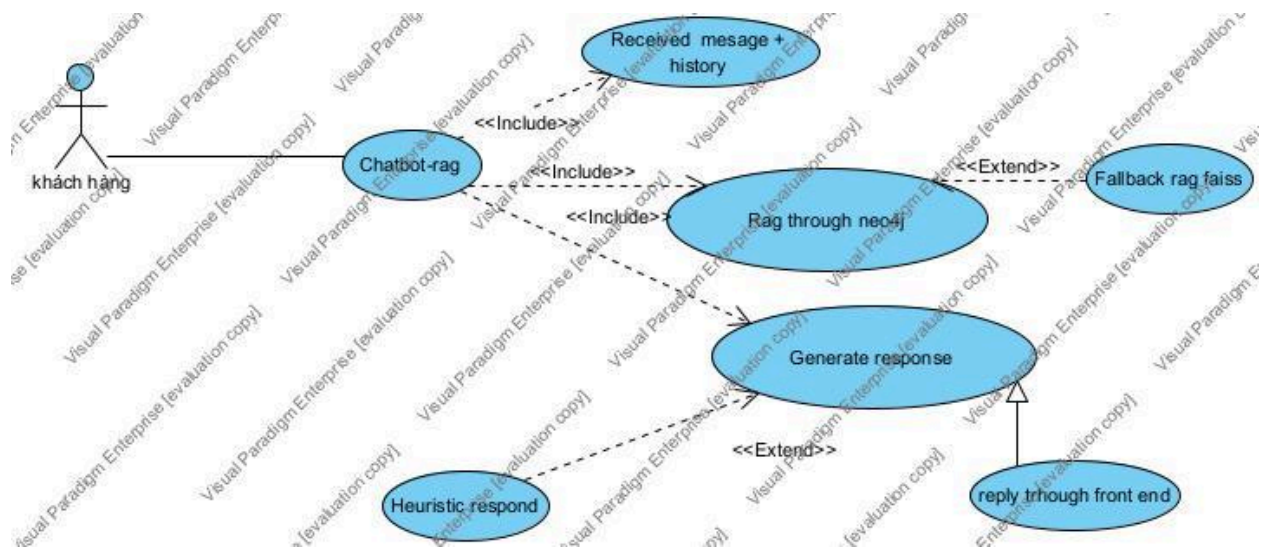
3. Bộ lọc kết hợp (Hybrid Scorer)

Đây là "bộ não" tổng hợp:

- **Weighted Sum (Tổng trọng số):** Hệ thống không tin hoàn toàn vào một mô hình duy nhất mà kết hợp chúng lại theo tỷ lệ: $0.4 \times \text{LSTM} + 0.35 \times \text{Graph} + 0.25 \times \text{RAG}$. Trọng số này có thể được điều chỉnh tùy theo hiệu quả thực tế của từng mô hình.
- **Sort & Filter (Sắp xếp & Lọc):** Sau khi tính điểm tổng, danh sách sản phẩm được sắp xếp theo điểm giảm dần và loại bỏ các sản phẩm không phù hợp hoặc đã hết hàng.
- **Top-K Recommendations:** Trả về K sản phẩm có điểm số cao nhất để hiển thị cho người dùng.

3.8 UC-AI

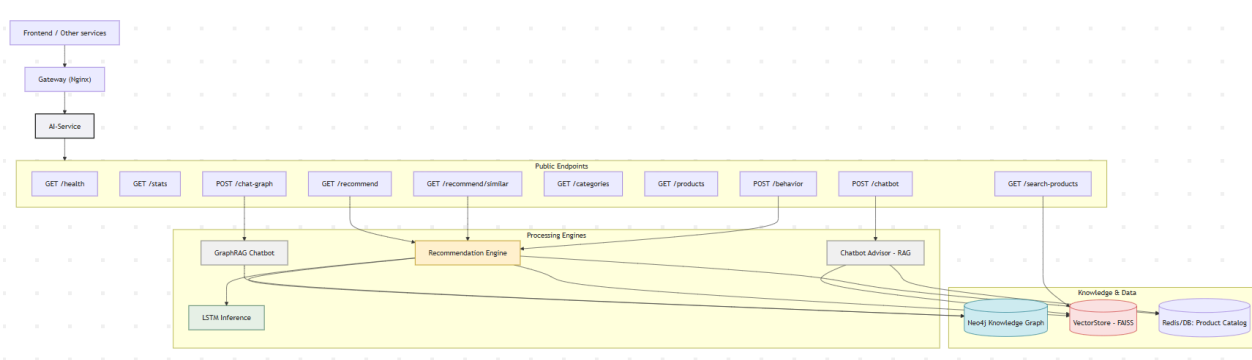




3.8 API Endpoints của AI Service

Method	Endpoint	Mô tả
GET	/recommend	Gợi ý sản phẩm theo user_id và category
GET	/recommend/similar	Sản phẩm tương tự theo product_id
POST	/behavior	Log hành vi người dùng
POST	/chatbot	Chatbot tư vấn (non-streaming)
POST	/chat-graph	Chatbot với Graph RAG context
GET	/search-products	Tìm kiếm sản phẩm cross-service

GET	/categories	Danh sách danh mục với số lượng sản phẩm
GET	/products	Tổng hợp sản phẩm cho frontend
GET	/health	Health check
GET	/stats	Thống kê: users, interactions, products



Bảng 3.3: API Endpoints của AI Service

3.9 Tech Stack và Triển khai

Thành phần	Công nghệ	Vai trò trong ShopMe
Framework	FastAPI (Python)	REST API, async endpoints, lifespan hooks
LSTM Model	PyTorch	Sequence recommendation model

Knowledge Graph	Neo4j	Product similarity, user interaction graph
Vector Search	FAISS / Cosine fallback	Semantic product search
Behavior Store	MongoDB	Lưu hành vi người dùng theo document
LLM	Ollama (phi3, llama3)	Chatbot response generation
Cache	Redis	Cache recommendation results
Containerization	Docker	Isolated deployment

Bảng 3.4: Tech Stack của AI Service

3.10 Kết luận Chương 3

AI Service của ShopMe áp dụng Hybrid AI kết hợp LSTM, Knowledge Graph và RAG để cá nhân hóa trải nghiệm mua sắm. Kiến trúc được thiết kế để hoạt động ổn định ngay cả khi một trong các thành phần (Neo4j, MongoDB, FAISS) không khả dụng — hệ thống tự động fallback về cosine similarity. Ollama cho phép chạy LLM hoàn toàn local, không phụ thuộc cloud API, đảm bảo privacy và không mất phí inference.

CHƯƠNG 4. Xây dựng Hệ thống ShopMe Hoàn chỉnh

4.1 Kiến trúc tổng thể

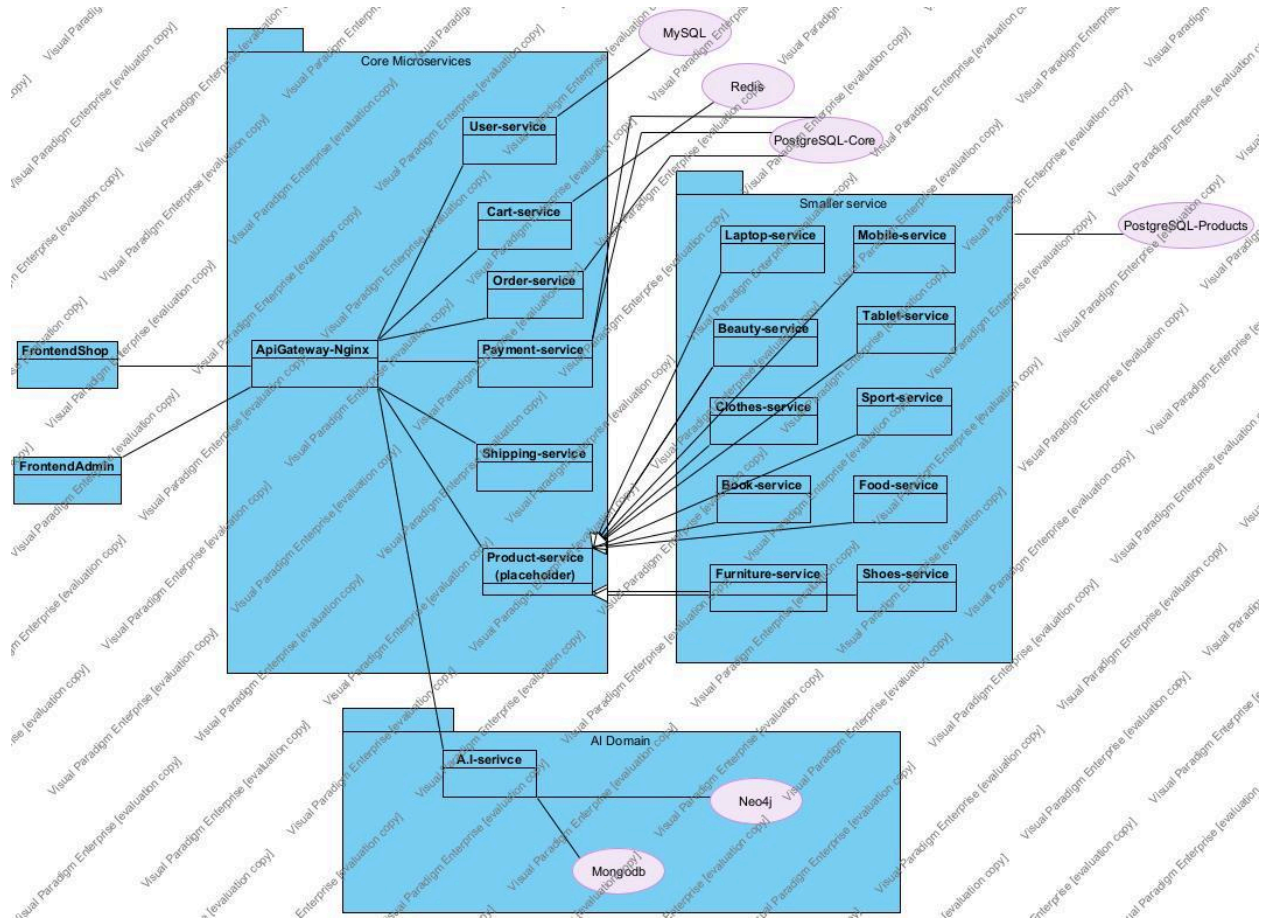
4.1.1 System Overview

ShopMe là hệ thống thương mại điện tử phân tán được xây dựng hoàn toàn theo kiến trúc microservices. Hệ thống gồm 25 container Docker, 17 service nghiệp vụ, 6 database riêng biệt, và một AI service tích hợp 3 kỹ thuật machine learning. Tất cả được orchestrate bằng Docker Compose, khởi động bằng một lệnh duy nhất.

Nhóm	Service	Số lượng	Port
Frontend	frontend-shop, frontend-admin	2	3000, 3001
API Gateway	gateway (Nginx)	1	80
Core services	user, cart, order, payment, shipping	5	8000–8004
AI Service	ai-service (FastAPI)	1	8006
Product services	laptop, mobile, tablet, clothes, shoes, book, food, beauty, furniture, sport	10	8010–8019
Databases	MySQL, PostgreSQL×2, Redis, MongoDB, Neo4j	6	3306, 5432–5433, 6379, 27017, 7474

Bảng 4.1: Tổng quan hệ thống ShopMe

4.1.2 Architecture Diagram



Hình 4.1: System Architecture Diagram — ShopMe

4.2 API Gateway (Nginx)

4.2.1 Vai trò

- Single entry point cho toàn bộ hệ thống — không có service nào expose trực tiếp ra ngoài
- Routing request đến đúng service dựa trên URL prefix
- Xác thực JWT token trước khi forward đến service
- Load balancing và timeout configuration
- Phục vụ frontend static files

4.2.2 Cấu hình nginx.conf

```
events {  
    worker_connections 1024;  
}  
  
http {  
    upstream user_service      { server user-service:8000; }  
    upstream cart_service     { server cart-service:8001; }  
    upstream order_service    { server order-service:8002; }  
    upstream payment_service  { server payment-service:8003; }  
    upstream shipping_service { server shipping-service:8004; }  
    upstream ai_service       { server ai-service:8006; }  
  
    upstream laptop_service   { server laptop-service:8010; }  
    upstream mobile_service   { server mobile-service:8011; }  
    upstream tablet_service   { server tablet-service:8012; }  
    upstream clothes_service  { server clothes-service:8013; }  
    upstream shoes_service    { server shoes-service:8014; }  
    upstream book_service     { server book-service:8015; }  
    upstream food_service     { server food-service:8016; }  
    upstream beauty_service   { server beauty-service:8017; }  
    upstream furniture_service { server furniture-service:8018; }  
    upstream sport_service    { server sport-service:8019; }  
  
    upstream frontend_shop    { server frontend-shop:80; }  
    upstream frontend_admin   { server frontend-admin:80; }
```

```
server {
    listen 80;
    server_name _;

    # CORS headers
    add_header 'Access-Control-Allow-Origin' '*' always;
    add_header 'Access-Control-Allow-Methods' 'GET, POST, PUT, DELETE, OPTIONS' always;
    add_header 'Access-Control-Allow-Headers' 'Authorization, Content-Type' always;

    # — Auth & Users —
    location /auth/ {
        proxy_pass http://user_service;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
    location /users/ {
        proxy_pass http://user_service;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # — Cart —
    location /cart/ {
        proxy_pass http://cart_service;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # — Orders —
    location /orders/ {
        proxy_pass http://order_service;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    # — Payment —
    location /payment/ {
        proxy_pass http://payment_service;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
}
```

```
# — Shipping —
location /shipping/ {
    proxy_pass http://shipping_service;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}

# — AI Service (extended timeout for LLM calls) —
location /recommend {
    proxy_pass http://ai_service;
    proxy_set_header Host $host;
    proxy_read_timeout 120s;
    proxy_send_timeout 120s;
    proxy_buffer_size 16k;
    proxy_buffers 4 32k;
}

location /chatbot {
    proxy_pass http://ai_service;
    proxy_set_header Host $host;
    proxy_read_timeout 120s;
    proxy_send_timeout 120s;
    proxy_buffer_size 16k;
    proxy_buffers 4 32k;
}

location /chat-graph {
    proxy_pass http://ai_service;
    proxy_set_header Host $host;
    proxy_read_timeout 120s;
    proxy_send_timeout 120s;
    proxy_buffer_size 16k;
    proxy_buffers 4 32k;
}

location /behavior {
    proxy_pass http://ai_service;
    proxy_set_header Host $host;
    proxy_read_timeout 30s;
}
```

```
location /categories {
    proxy_pass http://ai_service;
    proxy_set_header Host $host;
    proxy_read_timeout 15s;
}
location /products {
    proxy_pass http://ai_service;
    proxy_set_header Host $host;
    proxy_read_timeout 15s;
}
location /search-products {
    proxy_pass http://ai_service;
    proxy_set_header Host $host;
    proxy_read_timeout 30s;
}
location /health {
    proxy_pass http://ai_service;
    proxy_set_header Host $host;
    proxy_read_timeout 15s;
}
location /stats {
    proxy_pass http://ai_service;
    proxy_set_header Host $host;
    proxy_read_timeout 15s;
}
location /ai/ {
    proxy_pass http://ai_service;
    proxy_set_header Host $host;
    proxy_read_timeout 120s;
}

# — Product Services —
location /laptops/ {
    proxy_pass http://laptop_service;
    proxy_set_header Host $host;
}
location /mobiles/ {
    proxy_pass http://mobile_service;
    proxy_set_header Host $host;
}
```

```
location /tablets/ {
    proxy_pass http://tablet_service;
    proxy_set_header Host $host;
}
location /clothes/ {
    proxy_pass http://clothes_service;
    proxy_set_header Host $host;
}
location /shoes/ {
    proxy_pass http://shoes_service;
    proxy_set_header Host $host;
}
location /books/ {
    proxy_pass http://book_service;
    proxy_set_header Host $host;
}
location /foods/ {
    proxy_pass http://food_service;
    proxy_set_header Host $host;
}
location /beauty/ {
    proxy_pass http://beauty_service;
    proxy_set_header Host $host;
}
location /furniture/ {
    proxy_pass http://furniture_service;
    proxy_set_header Host $host;
}
location /sports/ {
    proxy_pass http://sport_service;
    proxy_set_header Host $host;
}

# — Frontends —
location /admin/ {
    proxy_pass http://frontend_admin/;
    proxy_set_header Host $host;
}
location / {
    proxy_pass http://frontend_shop;
    proxy_set_header Host $host;
}
```


4.3 Authentication — JWT- Luồng xác thực

- Client login → nhận access_token và refresh_token
- Mọi request sau đó gửi kèm header: Authorization: Bearer <token>
- Gateway verify token qua user-service trước khi forward
- Sau 1 giờ, dùng refresh_token để lấy access_token mới

4.4 Giao tiếp giữa các Service

- Timeout: mọi outbound call đặt timeout 10–30 giây
- Retry: tự động retry 3 lần với exponential backoff cho critical calls
- Graceful degradation: AI service fallback về heuristic khi model không sẵn sàng
- Health check: mỗi service có /health endpoint, Docker healthcheck kiểm tra định kỳ

4.5 Docker hóa hệ thống

4.5.1 Dockerfile — Django services

- 1 Dockerfile cho từng service

4.5.2 docker-compose.yml — Cấu trúc

```
# ----- FRONTENDS -----
▷Run Service
frontend-shop:
  build: ./frontend-shop
  container_name: shopme-frontend-shop
  ports:
    - "3000:80"
  networks:
    - shopme-net
  restart: unless-stopped

▷Run Service
frontend-admin:
  build: ./frontend-admin
  container_name: shopme-frontend-admin
  ports:
    - "3001:80"
  networks:
    - shopme-net
  restart: unless-stopped
```

```
# ----- CORE SERVICES -----
```

```
▷Run Service
```

```
user-service:
```

```
  build: ./user-service
```

```
  container_name: shopme-user
```

```
  ports:
```

```
    - "8000:8000"
```

```
  environment:
```

```
    - DB_HOST=db-mysql
```

```
    - DB_NAME=shopme_users
```

```
    - DB_USER=shopme
```

```
    - DB_PASS=shopme123
```

```
    - SECRET_KEY=shopme-user-secret-key-2026
```

```
    - DEBUG=1
```

```
  depends_on:
```

```
    db-mysql:
```

```
      condition: service_healthy
```

```
  networks:
```

```
    - shopme-net
```

```
  restart: unless-stopped
```

```
▷Run Service
```

```
cart-service:
```

```
  build: ./cart-service
```

```
  container_name: shopme-cart
```

```
  ports:
```

```
    - "8001:8001"
```

```
  environment:
```

```
    - REDIS_HOST=db-redis
```

```
    - REDIS_PORT=6379
```

```
    - SECRET_KEY=shopme-cart-secret-key-2026
```

```
    - DEBUG=1
```

```
  depends_on:
```

```
    db-redis:
```

```
      condition: service_healthy
```

```
  networks:
```

```
    - shopme-net
```

```
  restart: unless-stopped
```

▷ Run Service

```
order-service:
  build: ./order-service
  container_name: shopme-order
  ports:
    - "8002:8002"
  environment:
    - DB_HOST=db-postgres-core
    - DB_NAME=shopme_orders
    - DB_USER=shopme
    - DB_PASS=shopme123
    - SECRET_KEY=shopme-order-secret-key-2026
    - USER_SERVICE_URL=http://user-service:8000
    - PAYMENT_SERVICE_URL=http://payment-service:8003
    - SHIPPING_SERVICE_URL=http://shipping-service:8004
    - DEBUG=1
  depends_on:
    db-postgres-core:
      condition: service_healthy
  networks:
    - shopme-net
  restart: unless-stopped
```

▷ Run Service

```
payment-service:
  build: ./payment-service
  container_name: shopme-payment
  ports:
    - "8003:8003"
  environment:
    - DB_HOST=db-postgres-core
    - DB_NAME=shopme_payments
    - DB_USER=shopme
    - DB_PASS=shopme123
    - SECRET_KEY=shopme-payment-secret-key-2026
    - ORDER_SERVICE_URL=http://order-service:8002
    - DEBUG=1
  depends_on:
    db-postgres-core:
      condition: service_healthy
  networks:
    - shopme-net
  restart: unless-stopped
```

▷Run Service

```
shipping-service:
  build: ./shipping-service
  container_name: shopme-shipping
  ports:
    - "8004:8004"
  environment:
    - DB_HOST=db-postgres-core
    - DB_NAME=shopme_shipping
    - DB_USER=shopme
    - DB_PASS=shopme123
    - SECRET_KEY=shopme-shipping-secret-key-2026
    - DEBUG=1
  depends_on:
    db-postgres-core:
      condition: service_healthy
  networks:
    - shopme-net
  restart: unless-stopped
```

AI SERVICE

▷Run Service

```
ai-service:
  build: ./ai-service
  container_name: shopme-ai
  ports:
    - "8006:8006"
  env_file:
    - .env
  environment:
    - MONGO_URI=mongodb://db-mongo:27017/shopme_ai
    - OLLAMA_URL=http://host.docker.internal:11434
    - NEO4J_URI=neo4j+s://a31b1e2e.databases.neo4j.io
    - NEO4J_USERNAME=a31b1e2e
    - NEO4J_PASSWORD=6-iZf1N0cz6TrRXLkRCGGRH3eKaAzzYl6rogA8rvCKk
    - NEO4J_DATABASE=a31b1e2e
    - AURA_INSTANCEID=a31b1e2e
    - AURA_INSTANCENAME=ShopMe
    - GRAPH_RAG_ENABLED=true
    - OLLAMA_MODEL=${OLLAMA_MODEL:-phi3}
    - DEBUG=1
  depends_on:
    db-mongo:
      condition: service_healthy
    db-neo4j:
      condition: service_started
  networks:
    - shopme-net
  extra_hosts:
    - "host.docker.internal:host-gateway"
  restart: unless-stopped
```

```
# PRODUCT SERVICES
```

```
▷ Run Service
```

```
laptop-service:
```

```
  build: ./product-services/laptop-service
```

```
  container_name: shopme-laptop
```

```
  ports:
```

```
  - "8010:8010"
```

```
  environment:
```

```
  - DB_HOST=db-postgres-products
```

```
  - DB_NAME=shopme_laptop
```

```
  - DB_USER=shopme
```

```
  - DB_PASS=shopme123
```

```
  - SECRET_KEY=shopme-laptop-secret-2026
```

```
  - SERVICE_PORT=8010
```

```
  - DEBUG=1
```

```
  depends_on:
```

```
    db-postgres-products:
```

```
      condition: service_healthy
```

```
  networks:
```

```
  - shopme-net
```

```
  restart: unless-stopped
```

```
▷ Run Service
```

```
mobile-service:
```

```
  build: ./product-services/mobile-service
```

```
  container_name: shopme-mobile
```

```
  ports:
```

```
  - "8011:8011"
```

```
  environment:
```

```
  - DB_HOST=db-postgres-products
```

```
  - DB_NAME=shopme_mobile
```

```
  - DB_USER=shopme
```

```
  - DB_PASS=shopme123
```

```
  - SECRET_KEY=shopme-mobile-secret-2026
```

```
  - SERVICE_PORT=8011
```

```
  - DEBUG=1
```

```
  depends_on:
```

```
    db-postgres-products:
```

```
      condition: service_healthy
```

```
  networks:
```

```
  - shopme-net
```

```
  restart: unless-stopped
```

```
▷ Run Service
```

```
tablet-service:
```

```
  build: ./product-services/tablet-service
```

```
  container_name: shopme-tablet
```

```
  ports:
```

```
  - "8012:8012"
```

```
  environment:
```

```
  - DB_HOST=db-postgres-products
```

```
  - DB_NAME=shopme_tablet
```

```
  - DB_USER=shopme
```

```
  - DB_PASS=shopme123
```

```
  - SECRET_KEY=shopme-tablet-secret-2026
```

```
  - SERVICE_PORT=8012
```

```
  - DEBUG=1
```

```
  depends_on:
```

```
    db-postgres-products:
```

```
      condition: service_healthy
```

```
  networks:
```

```
  - shopme-net
```

```
  restart: unless-stopped
```

```
clothes-service:
  build: ./product-services/clothes-service
  container_name: shopme-clothes
  ports:
    - "8013:8013"
  environment:
    - DB_HOST=db-postgres-products
    - DB_NAME=shopme_clothes
    - DB_USER=shopme
    - DB_PASS=shopme123
    - SECRET_KEY=shopme-clothes-secret-2026
    - SERVICE_PORT=8013
    - DEBUG=1
  depends_on:
    db-postgres-products:
      condition: service_healthy
  networks:
    - shopme-net
  restart: unless-stopped
```

▷ Run Service

```
shoes-service:
  build: ./product-services/shoes-service
  container_name: shopme-shoes
  ports:
    - "8014:8014"
  environment:
    - DB_HOST=db-postgres-products
    - DB_NAME=shopme_shoes
    - DB_USER=shopme
    - DB_PASS=shopme123
    - SECRET_KEY=shopme-shoes-secret-2026
    - SERVICE_PORT=8014
    - DEBUG=1
  depends_on:
    db-postgres-products:
      condition: service_healthy
  networks:
    - shopme-net
  restart: unless-stopped
```

▷ Run Service

```
book-service:
  build: ./product-services/book-service
  container_name: shopme-book
  ports:
    - "8015:8015"
  environment:
    - DB_HOST=db-postgres-products
    - DB_NAME=shopme_book
    - DB_USER=shopme
    - DB_PASS=shopme123
    - SECRET_KEY=shopme-book-secret-2026
    - SERVICE_PORT=8015
    - DEBUG=1
  depends_on:
    db-postgres-products:
      condition: service_healthy
  networks:
    - shopme-net
  restart: unless-stopped
```

↳ Run Service

```
food-service:
  build: ./product-services/food-service
  container_name: shopme-food
  ports:
    - "8016:8016"
  environment:
    - DB_HOST=db-postgres-products
    - DB_NAME=shopme_food
    - DB_USER=shopme
    - DB_PASS=shopme123
    - SECRET_KEY=shopme-food-secret-2026
    - SERVICE_PORT=8016
    - DEBUG=1
  depends_on:
    db-postgres-products:
      condition: service_healthy
  networks:
    - shopme-net
  restart: unless-stopped
```

↳ Run Service

```
beauty-service:
  build: ./product-services/beauty-service
  container_name: shopme-beauty
  ports:
    - "8017:8017"
  environment:
    - DB_HOST=db-postgres-products
    - DB_NAME=shopme_beauty
    - DB_USER=shopme
    - DB_PASS=shopme123
    - SECRET_KEY=shopme-beauty-secret-2026
    - SERVICE_PORT=8017
    - DEBUG=1
  depends_on:
    db-postgres-products:
      condition: service_healthy
  networks:
    - shopme-net
  restart: unless-stopped
```

▷ Run Service

```
furniture-service:
  build: ./product-services/furniture-service
  container_name: shopme-furniture
  ports:
    - "8018:8018"
  environment:
    - DB_HOST=db-postgres-products
    - DB_NAME=shopme_furniture
    - DB_USER=shopme
    - DB_PASS=shopme123
    - SECRET_KEY=shopme-furniture-secret-2026
    - SERVICE_PORT=8018
    - DEBUG=1
  depends_on:
    db-postgres-products:
      condition: service_healthy
  networks:
    - shopme-net
  restart: unless-stopped
```

▷ Run Service

```
sport-service:
  build: ./product-services/sport-service
  container_name: shopme-sport
  ports:
    - "8019:8019"
  environment:
    - DB_HOST=db-postgres-products
    - DB_NAME=shopme_sport
    - DB_USER=shopme
    - DB_PASS=shopme123
    - SECRET_KEY=shopme-sport-secret-2026
    - SERVICE_PORT=8019
    - DEBUG=1
  depends_on:
    db-postgres-products:
      condition: service_healthy
  networks:
    - shopme-net
  restart: unless-stopped
```



```
db-mysql:
  image: mysql:8
  container_name: shopme-db-mysql
  environment:
    - MYSQL_ROOT_PASSWORD=root123
    - MYSQL_DATABASE=shopme_users
    - MYSQL_USER=shopme
    - MYSQL_PASSWORD=shopme123
  ports:
    - "3306:3306"
  volumes:
    - mysql_data:/var/lib/mysql
  networks:
    - shopme-net
  restart: unless-stopped
  healthcheck:
    test: ["CMD", "mysqladmin", "ping", "-h", "localhost", "-u", "shopme", "-pshopme123"]
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 30s
```

>Run Service

```
db-postgres-products:
  image: postgres:15
  container_name: shopme-db-postgres-products
  environment:
    - POSTGRES_USER=shopme
    - POSTGRES_PASSWORD=shopme123
    - POSTGRES_DB=shopme_products
  ports:
    - "5433:5432"
  volumes:
    - postgres_products_data:/var/lib/postgresql/data
    - ./init-scripts/init-product-dbs.sql:/docker-entrypoint-initdb.d/init.sql:ro
  networks:
    - shopme-net
  restart: unless-stopped
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U shopme -d shopme_products"]
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 20s
```

▷Run Service

db-postgres-core:

image: postgres:15

container_name: shopme-db-postgres-core

environment:

- POSTGRES_USER=shopme
- POSTGRES_PASSWORD=shopme123
- POSTGRES_DB=shopme_core

ports:

- "5432:5432"

volumes:

- postgres_core_data:/var/lib/postgresql/data
- ./init-scripts/init-core-dbs.sql:/docker-entrypoint-initdb.d/init.sql:ro

networks:

- shopme-net

restart: unless-stopped

healthcheck:

test: ["CMD-SHELL", "pg_isready -U shopme -d shopme_core"]
interval: 10s
timeout: 5s
retries: 10
start_period: 20s

▷Run Service

db-redis:

image: redis:7-alpine

container_name: shopme-db-redis

ports:

- "6379:6379"

volumes:

- redis_data:/data

networks:

- shopme-net

restart: unless-stopped

healthcheck:

test: ["CMD", "redis-cli", "ping"]
interval: 10s
timeout: 5s
retries: 10
start_period: 10s

```
db-mongo:
  image: mongo:7
  container_name: shopme-db-mongo
  ports:
    - "27017:27017"
  volumes:
    - mongo_data:/data/db
  networks:
    - shopme-net
  restart: unless-stopped
  healthcheck:
    test: ["CMD", "mongosh", "--eval", "db.adminCommand('ping')"]
    interval: 10s
    timeout: 5s
    retries: 10
    start_period: 20s
```

```
# ----- NEO4J (Knowledge Graph) -----
> Run Service
db-neo4j:
  image: neo4j:5.18-community
  container_name: shopme-db-neo4j
  ports:
    - "7474:7474"
    - "7687:7687"
  environment:
    - NEO4J_AUTH=neo4j/shopme123
    - NEO4J_PLUGINS=[]
    - NEO4J_dbms_memory_pagecache_size=256M
    - NEO4J_dbms_memory_heap_initial_size=256M
    - NEO4J_dbms_memory_heap_max_size=512M
  volumes:
    - neo4j_data:/data
    - neo4j_logs:/logs
  networks:
    - shopme-net
  restart: unless-stopped

networks:
  shopme-net:
    driver: bridge

volumes:
  mysql_data:
  postgres_products_data:
  postgres_core_data:
  redis_data:
  mongo_data:
  neo4j_data:
  neo4j_logs:
```

4.6 Triển khai và Vận hành

4.6.1. Triển khai

Khởi động lần đầu

```
PS D:\shopme> cd D:/
PS D:\> git pull origin main
>> docker-compose up -d --build
```

Chạy migration cho các Django service:

```
PS D:\shopme> docker-compose exec user-service python manage.py migrate
>> docker-compose exec order-service python manage.py migrate
>> docker-compose exec payment-service python manage.py migrate
```

Kiểm tra container:

```
PS D:\shopme> docker-compose ps
>> docker-compose logs -f ai-service
```

Khởi động:

```
PS D:\shopme> docker-compose up -d
```

Chạy lại 1 service nhất định:

```
PS D:\shopme> docker-compose restart payment-service
```

Dừng toàn bộ:

```
PS D:\shopme> docker-compose down
```

4.6.2. Kiểm tra

Health Checks

```
PS D:\shopme> docker-compose ps
>> docker-compose logs --tail=200 ai-service
```

Kiểm tra service:

```
PS D:\shopme> curl http://localhost/health
```

4.7 Minh chứng dự án

Cấu trúc thư mục:

D:.

```
├─ai-service
|   ├─chatbot
|   |   └─__pycache__
|   ├─data
|   ├─ml_pipeline
|   |   ├─evidence
|   |   └─outputs
|   |   └─__pycache__
|   └─recommendation
|       └─__pycache__
|   └─routers
|       └─__pycache__
|   └─__pycache__
|   └─
├─cart-service
|   └─cart
|       └─management
|           └─commands
|               └─__pycache__
```

| | | $\hookleftarrow$ __pycache__
| | $\hookleftarrow$ __pycache__
| $\vdash$ config
| | $\hookleftarrow$ __pycache__
| $\hookleftarrow$ __pycache__
 $\vdash$ Diagram
 $\vdash$ frontend-admin
 $\vdash$ frontend-shop
 $\vdash$ gateway
 $\vdash$ init-scripts
 $\vdash$ order-service
| $\vdash$ config
| | $\hookleftarrow$ __pycache__
| $\vdash$ orders
| | $\vdash$ management
| | | $\vdash$ commands
| | | | $\hookleftarrow$ __pycache__
| | | $\hookleftarrow$ __pycache__
| | $\vdash$ migrations
| | | $\hookleftarrow$ __pycache__
| | $\hookleftarrow$ __pycache__
| $\hookleftarrow$ __pycache__
 $\vdash$ Para
 $\vdash$ payment-service
| $\vdash$ config
| | $\hookleftarrow$ __pycache__
| $\vdash$ payments
| | $\vdash$ migrations
| | | $\hookleftarrow$ __pycache__
| | $\hookleftarrow$ __pycache__
| $\hookleftarrow$ __pycache__
 $\vdash$ product-services
| $\vdash$ beauty-service
| | $\vdash$ config
| | | $\hookleftarrow$ __pycache__

| | ├──products
| | | ├──management
| | | | ├──commands
| | | | | └──__pycache__
| | | | └──__pycache__
| | | ├──migrations
| | | | └──__pycache__
| | | └──__pycache__
| | └──__pycache__
| ├──book-service
| | ├──config
| | | └──__pycache__
| | ├──products
| | | ├──management
| | | | ├──commands
| | | | | └──__pycache__
| | | | └──__pycache__
| | | ├──migrations
| | | | └──__pycache__
| | | └──__pycache__
| | └──__pycache__
| ├──clothes-service
| | ├──config
| | | └──__pycache__
| | ├──products
| | | ├──management
| | | | ├──commands
| | | | | └──__pycache__
| | | | └──__pycache__
| | | ├──migrations
| | | | └──__pycache__
| | | └──__pycache__
| | └──__pycache__
| ├──food-service
| | ├──config

| | | $\hookleftarrow$ _pycache_
| | $\vdash$ products
| | | $\vdash$ management
| | | | $\vdash$ commands
| | | | | $\hookleftarrow$ _pycache_
| | | | $\hookleftarrow$ _pycache_
| | | $\vdash$ migrations
| | | | $\hookleftarrow$ _pycache_
| | | $\hookleftarrow$ _pycache_
| | $\hookleftarrow$ _pycache_
| $\vdash$ furniture-service
| | $\vdash$ config
| | | $\hookleftarrow$ _pycache_
| | $\vdash$ products
| | | $\vdash$ management
| | | | $\vdash$ commands
| | | | | $\hookleftarrow$ _pycache_
| | | | $\hookleftarrow$ _pycache_
| | | $\vdash$ migrations
| | | | $\hookleftarrow$ _pycache_
| | | $\hookleftarrow$ _pycache_
| | $\hookleftarrow$ _pycache_
| $\vdash$ laptop-service
| | $\vdash$ config
| | | $\hookleftarrow$ _pycache_
| | $\vdash$ products
| | | $\vdash$ management
| | | | $\vdash$ commands
| | | | | $\hookleftarrow$ _pycache_
| | | | $\hookleftarrow$ _pycache_
| | | $\vdash$ migrations
| | | | $\hookleftarrow$ _pycache_
| | | $\hookleftarrow$ _pycache_
| | $\hookleftarrow$ _pycache_
| $\vdash$ mobile-service

| | ├──config
| | | └─__pycache__
| | ├──products
| | | ├──management
| | | | ├──commands
| | | | | └─__pycache__
| | | | └─__pycache__
| | | ├──migrations
| | | | └─__pycache__
| | | └─__pycache__
| | └─__pycache__
| ├──shoes-service
| | ├──config
| | | └─__pycache__
| | ├──products
| | | ├──management
| | | | ├──commands
| | | | | └─__pycache__
| | | | └─__pycache__
| | | ├──migrations
| | | | └─__pycache__
| | | └─__pycache__
| | └─__pycache__
| ├──sport-service
| | ├──config
| | | └─__pycache__
| | ├──products
| | | ├──management
| | | | ├──commands
| | | | | └─__pycache__
| | | | └─__pycache__
| | | ├──migrations
| | | | └─__pycache__
| | | └─__pycache__
| | └─__pycache__

- | |─tablet-service
 - | | |─config
 - | | | |─__pycache__
 - | | |─products
 - | | | |─management
 - | | | | |─commands
 - | | | | | |─__pycache__
 - | | | | |─__pycache__
 - | | |─migrations
 - | | | |─__pycache__
 - | | |─__pycache__
- | | |─__pycache__

- | |─shipping-service
 - | | |─config
 - | | | |─__pycache__
 - | | |─shipping
 - | | |─migrations
 - | | | |─__pycache__
 - | | |─__pycache__
- | | |─__pycache__

- | |─user-service
 - | | |─config
 - | | | |─__pycache__
 - | | |─users
 - | | | |─management
 - | | | | |─commands
 - | | | | | |─__pycache__
 - | | | | |─__pycache__
 - | | |─migrations
 - | | | |─__pycache__
 - | | |─__pycache__
- | | |─__pycache__

✓ SHOPME



> ai-service

> cart-service

> Diagram

> frontend-admin

> frontend-shop

> gateway

> init-scripts

> order-service

> Para

> payment-service

> product-services

> shipping-service

> user-service

⚙ .env

⚙ .env.example

🐳 docker-compose.yml

ℹ README.md

Docker:

Containers [Give feedback](#)

Container CPU usage

35.47% / 1600% (16 CPUs available)

Container memory usage

2.77GB / 6.54GB

[Show charts](#)

Q

Search

Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Memory usage...	Memory (%)	Disk read/write	Network I/O	Actions
☐	shopme	-	-	-	35.47%	2.77GB / 167.42GB	41.46%	795.21MB / 23.8MB	589.4KB / 544.05KB	
☐	gateway	d232e1766290	nginx:alpine	80:80	0%	3.19MB / 6.7GB	0.05%	36.9KB / 4.1KB	6.38KB / 3.94KB	
☐	order	802997aa7cd8	shopme-order	8002:8002	2.02%	76.89MB / 6.7GB	1.12%	1.54MB / 0B	16.2KB / 10.9KB	
☐	shipping	e7018aa69dd6	shopme-shipping	8004:8004	2.55%	70.6MB / 6.7GB	1.03%	2MB / 0B	15.8KB / 10.6KB	
☐	mobile	bd949ff9fde5	shopme-mobile	8011:8011	1.29%	71.74MB / 6.7GB	1.05%	2.38MB / 0B	24KB / 14.5KB	
☐	clothes	636326ffa999	shopme-clothes	8013:8013	1.33%	71.36MB / 6.7GB	1.04%	1.51MB / 0B	23.9KB / 14.6KB	
☐	sport	62a8bf938173	shopme-sport	8019:8019	1.48%	70.92MB / 6.7GB	1.03%	983KB / 0B	23.6KB / 14.5KB	
☐	laptop	ef49adda6767	shopme-laptop	8010:8010	1.34%	89.06MB / 6.7GB	1.3%	19.5MB / 0B	21.9KB / 14.4KB	
☐	tablet	210ab0ba9733	shopme-tablet	8012:8012	1.66%	71.98MB / 6.7GB	1.05%	2.25MB / 0B	21.2KB / 14.4KB	
☐	beauty	9b2bcb01f19b	shopme-beauty	8017:8017	1.35%	70.65MB / 6.7GB	1.03%	983KB / 0B	23.1KB / 14.6KB	
☐	payment	c7e05dcee3c2	shopme-payment	8003:8003	1.99%	75.89MB / 6.7GB	1.11%	414KB / 0B	16.1KB / 10.8KB	
☐	book	321eae7c896d	shopme-book	8015:8015	1.21%	75.12MB / 6.7GB	1.1%	6.66MB / 0B	23.7KB / 13.9KB	
☐	ai	4d77e2ee3c28	shopme-ai	8006:8006	0.14%	247.3MB / 6.7GB	3.61%	198MB / 0B	40.3KB / 34.6KB	
☐	food	0e2c106c1f60	shopme-food	8016:8016	1.42%	79.64MB / 6.7GB	1.16%	11.1MB / 0B	24.3KB / 13.9KB	
☐	furniture	7a8a8d5054ee	shopme-furniture	8018:8018	2.95%	70.68MB / 6.7GB	1.03%	983KB / 0B	22.7KB / 14.5KB	
☐	shoes	ae75133499f8	shopme-shoes	8014:8014	1.52%	71.16MB / 6.7GB	1.04%	1.45MB / 0B	23.9KB / 14.6KB	

Showing 26 items

Q

Search

Delete

Space to be reclaimed 2.7 GB

Selected 6 of 24

Volumes [Give feedback](#)

Search



Create

☐	Name ↑	Created	Size	Actions
☐	a4b849e2bb8174c6eb010d2c92e5aff605404282f098502e3a50e6b6bc7143f4	1 day ago	0 Bytes	
☐	shopme_mongo_data	1 day ago	301.3 MB	
☐	shopme_mysql_data	1 day ago	209.3 MB	
☐	shopme_neo4j_data	1 day ago	515.9 MB	
☐	shopme_neo4j_logs	1 day ago	181.8 kB	
☐	shopme_postgres_core_data	1 day ago	84.9 MB	
☐	shopme_postgres_products_data	1 day ago	171.6 MB	
☐	shopme_redis_data	1 day ago	0 Bytes	

Build history [Active builds](#)

Search



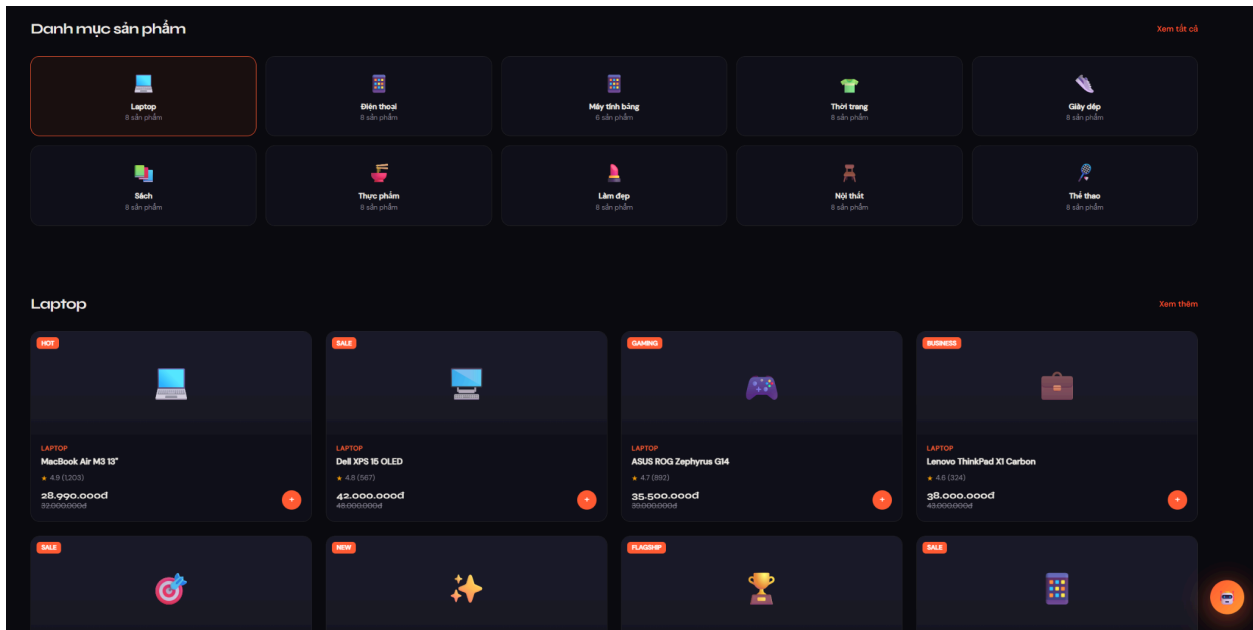
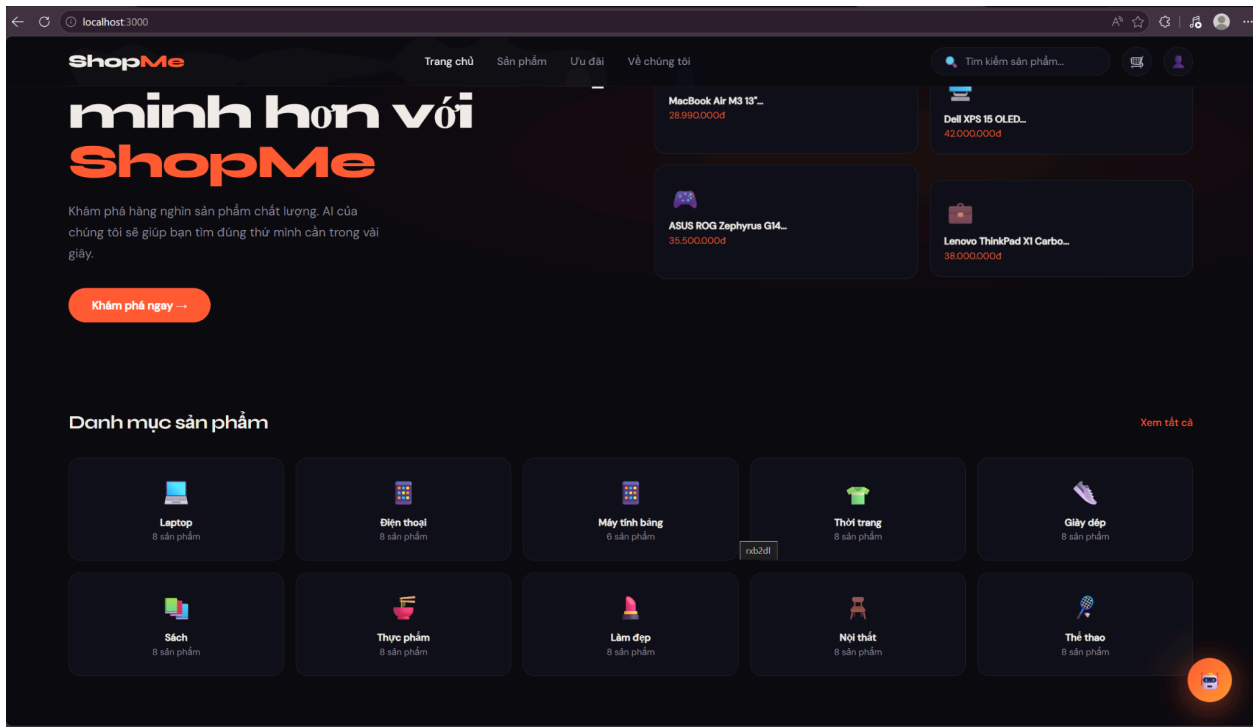
Show only my builds

☐	Name	ID	Builder	Duration	Created	Author
☐	book-service	qxpqg5	desktop-linux	46.5s	1 day ago	N/A
☐	food-service	rx2dl	desktop-linux	46.5s	1 day ago	N/A
☐	payment-service	re1jv	desktop-linux	48.0s	1 day ago	N/A
☐	sport-service	scv1e	desktop-linux	46.5s	1 day ago	N/A
☐	tablet-service	t80scs	desktop-linux	46.5s	1 day ago	N/A
☐	clothes-service	tihc8k	desktop-linux	46.5s	1 day ago	N/A
☐	shipping-service	pihqm1	desktop-linux	40.2s	1 day ago	N/A
☐	furniture-service	yaceau	desktop-linux	46.4s	1 day ago	N/A
☐	laptop-service	itjbt6	desktop-linux	46.5s	1 day ago	N/A
☐	beauty-service	iu2ror	desktop-linux	46.5s	1 day ago	N/A
☐	user-service	pbd02a	desktop-linux	1m 24s	1 day ago	N/A
☐	ai-service	wjspue	desktop-linux	3m 32s	1 day ago	N/A
☐	cart-service	s6nu1z	desktop-linux	37.7s	1 day ago	N/A
☐	mobile-service	f7a6al	desktop-linux	46.4s	1 day ago	N/A
☐	shoes-service	k6f027	desktop-linux	46.5s	1 day ago	N/A
☐	frontend-admin	ka6lhx	desktop-linux	1.8s	1 day ago	N/A

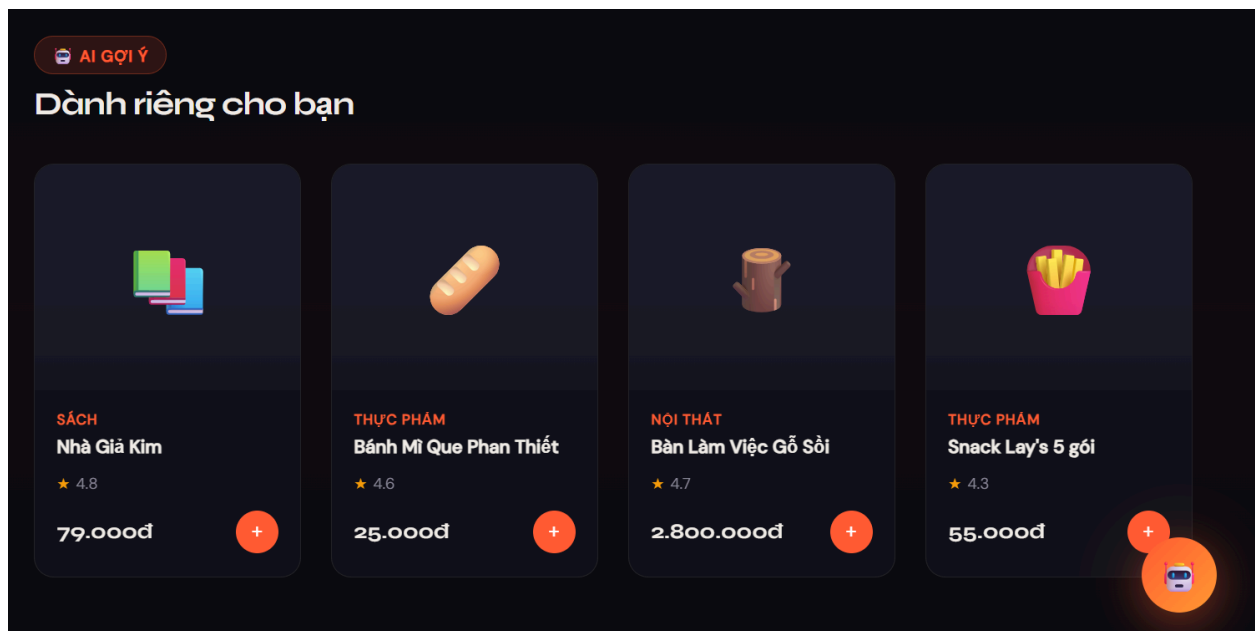
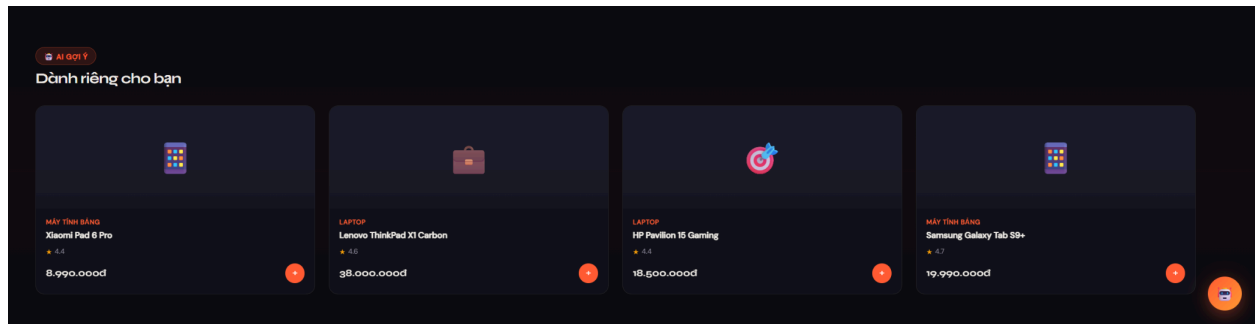
Rows per page: 100

1-18 of 18

Giao diện khách hàng:

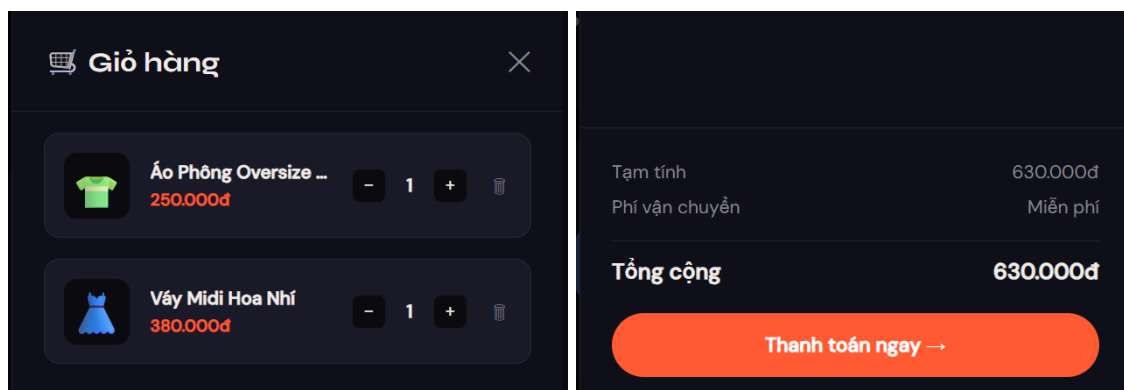


A.I gợi ý (thay đổi mỗi lần mua hàng/refresh):



(hệ thống thay đổi sau 1 lần thanh toán)

Giỏ hàng:



Thanh toán:

Thông tin giao hàng

Thông tin

Thanh toán

Xác nhận

HỌ VÀ TÊN

tin

SỐ ĐIỆN THOẠI

094037294

THÀNH PHỐ

hà nội

ĐỊA CHỈ GIAO HÀNG

w3

Hủy

Tiếp theo →

Chọn thanh toán

✓ Thông tin

Thanh toán

Xác nhận

Chọn hình thức thanh toán phù hợp với bạn

Tiền mặt

Chuyển khoản

Thẻ tín dụng

MoMo

← Quay lại

Xác nhận →

Hệ thống chatbot:

ShopMe AI

• Đang hoạt động

Graph

Xin chào! 🙌 Tôi là ShopMe AI. Tôi có thể giúp bạn tìm sản phẩm phù hợp, so sánh giá, hoặc tư vấn theo nhu cầu của bạn. Bạn cần tìm gì hôm nay?

có laptop không

Gợi ý: MacBook Air M3 (28,990,000đ) hoặc HP Pavilion Gaming (18,500,000đ). Bạn cần dùng cho công việc hay gaming?

Laptop tốt nhất

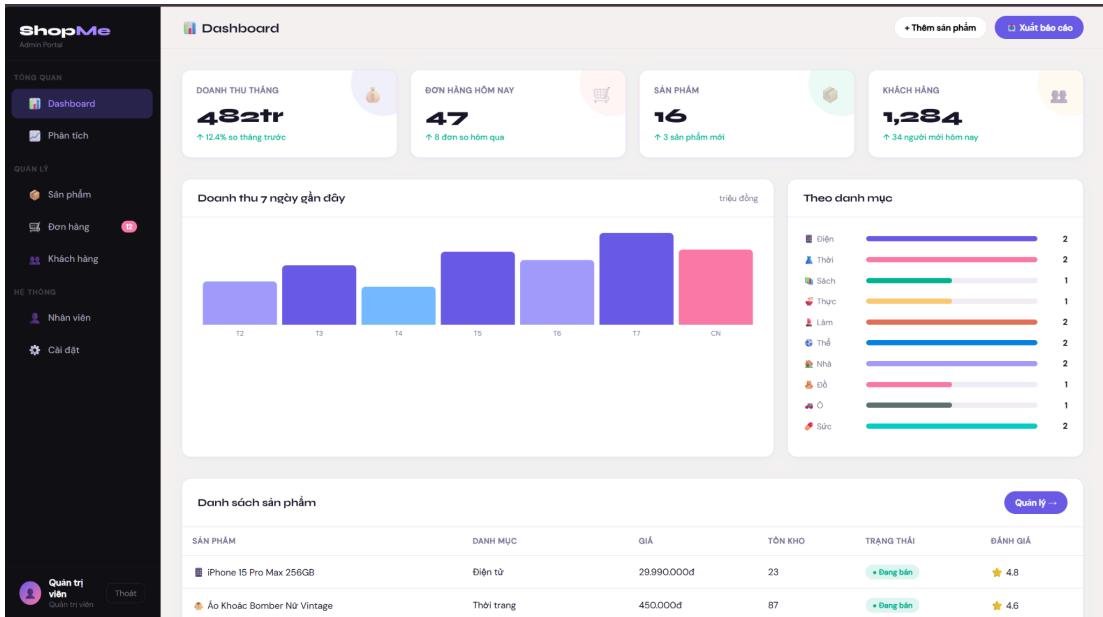
Áo thun giá rẻ

Sách hay nên đọc

Vitamin sức khỏe

Hỏi AI về sản phẩm...

Giao diện quản lý/dashboard:



Danh sách sản phẩm

Quản lý -->

SẢN PHẨM	DANH MỤC	GIÁ	TỒN KHO	TRẠNG THÁI	ĐÁNH GIÁ
iPhone 15 Pro Max 256GB	Điện tử	29.990.000đ	23	Đang bán	4.8
Áo Khoác Bomber Nữ Vintage	Thời trang	450.000đ	87	Đang bán	4.6
Đắc Nhân Tâm – Dale Carnegie	Sách & Văn phòng	89.000đ	152	Đang bán	4.9
Mì Hào Hào Combo 30 Gói	Thực phẩm	95.000đ	340	Đang bán	4.5
Son Môi Velvet Rouge #07	Làm đẹp	380.000đ	64	Đang bán	4.7
Giày Chạy Bộ Nike Air Zoom	Thế thao	2.590.000đ	31	Đang bán	4.8
Nồi Chiên Không Dầu 5L	Nhà cửa	1.290.000đ	19	Đang bán	4.6
LEGO Technic Siêu Xe 42143	Đồ chơi	3.490.000đ	8	Đang bán	4.9

Đơn hàng gần đây

Xem tất cả -->

MÃ ĐƠN	KHÁCH HÀNG	SẢN PHẨM	TỔNG TIỀN	TT THANH TOÁN	GIAO HÀNG
#SM240425	Nguyễn Văn A	iPhone 15 Pro Max x1	29.990.000đ	✓ Chuyển khoản	Đang giao
#SM240424	Trần Thị B	Son Môi x2, Kem x1	1.080.000đ	✓ MoMo	Đã giao
#SM240423	Lê Văn C	MacBook Air M3 x1	27.990.000đ	✓ Thanh toán	Đang xử lý
#SM240422	Phạm Thị D	LEGO Technic x1	3.490.000đ	✓ Tiền mặt	Đã giao
#SM240421	Hoàng Văn E	Vitamin C x3	555.000đ	✓ MoMo	Đã giao

4.8 Đánh giá hệ thống

4.8.1 Hiệu năng

- Response time trung bình: < 200ms cho product endpoints
- AI recommendation endpoint: < 500ms (cache hit) / < 2s (cold)
- Chatbot response (Ollama local): 2–8 giây tùy model và hardware
- Frontend load time: < 1 giây (static files qua Nginx)

4.8.2 Khả năng mở rộng

- Thêm danh mục sản phẩm mới: copy template product-service, đổi tên và cấu hình — không ảnh hưởng service nào
- Scale product service theo traffic: docker compose up --scale laptop-service=3
- AI model upgrade: chỉ cần update ai-service, restart container
- Thêm payment gateway mới: chỉ sửa payment-service

4.8.3 Ưu điểm

- Fault isolation hoàn toàn: 1 service crash không kéo theo service khác
- Team có thể phát triển độc lập từng service
- Database per service: không coupling dữ liệu
- AI service fallback nhiều tầng: Neo4j → MongoDB → cosine similarity → random
- Frontend tự xử lý fallback khi API không khả dụng (FALLBACK_PRODUCTS hardcoded)

4.8.4 Nhược điểm và Hướng khắc phục

Nhược điểm	Hướng khắc phục
Race condition khi startup (service start trước DB)	Thêm retry loop và wait-for-db trong Dockerfile

Không có centralized logging	Tích hợp ELK Stack hoặc Loki + Grafana
Chatbot Ollama chạy chậm trên hardware yếu	Dùng OpenRouter API như provider cloud fallback
Không có service discovery động	Có thể nâng cấp lên Consul hoặc Kubernetes DNS
Frontend product data phụ thuộc AI service	Đã có FALLBACK_PRODUCTS hardcoded trong frontend

Bảng 4.2: Nhược điểm và Hướng khắc phục

4.9 Kết luận Chương 4

ShopMe được xây dựng thành công theo kiến trúc Microservices với đầy đủ các thành phần: API Gateway, 17 service nghiệp vụ, 6 database riêng biệt, và AI service tích hợp hybrid recommendation. Hệ thống có thể khởi động bằng một lệnh docker compose up --build duy nhất, với seed data tự động cho tất cả service. Kiến trúc này đáp ứng đầy đủ các yêu cầu về scalability, fault isolation, và maintainability theo đúng nguyên tắc Microservices và DDD.

Chương 5. Kết luận

Tiểu luận này đã trình bày quá trình phân tích, thiết kế và xây dựng hệ thống thương mại điện tử ShopMe theo kiến trúc Microservices kết hợp Domain-Driven Design và AI service.

Các kết quả đạt được:

1. Chương 1: Phân tích rõ ràng sự khác biệt giữa Monolithic và Microservices, áp dụng DDD để xác định 7 bounded context chính trong ShopMe.
2. Chương 2: Thiết kế và triển khai 17 microservice với database per service, luồng mua hàng end-to-end qua 5 service, JWT authentication.
3. Chương 3: Xây dựng AI service kết hợp LSTM + Knowledge Graph (Neo4j) + RAG (FAISS + Ollama LLM) cho gợi ý sản phẩm và chatbot tư vấn.
4. Chương 4: Containerize toàn hệ thống bằng Docker, orchestrate bằng Docker Compose, đánh giá ưu nhược điểm và hướng phát triển tiếp theo.

Hướng phát triển tiếp theo:

- Tích hợp message queue (Redis/RabbitMQ) cho asynchronous communication
- Triển khai Kubernetes cho production environment
- Thêm centralized logging với ELK Stack
- Implement Circuit Breaker pattern cho giao tiếp giữa service
- Fine-tune LSTM model với nhiều dữ liệu hành vi hơn
- Thêm A/B testing cho recommendation engine

ShopMe là minh chứng thực tế rằng kiến trúc Microservices, khi kết hợp đúng với DDD và AI, có thể tạo ra hệ thống thương mại điện tử hiện đại, linh hoạt, dễ mở rộng và cung cấp trải nghiệm mua sắm được cá nhân hóa cho người dùng.

TÀI LIỆU THAM KHẢO. Tài liệu tham khảo

5. Thomas Erl, "Service-Oriented Architecture: Analysis and Design for Services and Microservices", 2nd Edition, Prentice Hall, 2017.
 6. Chris Richardson, "Microservices Patterns: With Examples in Java", Manning Publications, 2018.
 7. Eric Evans, "Domain-Driven Design: Tackling Complexity in the Heart of Software", Addison-Wesley, 2003.
 8. Hung Dang, "Bài tập — Phát triển phần mềm hướng dịch vụ", Tài liệu nội bộ môn SoAD.
 9. Trần Đình Quế, "Tiểu luận SoAD: Xây dựng Hệ thống E-Commerce theo Microservices và AI", 2026.
 10. Docker Inc., "Docker Compose documentation", <https://docs.docker.com/compose/>, 2024.
 11. FastAPI, "FastAPI documentation", <https://fastapi.tiangolo.com/>, 2024.
 12. Neo4j Inc., "Neo4j Graph Database documentation", <https://neo4j.com/docs/>, 2024.
 13. Django Software Foundation, "Django REST Framework documentation", <https://www.django-rest-framework.org/>, 2024.
- Ollama, "Ollama — Run large language models locally", <https://ollama.com/>, 202